Amaliy kriptografiya

Tinglovchi: Isakov M. M.

Mavzu №1-2: Maxfiy aloqani tashkil qilish protokollari.

Ta'rif: Biror amaliy masalani yechish uchun 2 va undan ortiq ishtirok etuvchi tomonlar qabul qilgan vazifalar ketma-ketligi protokol d-di.

Kriptografik protokollar yonalishi kriptografiya yonalishlariga mos ravishda quyidagicha:

- foydalanuvchilar ortasida identifikatsiyani ta'minlash protokollari.
- generatsiya qilingan kalitni uzatish protokollari;
- seans kalitni birgalikda generatsiya qilishni ta'minlovchi protokollari turlarida bolishi mumkin.

Identifikatsiya protokollarining asosiy vazifalari qatoriga turli resurs-

larga nazorat shaklida ruxsat berish kiradi.

Barcha identifikatsiya protokollarida 2 ta ishtirokchi bolib:

1. A-isbot qiluvchi, ya'ni identifikatsiyadan otuvchi tomon.

2. B-tekshiruvchi tomon, ya'ni isbot qiluvchi autentifikatsiyani tekshiruvchi tomon.

Umumiy holda identifikatsiya protokollari qaysi maqsadlarga qaratilishiga qarab quyidagi 3 ta kategoriyaga bolinadi.

1. Ikkala tomonga ham ma'lum bolgan ma'lumotlarga asoslangan protokollar. Ushbu tomonlarga ma'lum ma'lumotlar sifatida parollar, shaxsiy identifikatsion raqamlar, maxfiy yoki ochiq kalitlardan protokol qadam-

lari davomida foydalanishi mumkin.

2. Isbot qiluvchi tomon orqali fizik priborlardan foydalanib amalga oshiriluvchi protokollar. Bunda fizik priborlar sifatida magnit yoki intellektual plastik kartochkalari, vaqt o'zgarishi davomida mos holda parollar o'zgarishini ta'minlovchi generatsiya priborlari ishlatilishi mumkin.

3. Isbot qiluvchi tomon (ajralmas tashkil qiluvchisining) fizik parametrlaridan foydalanib amalga oshiriluvchi protokollar. Afsuski bu toifa protokollar kriptografik xususiyatlarga ega ~~emas~~ bolmaganligi uchun ular haqida keyinchalik alohida to'xtab o'tilmaydi.

Protokollarda tasodifiy son va vaqtga bog'liq parametrlardan foydalanish

Umumiy holda protokollarda vaqtga bog'liq 3 ta parametrdan foydalaniladi.

- tasodifiy sonlar.
- ketma-ket nomerlar.
- vaqt metkalar.

Vaqt metkasi yordamida bir tomonlama identifikatsiyani ta'minlash.

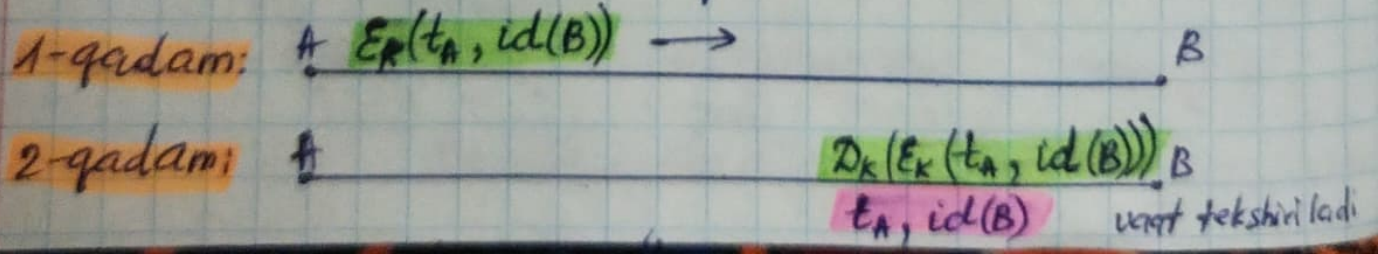
t_A – A foydalanuvchining vaqt metkasi.

id(B) – B foydalanuvchining identifikatori (identifikatsion raqami)

K – A va B foydalanuvchilar uchun ma'lum bolgan umumiy kalit.

E – shifrlash algoritmi

D – deshifrlash algoritmi.



Tasodifiy sondan foydalanib bir tomonlama identifikatsiyani ta'minlash

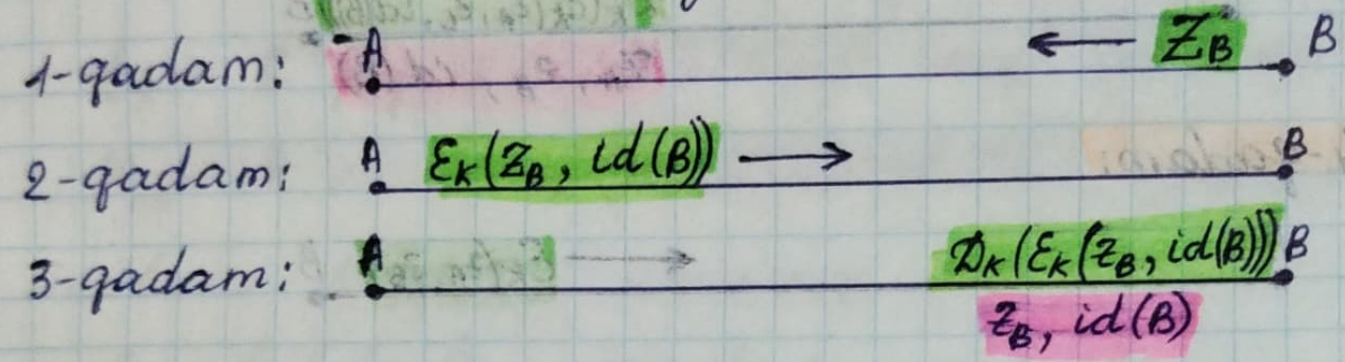
z_B – B foydalanuvchining tasodifiy soni.

$id(B)$ – B foydalanuvchining identifikatori.

K – A va B foydalanuvchilar uchun ma'lum bolgan umumiy kalit.

$\mathcal{E}$ – shifrlash algoritmi

$\mathcal{D}$ – deshifrlash algoritmi.



Tasodifiy sondan foydalanib ozaro identifikatsiyani ta'minlash.

z_A, z_B – mos holda A va B foydalanuvchilarning tasodifiy sonlari.

$id(B)$ – B foydalanuvchining identifikatori

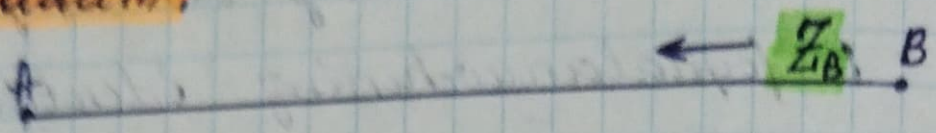
K – A va B foydalanuvchilar uchun ma'lum bolgan umumiy kalit.

8

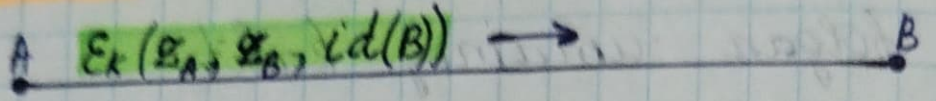
$\mathcal{E}$ – shifrlash algoritmi

$\mathcal{D}$ – deshifrlash algoritmi.

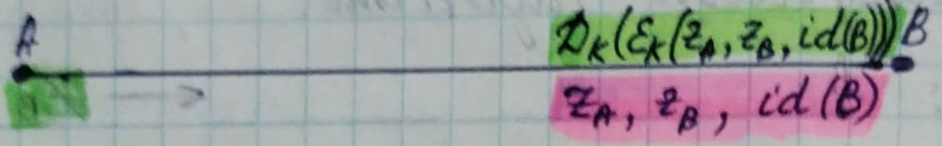
1-qadam:



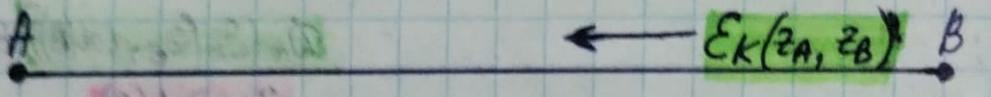
2-qadam:



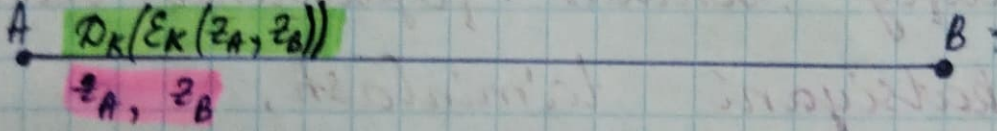
3-qadam:



4-qadam:



5-qadam:



01.03.2024.

Mavzu: Generatsiya qilingan kalitlarni uzatish protokollari.

Generatsiya qilingan kalitni vaqt metkasidan foydalanib, almashish protokoli.

B – B foydalanuvchining identifikatori.

k – ~~k~~ generatsiya qilingan kalit.

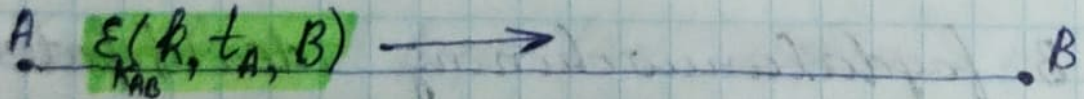
t_A – A foydalanuvchining vaqt metkasi.

K_{AB} – A va B foydalanuvchilar uchun ma'lum bolgan umumiy kalit.

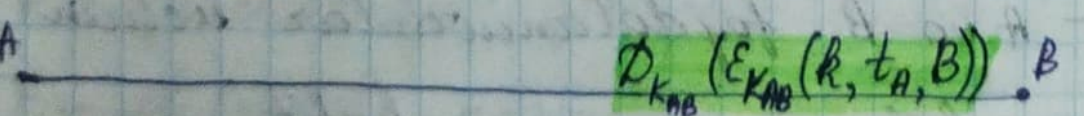
$\mathcal{E}$ – shifrlash algoritmi.

$\mathcal{D}$ – deshifrlash algoritmi.

1-qadam:



2-qadam:



Hujum.

K_{AB} 3-shaxsga ma'lum emas, demak u shifrmatnni ocha olmaydi. U faqat oldin olib qoyilgan shifrmatnni olib, joriy jonatilgan shifrmatn oruniga jonatish orqali vaqt metkasini o'zgartirish mumkin ($t_A \sim t_{A1}$, $C \sim C_1$). Agar C_1 ham K_{AB} da shifrlangan bolsa ham vaqt metkasi (t_{A1}) ancha oldin bolganligi uchun B abonent k kalitni qabul qilmaydi.

Generatsiya qilingan kalitni vaqt metkasi va hesh funksiyasidan foydalanib almashish protokoli.

B - B foydalanuvchining identifikatori

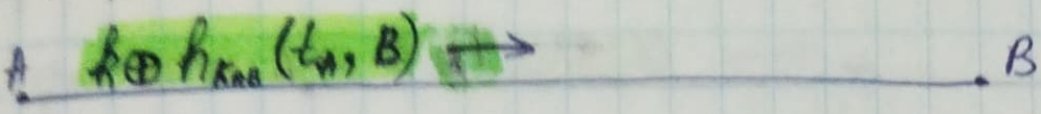
t_A - A foydalanuvchining vaqt metkasi

K_{AB} - A va B foydalanuvchilar uchun ma'lum bolgan umumiy kalit.

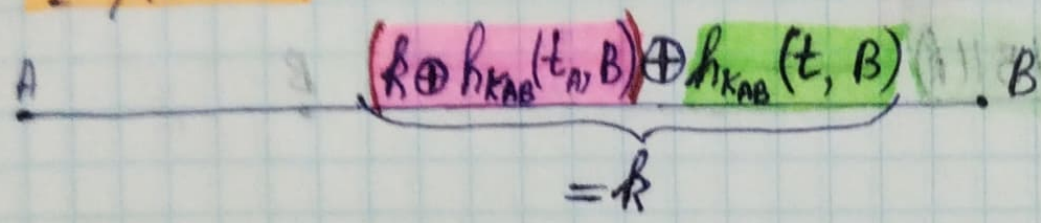
k - generatsiya qilingan kalit

h - hesh funksiya.

1-qadam:



2-qadam:



faqat ikkala holatda ham vaqt metkasini bir xil olish talab etiladi. Aks holda heshlar 2 xil chiqadi. ($t_A = t$)

Generatsiya qilingan kalitni tasodifiy sondan foydalanib kalit almashish protokoli.

B – B foydalanuvchining identifikatori.

r_B – B foydalanuvchining tasodifiy soni.

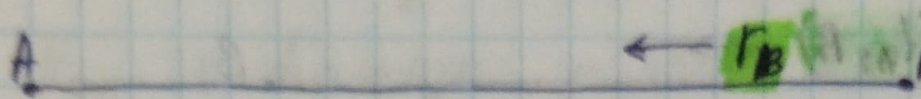
K_{AB} – A va B foyd. lar uchun ma'lum bolgan umumiy kalit.

k – generatsiya qilingan kalit.

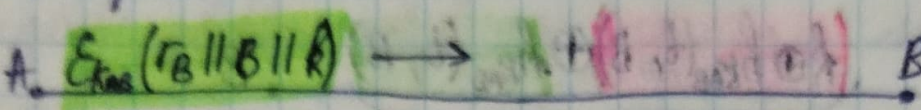
$\mathcal{E}$ – shifrlash algoritmi

$\mathcal{D}$ – deshifrlash algoritmi.

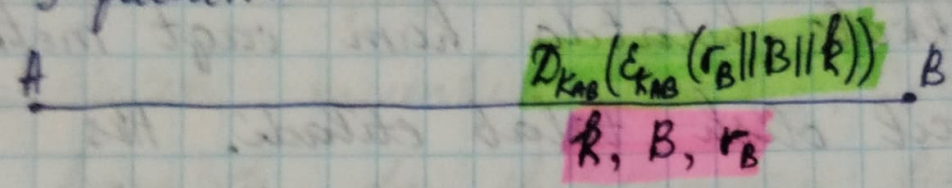
1-qadam.



2-qadam



3-qadam



Generatsiya qilingan kalitni tasodifiy son va xesh funksiyalardan foydalanib almashish protokoli.

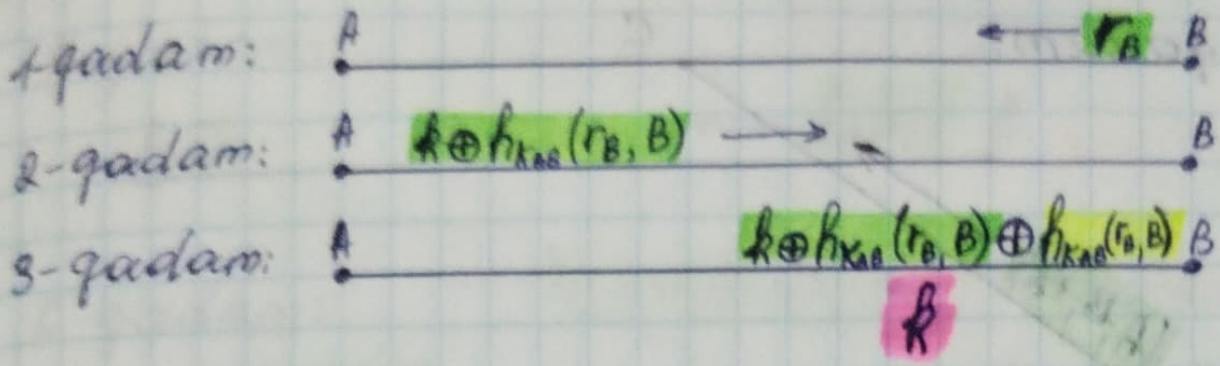
B – B foydalanuvchining identifikatori

r_B – B foyd. tasodifiy soni.

K_{AB} – A va B foydalanuvchilar uchun ma'lum bolgan umumiy kalit.

k – generatsiya qilingan va uzatilishi kerak bolgan kalit.

h – hesh funksiyasi



Arbitr ishtirokida kalitlarni almashish protokoli.

k - generatsiya qilingan va uzatilishi kerak bolgan kalit.

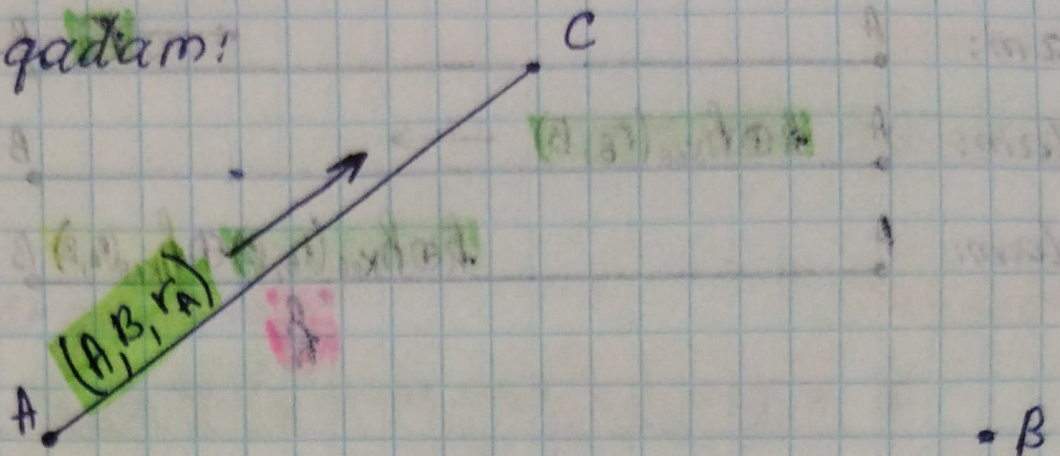
r_A, r_B - A va B foyd. larning tasodifiy sonlari.

k_{AT} - A foydalanuvchi va arbitrning umumiy kalitlari.

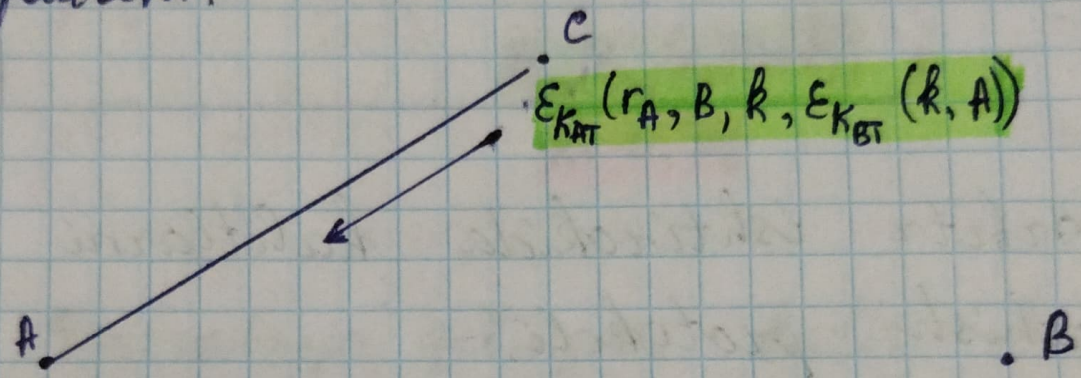
k_{BT} - B foydalanuvchi

14

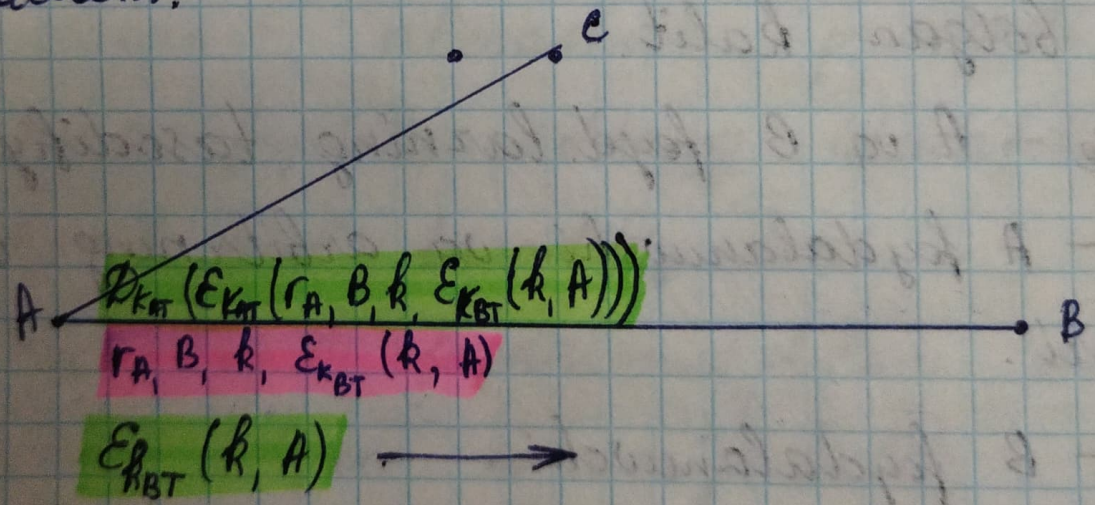
1-qadam:



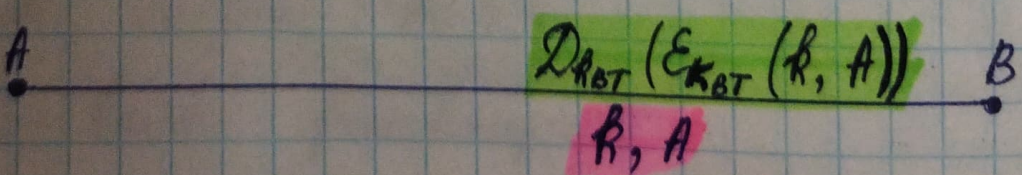
2-qadam:



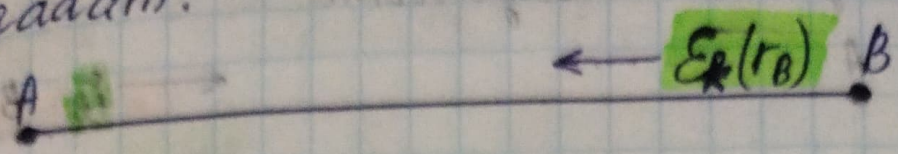
3-qadam:



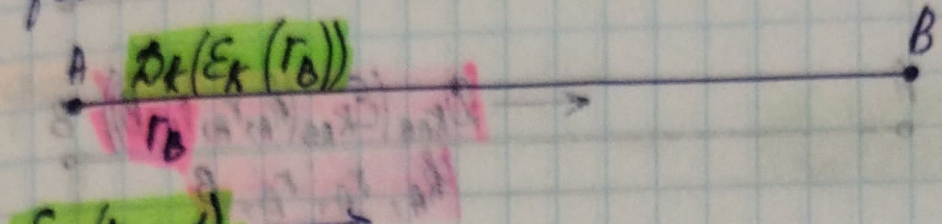
4-qadam:



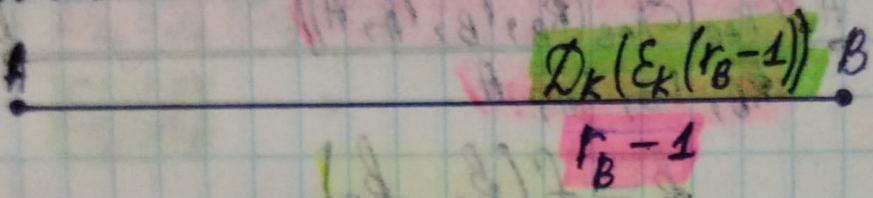
5-qadam:



6-qadam:



7-qadam:



Tasodifiy sondan foydalanib kalitni generatsiya qilish va ozaro almashish protokoli.

A, B – A va B foydalanuvchilarning identifikatorlari.

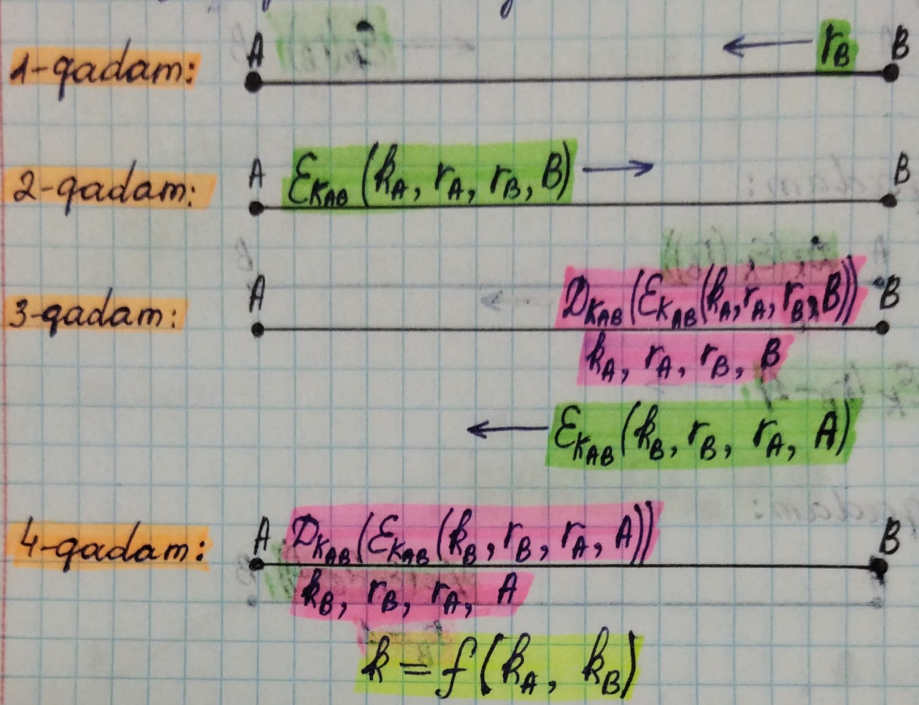
r_A, r_B – mos holda A va B foydalanuvchilarning tasodifiy sonlari.

k_A, k_B – A va B foydalanuvchilarning tasodifiy sonlari.

K_{AB} – A va B foydalanuvchilar uchun ma'lum bolgan umumiy kalit.

E – shifrlash algoritmi

$\mathcal{D}$ – deshifrlash algoritmi.



Umumiy maxfiy ma'lumotsiz kalitni generatsiya qilish va ozaro almashish protokoli (Shamir protokoli)

k – generatsiya qilingan va uzatilishi kerak bolgan kalit.

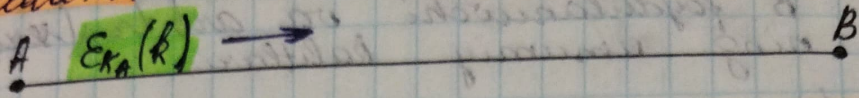
k_A, k_B – mos holda A va B foyd. larning kalitlari.

$\mathcal{E}$ – shifrlash algoritmi.

$\mathcal{D}$ – deshifrlash algoritmi.

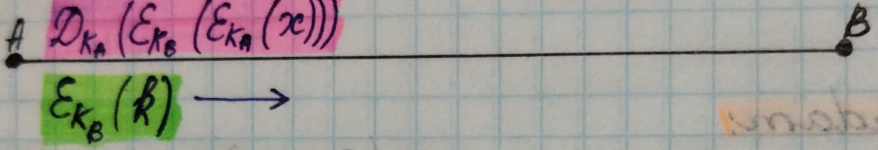
$$\left[\mathcal{E}_{K_1}\left(\mathcal{E}_{K_2}(x)\right) = \mathcal{E}_{K_2}\left(\mathcal{E}_{K_1}(x)\right)\right]$$

1-qadam:

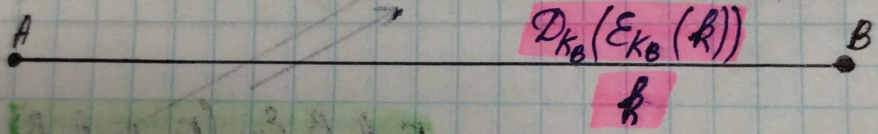


2-qadam:

3-qadam:



4-qadam:



$$\left[\mathcal{E}_{K_B}\left(\mathcal{E}_{K_A}(k)\right) = k^{ab} \bmod p = k^{ba} \bmod p = \mathcal{E}_{K_A}\left(\mathcal{E}_{K_B}(k)\right)\right].$$

Server ishtirokida kalitni almashish protokoli.

k – generatsiya qilingan va uzatilishi kerak bolgan kalit.

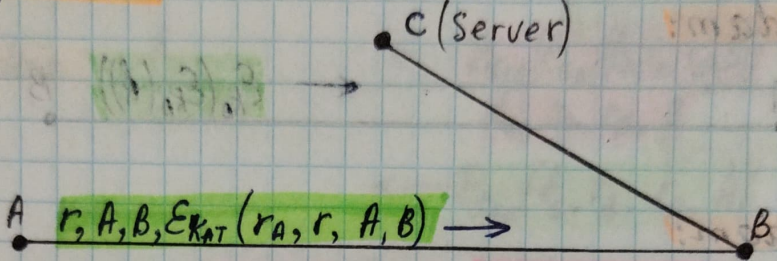
r_A, r_B – mos holda A va B foyd. larning tasodifiy sonlari.

r – tasodifiy son. (seans aloqasini autentifikatsiya qilish uchun).

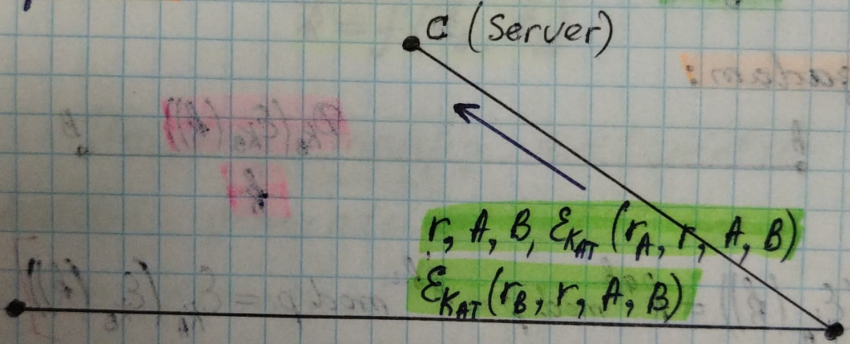
K_{AT} – A foydalanuvchi va arbitr (server)-ning umumiy kalitlari.

K_{BT} – B foydalanuvchi va arbitr (server)-ning umumiy kalitlari.

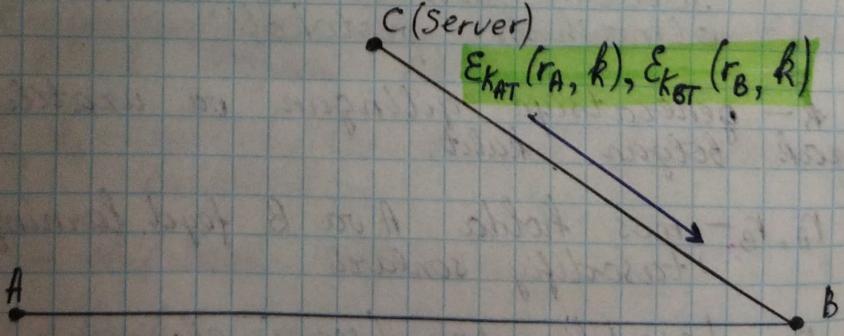
1-qadam:



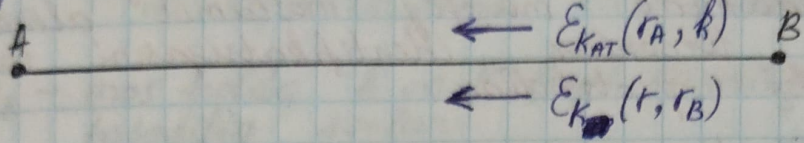
2-qadam:



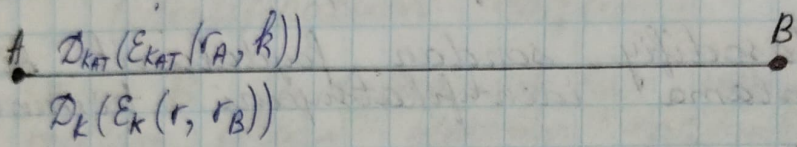
3-qadam:



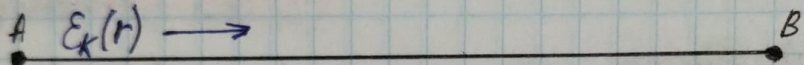
4-qadam:



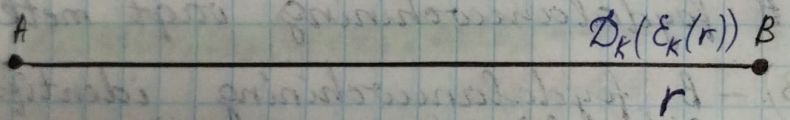
5-qadam:



6-qadam:



7-qadam:



Asimmetrik shifrlash algoritmi yordamida maxfiy ma'lumot almashuvchi tomonlar identifikatsiyasini ta'minlovchi protokollar.

1. Vaqt metkasi yordamida bir tomonlama identifikatsiyani ta'minlash.

2. Tasodifiy sondan foydalanib bir tomonlama identifikatsiyani ta'minlash.

3. Tasodifiy sondan foydalanib ozaro identifikatsiyani ta'minlash.

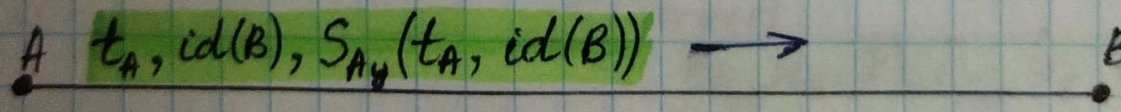
Vaqt metkasi yordamida bir tomonlama identifikatsiyani ta'minlash.

t_A – A foydalanuvchining vaqt metkasi

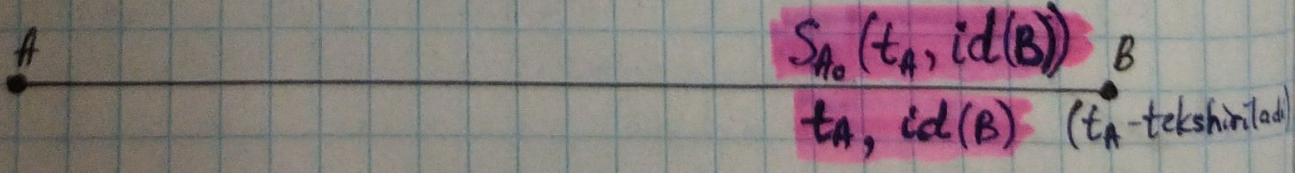
id(B) – B foydalanuvchining identifikatori (identifikatsion raqami)

S – A va B foydalanuvchilar uchun ma'lum bolgan elektron raqamli imzo algoritmi.

1-qadam:



2-qadam:



S_{Ay} – yopiq kalit bilan (A ning) imzo qoyish

S_{Ao} – ochiq kallt bilan imzoni tekshirish.

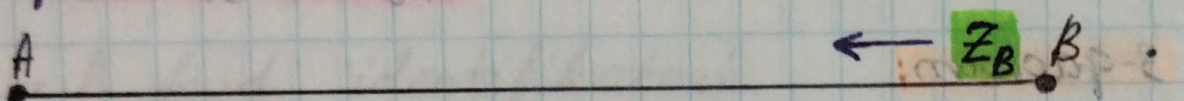
Tasodifiy sondan foydalanib bir tomonlama identifikatsiyani ta'minlash

z_A, z_B – mos holda A va B foyd. larning tasodifiy sonlar.

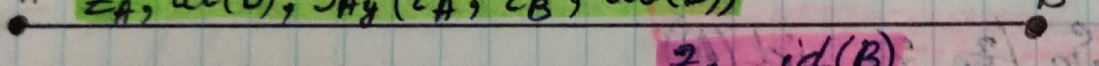
$id(B)$ – B foydalanuvchining identifikatori (identifikatsion raqami).

S – A va B foydalanuvchilar uchun ma'lum bolgan elektron raqamli imzo algoritmi.

1-qadam:



2-qadam:



$z_A, id(B)$

$S_{Ao}(z_A, z_B, id(B))$

$z_A, z_B, id(B)$

Tasodifiy sondan foydalanib ozaro identifikatsiyani ta'minlash.

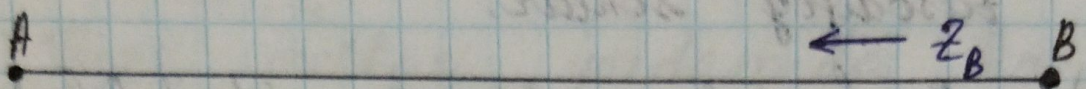
z_A, z_B – mos holda A va B foyd. larning tasodifiy sonlari.

$id(A), id(B)$ – A va B foyd. larning identifikatorlari (identifikatsion raqamlari)

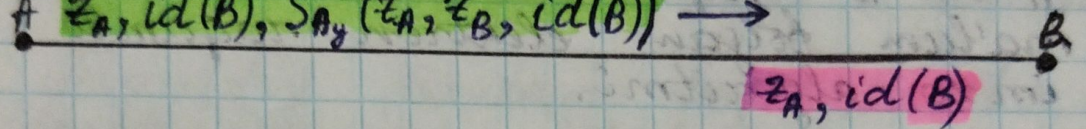
S – A va B foydalanuvchilar uchun

ma'lum bo'lgan elektron raqamli imzo algoritmi.

1-qadam:

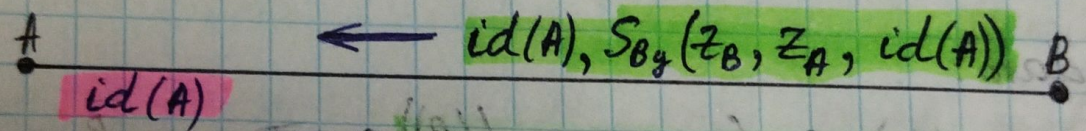


2-qadam:



$S_{A_o}(z_A, z_B, id(B))$

3-qadam:



$S_{B_o}(z_B, z_A, id(A))$

Generatsiya qilingan kalitni vaqt metkasidan foydalanib kalit almashish protokoli.

A – A foydalanuvchining identifikatori

k – generatsiya qilingan kalit.

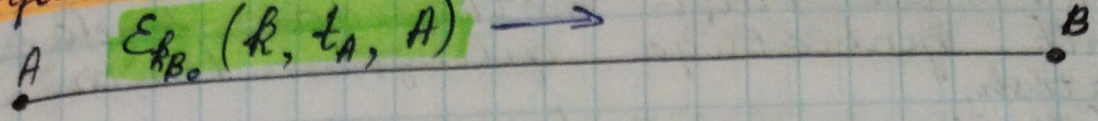
t_A – A foydalanuvchining vaqt metkasi

k_{B_o}; k_{B_y} – B foydalanuvchining ochiq va yopiq kaliti.

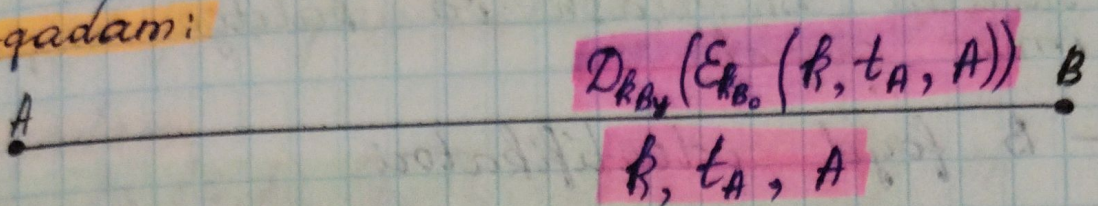
$\mathcal{E}$ – shifrlash algoritmi

$\mathcal{D}$ – deshifrlash algoritmi.

1-qadam:



2-qadam:



ERI va vaqt metkasidan foydalanib generatsiya qilingan kalitni uzatish, (raqamli imzo qoyilgan ma'lumotni shifrlash).

B – B foyd. identifikatori.

k – generatsiya qilingan kalit.

t_A – A foydalanuvchining vaqt metkasi.

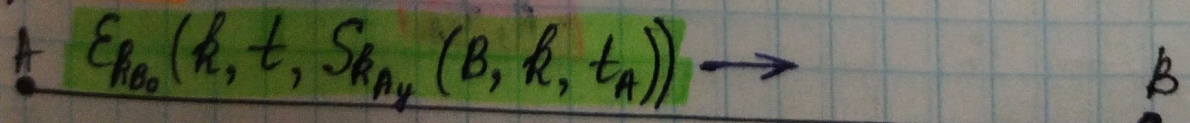
k_{Bo}, k_{By} – B foydalanuvchining ochiq va yopiq kaliti.

k_{Ao}, k_{Ay} – A foyd. ochiq va yopiq kaliti

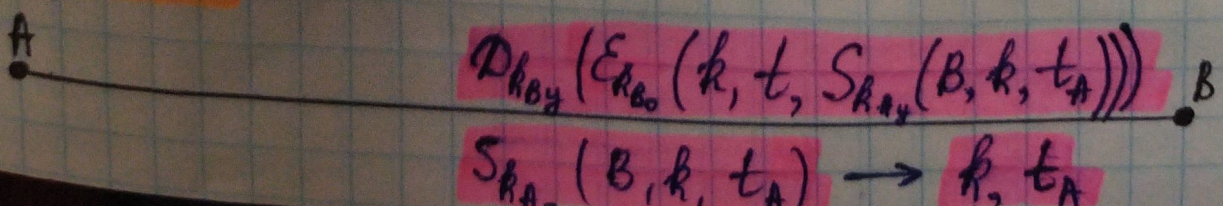
E – shifrlash algoritmi

D – deshifrlash algoritmi.

1-qadam:



2-qadam:



ERI va vaqt metkasidan foydalanib generatsiya qilingan kalitni uzatish.
(Kalitni shifrlash va kalitga raqamli imzo qoyish)

B – B foyd. identifikatori

k – generatsiya qilingan kalit

t_A – A foydalanuvchining vaqt metkasi.

k_{Bo}, k_{By} – B foyd. ochiq va yopiq kaliti.

k_{Ao}, k_{Ay} – A. foyd. ochiq va yopiq kaliti.

E – shifrlash algoritmi

D – deshifrlash algoritmi.

1-qadam:

A $E_{k_{Bo}}(k, t)$, $S_{k_{Ay}}(B, k, t_A)$ → B

$D_{k_{By}}(E_{k_{Bo}}(k, t_A))$

k, t_A

$S_{k_{Ao}}(B, k, t_A)$

k, t_A

ERI va vaqt metkasidan foydalanib generatsiya qilingan kalitni uzatish.

(Shifrlangan kalitga ERI qo'yish)

1-qadam.

$t_A, E_{K_{B_0}}(A,k), S_{K_{A_y}}(B, t_A, E_{K_{B_0}}(A,k)) \rightarrow$ B

A

$D_{K_{B_y}}(E_{K_{B_0}}(A,k)) \rightarrow A, k$

$S_{K_{A_0}}(B, t_A, E_{K_{B_y}}(A,k)) \rightarrow k$

Kalitlarni tarqatishning Diffi-Xellman protokoli

Kalitlarni tarqatishning STS protokoli

p, α – A va B foyd. larga ma'lum bolgan katta tub son.

x – A foyd. tasodifiy soni: $1 \leq x \leq p-2$.

y – B foyd. tasodifiy soni: $1 \leq y \leq p-2$.

k – Generatsiya qilinishi mumkin bolgan kalit.

S – ERI algoritmi

E, D – Asimmetrik shifrlash va deshifrlash algoritmi.

1-qadam:

A $z_1 = \alpha^x \bmod p$ → B

$k = z_1^y = \alpha^{xy} \bmod p$

2-qadam:

A ← $\alpha^y \bmod p,\ E_k(S_B(\alpha^y, \alpha^x))$ B

$k = z_2^x = \alpha^{yx} \bmod p$

$D_k(E_k(S_B(\alpha^y, \alpha^x))) \to S_B(\alpha^y, \alpha^x)$

3-qadam:

A $E_k(S_A(\alpha^x, \alpha^y))$ → B

$D_k(E_k(S_A(\alpha^x, \alpha^y))) \to S_A(\alpha^x, \alpha^y)$

DASS protokoli.

k_{A_o}, k_{A_y} – A foyd. ochiq va yopiq kalitlari.

k_{B_o}, k_{B_y} – B foyd. ochiq va yopiq kalitlari

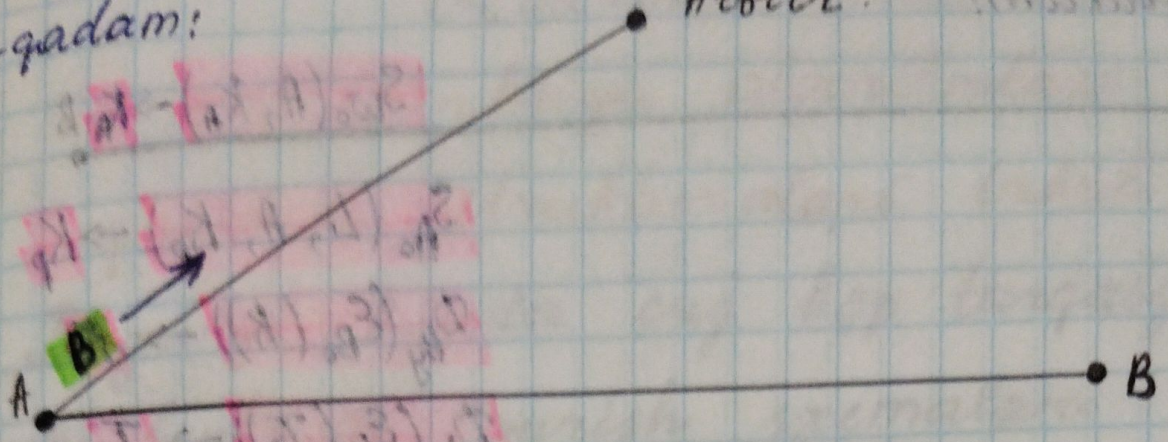
k_{W_o}, k_{W_y} – Arbitrning ochiq va yopiq kalitlari

S – ERI algoritmi

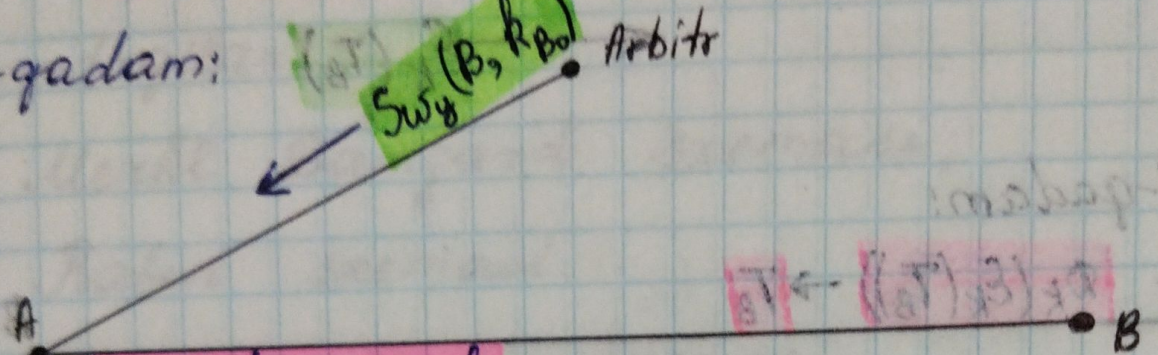
L – K generatsiya qilingan kalitning haqiqiylik vaqti.

K_p – Tasodifiy ochiq-yopiq juftlik kalit.

1-qadam:



2-qadam:



$S_{W_0}(B, k_{B_0}) \rightarrow B, k_{B_0}$

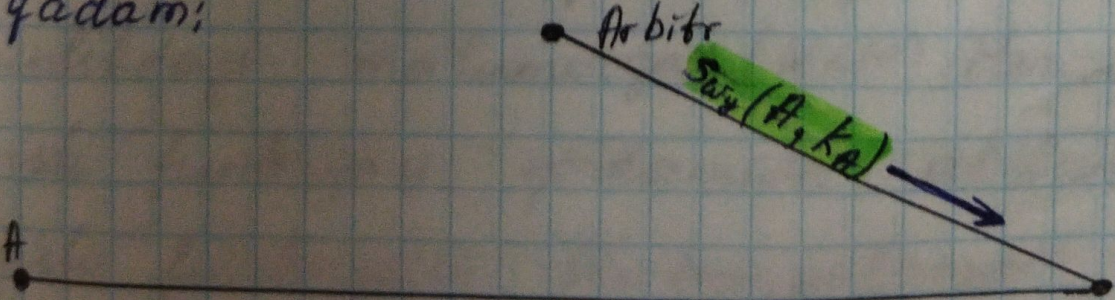
K, T_A, K_p

$\mathcal{E}_K(T_A), S_{A_y}(L, A, K_p), \longrightarrow$

$S_{K_p}(\mathcal{E}_{B_0}(K))$

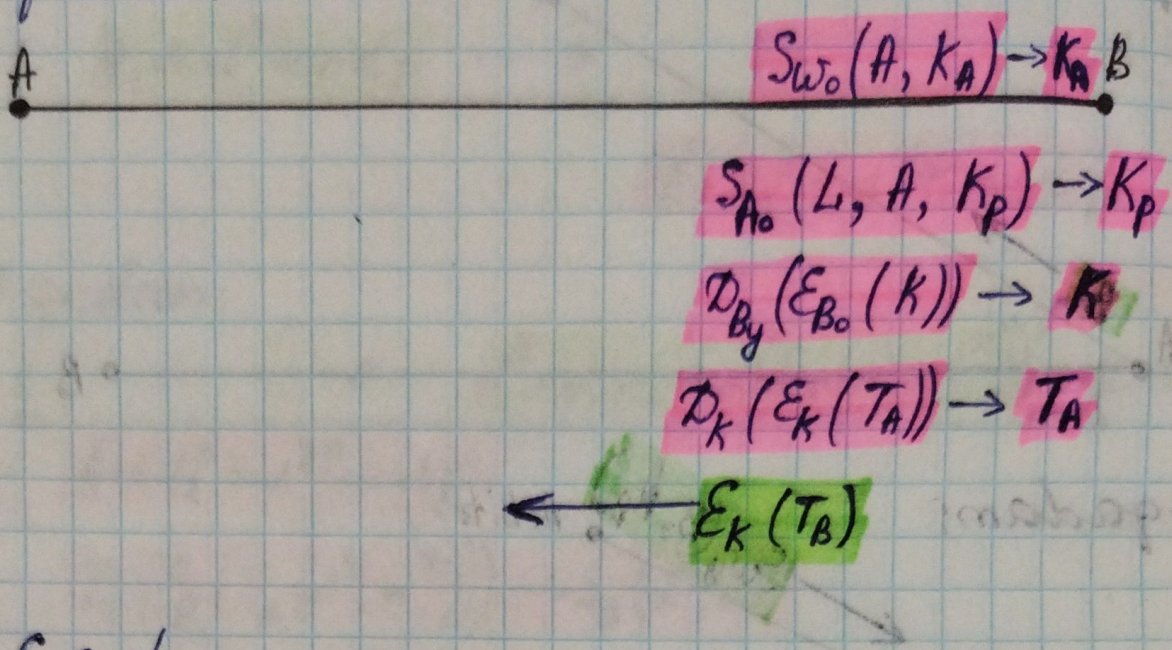
3-qadam:

4-qadam:

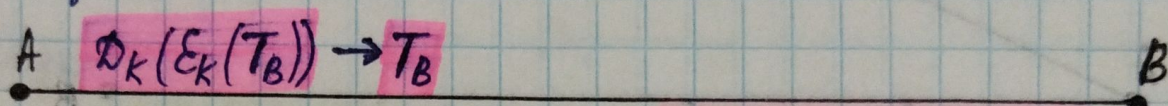


28

5-qadam:



6-qadam:



Xesh funksiya algoritmlari qurilish strukturalari tahlili

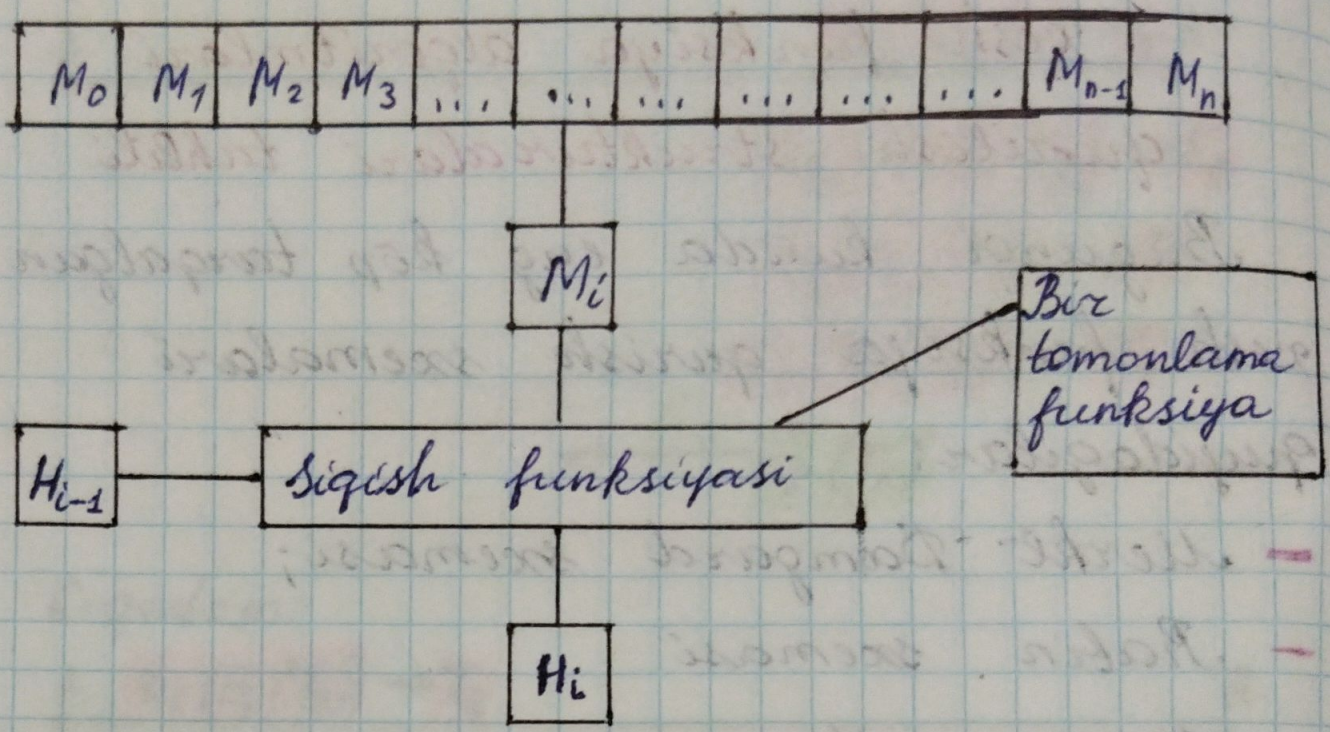
Bugungi kunda eng kop tarqalgan xesh funksiya qurish sxemalari quyidagilar:

- Merkl Damgard sxemasi;
- Rabin sxemasi
- Davies Mayer sxemasi.
- Matyas Mayer Oseas sxemasi
- Miaguchi Preinel sxemasi
- Keng kanalli Merkl Damgard sxemasi
- Gubka ("Sponge") prinsipi

Merkl Damgard sxemasi.

Ushbu sxema ixtiyoriy uzunlikdagi M ma'lumotni fiksirlangan N uzunlikdagi ma'lumotga (N<M) iterativ siqish funksiyasi yordamida akslantiradi. Siqish funksiyasi bir tomonlama funksiya yoki blokli shifrlash algoritmi

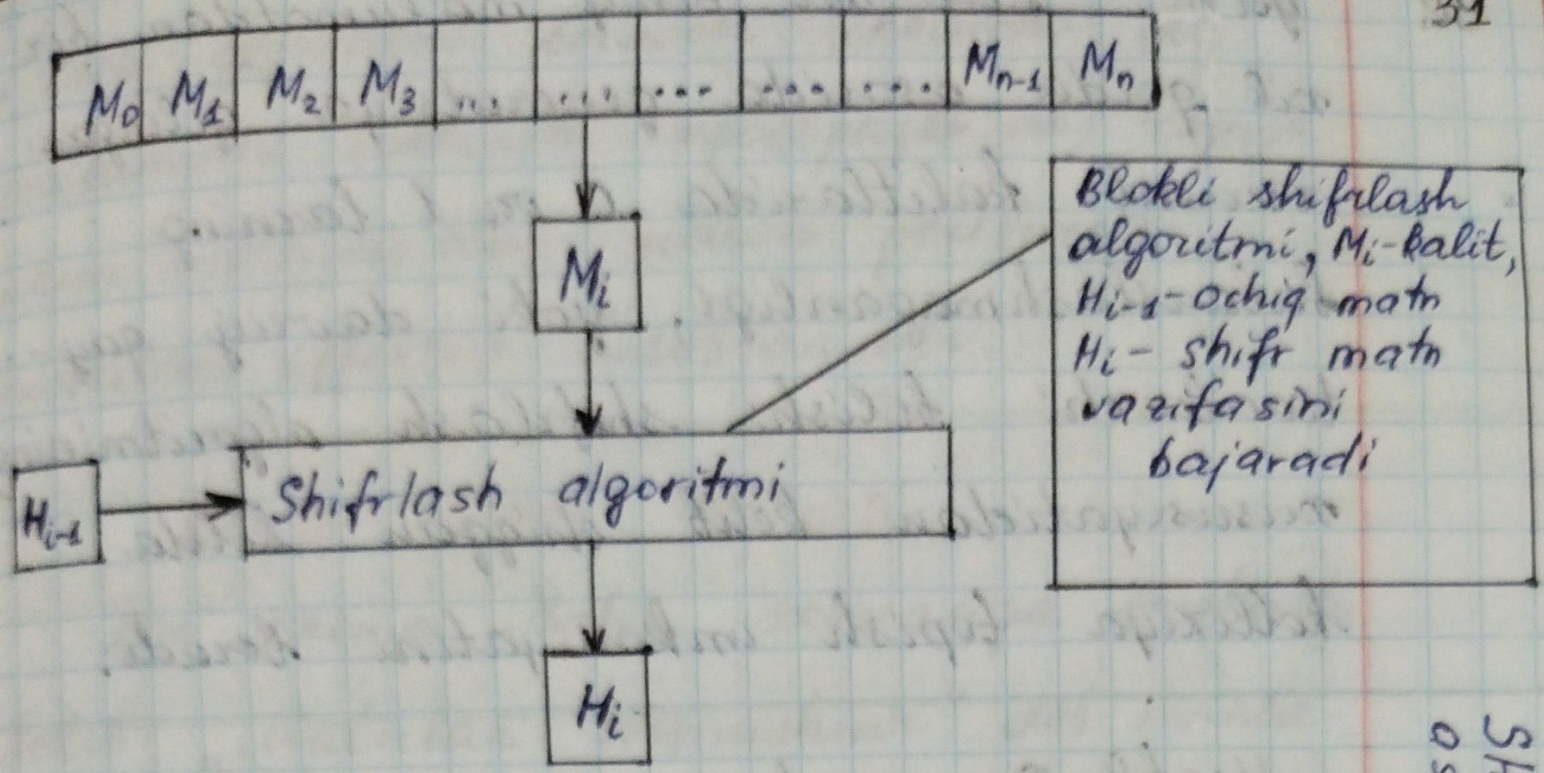
bolishi mumkin.



Rabin sxemasi

Xesh funksiya algoritmlari qurilishining Rabin sxemasi ixtiyoriy uzunlikdagi M ma'lumotni fiksirlangan N uzunlikdagi ma'lumotga. (N<M) blokli shifrlash algoritmi yordamida akslantiradi. Siqish funksiyasi vazifasini blokli shifrlash algoritmi bajaradi. Misol uchun: ГОСТ 34.11-94

Xesh funksiya algoritmining tezligi shifrlash algoritmining tezligiga bog'liq.



Sxemaning kamchiligi:

Ushbu usulda xesh qiymat uzunligi shifrlangan blok uzunligi bilan teng boladi. Masalan, siqish funksiyasi ornida DES shifrlash algoritmidan foydalanilsa, xesh qiymat 64 bit uzunlikka ega boladi. Ushbu xesh qiymatni „Matnlar toqnashuvi" deb nomlanuvchi kriptohujum usuli bilan kollixiyaga uchratish mumkin.

Ochiq ma'lumotdan generatsiya qilingan kalitlarning bardoshli bolmasligi,

SHA-3 (AQSh standarti) Gubka prinsipi asosida ishlab chiqilgan.

ya'ni har xil ochiq ma'lumotdan bir xil qoida asosida generatsiya qilingan raund kalitlarida .0 va 1 larning balanslashmaganligi, yoki davriy qaytariluvchi bolishi shifrlash algoritmining xususiyatidan kelib chiqqan holda. kolliziya topish imkoniyatini beradi.

Merkl-Damgard sxemasining ustunligi:

Bu sxemada agar f siqish funksiyasi kolliziyaga bardoshli bolsa, ushbu prinsip asosida qurilgan xesh funksiya algoritmlari ham kolliziyaga bardoshli boladi.

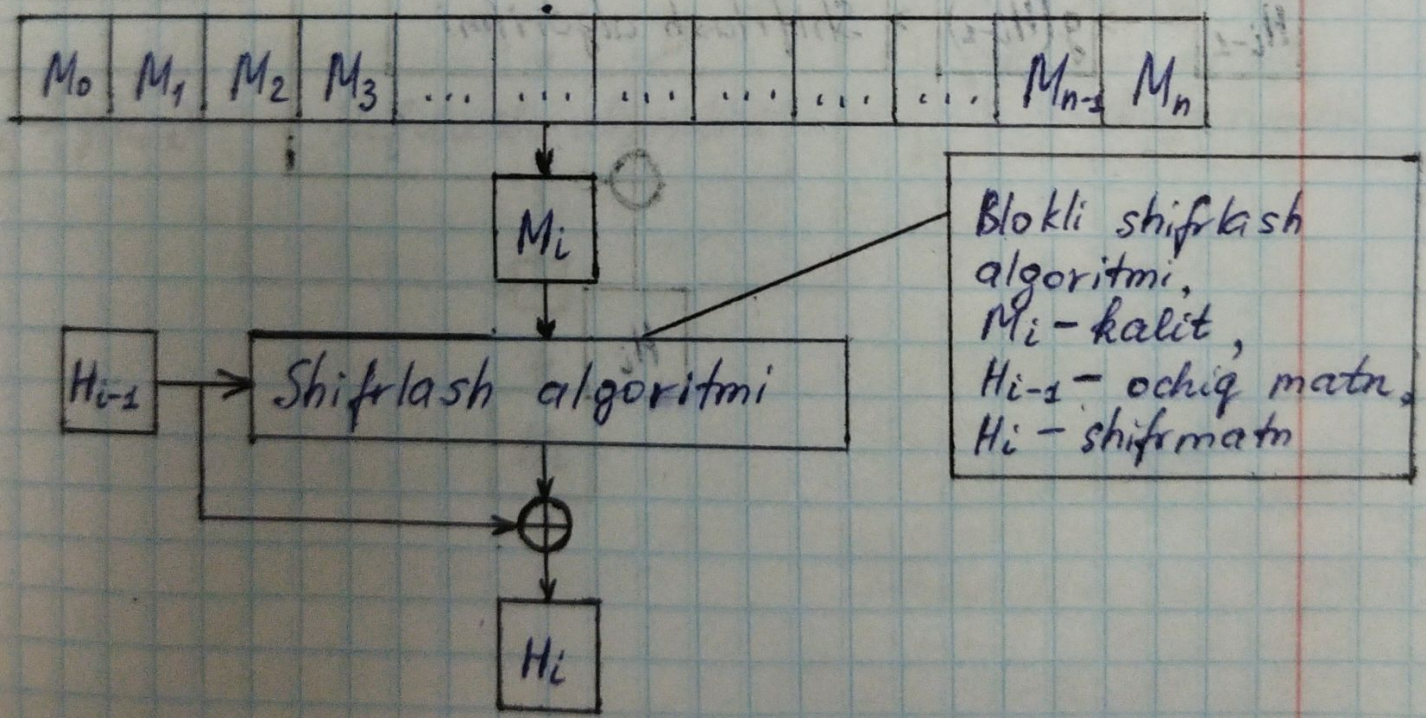
Merkl-Damgard sxemasining zaif tomonlari:

- Ma'lumotni kerakli uzunlikkacha toldirish, ya'ni agar kriptoanalitik bitta kolliziya topsa, boshqa kolliziyalarni topish oson.

- xesh qiymatni beruvchi ikkinchi bir ma'lumotni (ikkinchi прообраз-ni) topish, uzun ma'lumotlar uchun perebirga nisbatan effektiv hisoblanadi.

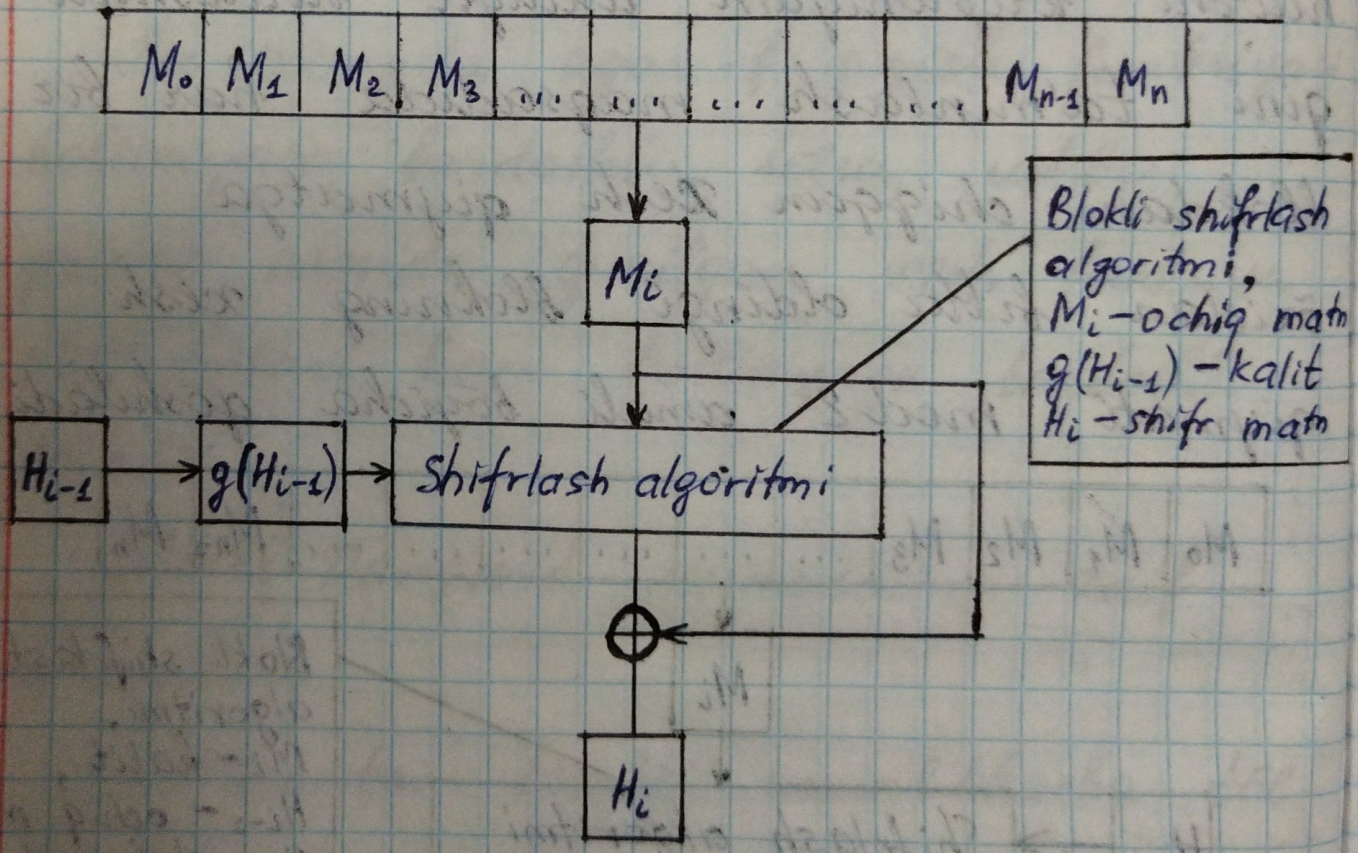
Devis-Meyer sxemasi.

Bu sxema Rabin sxemasiga o'xshash bo'lib, „Matnlar to'qnashuvi" deb nomlanuvchi kriptohujum usuliga bardoshliligini ta'minlash maqsadida har bir blokdan chiqqan xesh qiymatga o'zidan bitta oldingi blokning xesh qiymati mod 2 amali bo'yicha qo'shiladi.



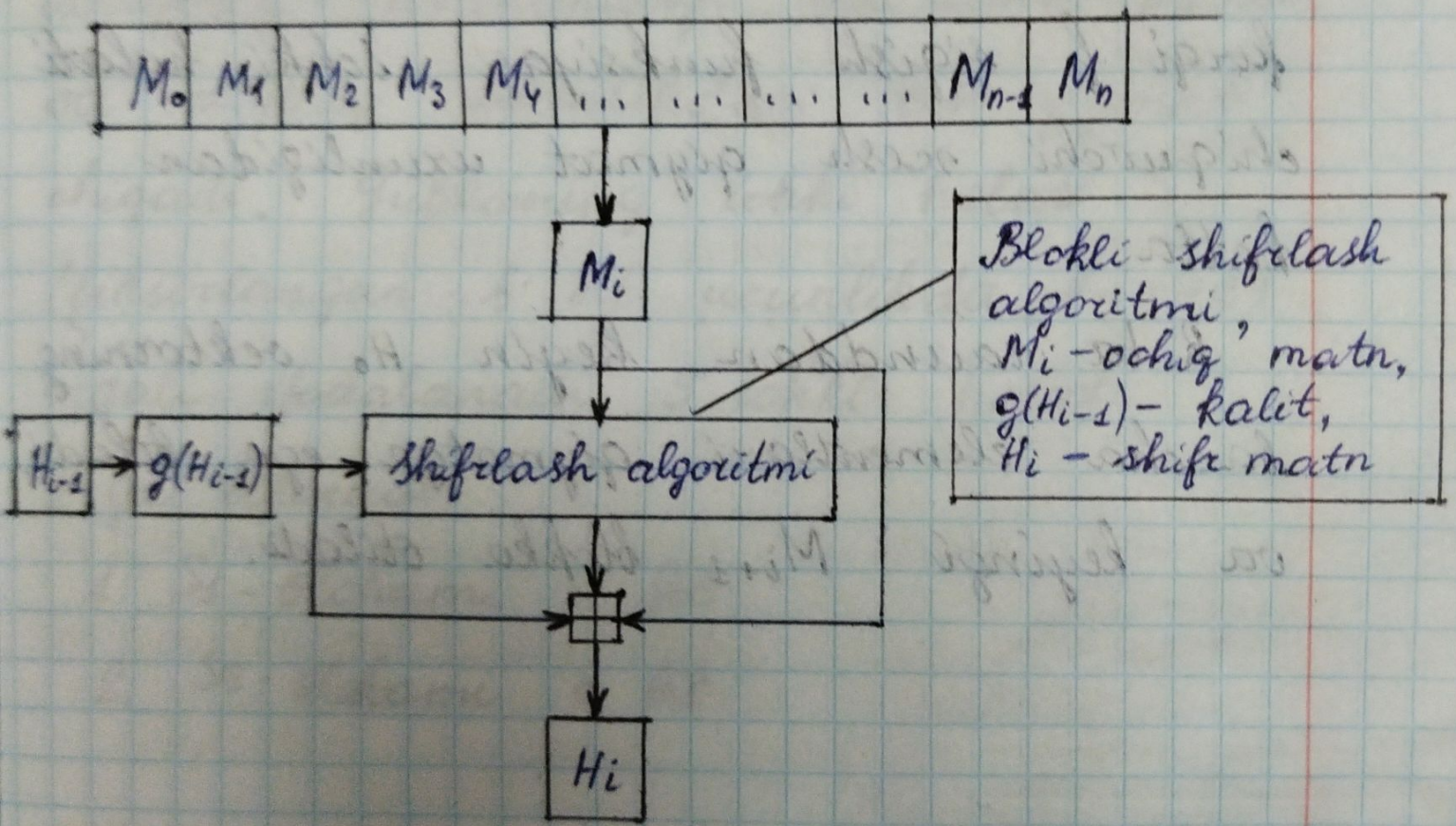
Matis – Meyer – Osease sxemasi

Matis – Meyer – Osease sxemasi Devis – Meyer sxemasiga o'xshash bo'lib, unda xeshlanadigan ma'lumot bloki ochiq matn vazifasini, initsializatsiya vektori H_0 g funksiyadan o'tkazilib, M_1 blokni shifrlashda kalit vazifasini bajaradi.



Miaguchi – Preneel sxemasi

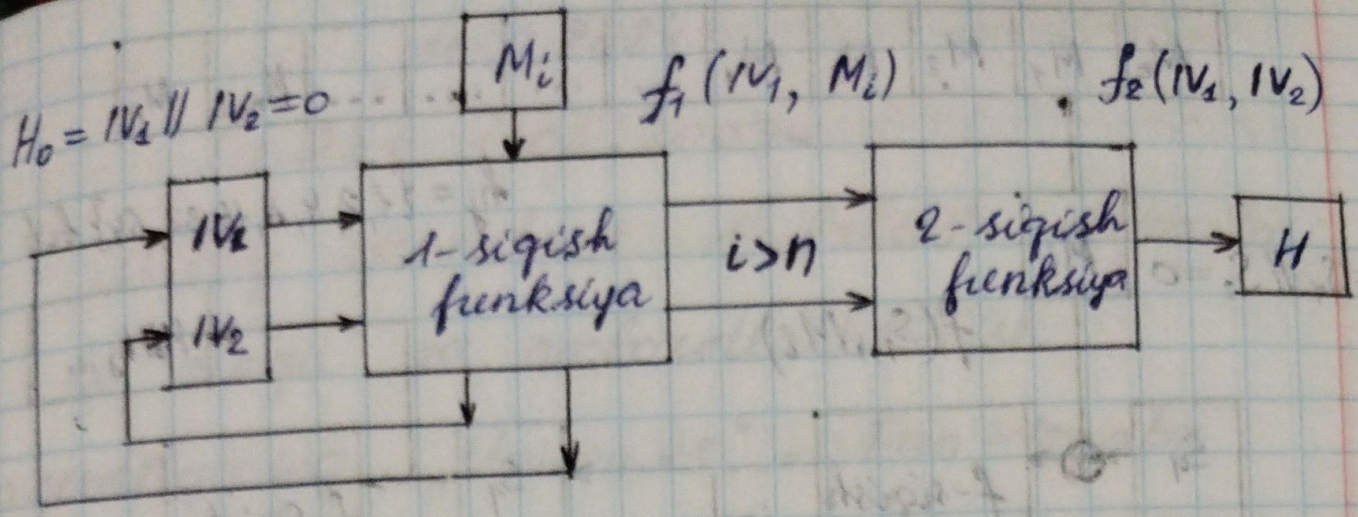
Matis – Meyer – Osease sxemasining kengaytirilgan varianti bolib, algoritmning kriptohujumlarga bardoshliligini oshirish maqsadida XOR amali orqa OR (yoki) amali ishlatiladi va har bir blokdan chiqqan xesh qiymatga undan bitta oldingi blokning xesh qiymati, mod 2^n amali boyicha qoshiladi.



Keng kanalli Merkl-Damgard sxemasi.

Ushbu sxema Markl-Damgard sxemasi kamchiliklarini bartaraf etish maqsadida, aynan ma'lumotni kerakli uzunlikka to'ldirish bilan bog'liq muammoni hal qilish uchun yaratilgan. Merkl-Damgard sxemasi bilan ishlash prinsipi deyarli o'xshash, farqi f siqish funksiyasi ichki holati chiquvchi xesh qiymat uzunligidan katta.

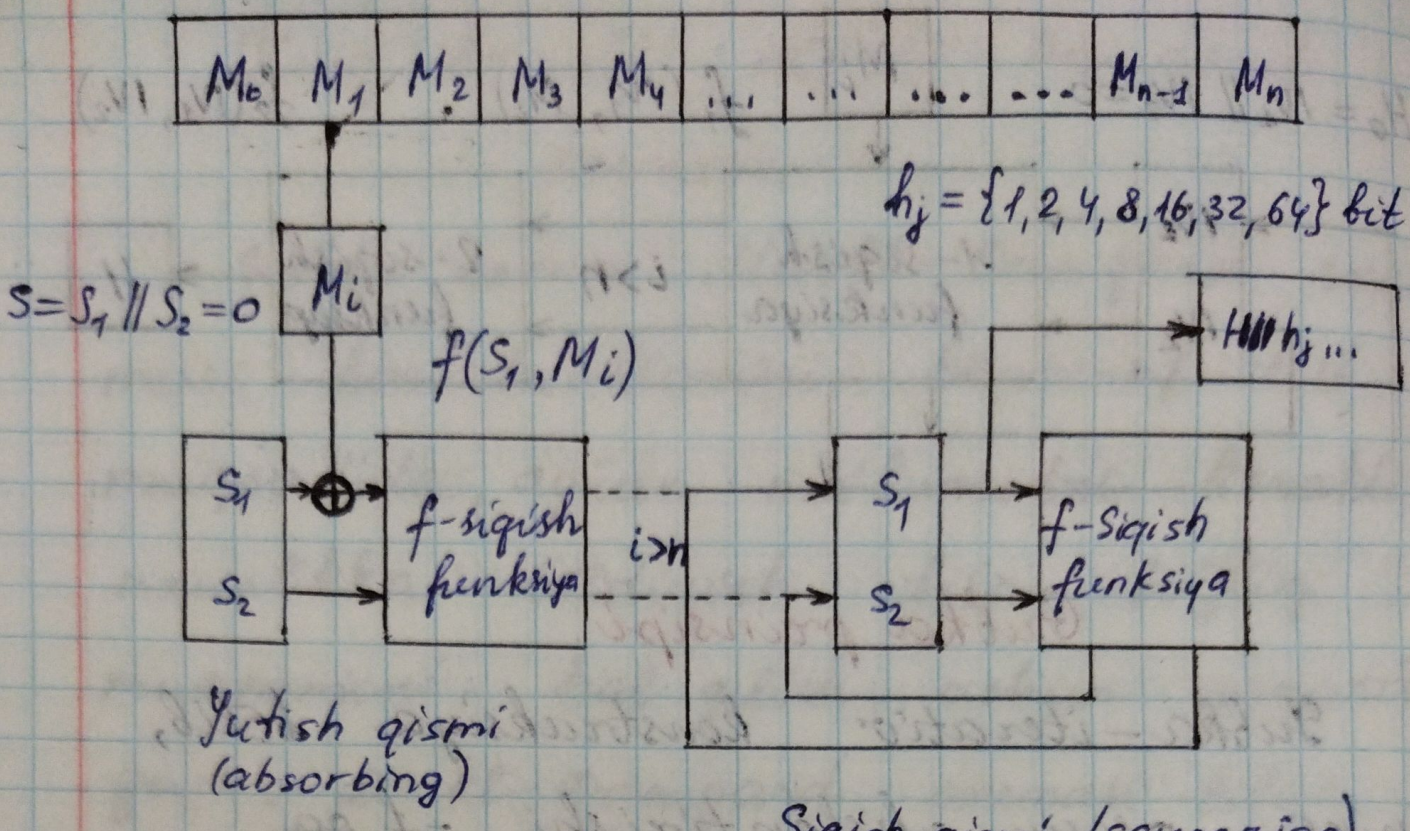
R ta raunddan keyin H_0 vektorning barcha elementlari qiymatga ega bo'ladi. va keyingi M_{i+1} blokka o'tiladi.



Gubka prinsipi.

Gubka – iterativ konstruksiya bolib, uning asosiy akslantirishi -f ga ixtiyoriy uzunlikdagi ma'lumot kiradi. va ixtiyoriy uzunlikdagi xesh qiymat chiqadi. Gubkaning ichki holati S (fiksirlangan b bit uzunlikda, $b = r + c$) orqali ifodalanadi. S ichki holat 2 ga bolinadi:

1. S_1 - olchami r bit
2. S_2 - olchami c bit.



R ta raunddan keyin S vektorning barcha elementlari qandaydir qiymatga ega bo'ladi va keyingi M_{i+1} blokka o'tiladi.

S_1 va S_2 vektorlarning o'lchami ixtiyoriy bo'lishi mumkin, faqat S_2 qismning uzunligi chiqishdagi xesh qiymat uzunligidan 2 marta katta bo'lishi kerak.

h_j – S vektorning bitta elementi uzunligiga teng.

11.03.2024

„Oz DSt 1105:2009"

Milliy standart shifrlash algoritmi

Oz. aloqa va axborotlashtirish agentligining „Unicon.uz" – Fan-texnika va marketing tadqiqotlari markazi DUK (Hozir MCHJ) tomonidan ishlab chiqilgan.

Algoritm SP tarmoğiga asoslangan

Blok uzunligi – 256 bit

Raundlar soni – 8 ta

Kalit uzunligi – 256 (512) bit

Rejimlari: – shifrmatn bloklarini ilaktirish,

– elektron kod kitobi.

Bu algoritmda Davlat siriga oid ma'lumotlarni shifrlashga ruxsat berilmaydi.

Algoritm dasturiy, apparat yoki apparat-dasturiy kriptografik modullarda amalga oshirish uchun mo'ljallangan.

Algoritm tarkibida quyidagi asosiy akslantirishlardan foydalaniladi:

Bayt Almash (Holat, B_a);

Sur (Holat);

Aralash (Holat, K_{se});

QōshBosqichKalit (Holat, K_e);

QōshHolat (Holat m; Holat)

ShaklSeansKalit (K_{st})

ShaklBosqichKalit (K_{se})

ShaklSeansKalitBayt (k, k_f)

Shifrlash algoritmining parametrlari va funksiyalari

M – dastlabki (ochiq) ma'lumotlar (foyd. xabari)

C – shifrmatn

m – shifrlash rejimi ($m = \{Sh_{Bil}^{CBC}; Ekk\}$);

sh – dastlabki matnni shifrmatnga ōgirish rejimi indeksi;

dsh – shifrmatnni dastlabki matnga ōgirish rejimi indeksi;

Holat – 4×8 (yoki 8×4) o'lchamli, bayt-lardan iborat holat massivi;

H_{ch}, H_o – ikki o'lchamli holat massi-vining chap (yuqori) va o'ng (quyi) qismi;

k – 256 yoki 512 bit uzunlikdagi shifrlash kaliti;

k_f – 256 bit uzunlikdagi funksional kalit;

k_{se} – seans bosqich kaliti.

B_a – elementlarni bayt sathida almash-tirish, chiziqli massivi;

K_{ss} – shifrlash seans kaliti element-larining chiziqli massivi;

K_s – shifrlash seans kalitining ikki o'lchamli massivi;

K_{sch}, K_{so} – shifrlash seans kaliti massivining chap (yuqori) va o'ng (quyi) qismi (yarmi);

K_e – 8x4 (yoki 4x8) tartibli bosqich kaliti massivi;

P_i $(P+1)$ – modul, $p=256$;

e – raundlar (bosqich) soni; $e=8$;

bosqich – shifrmatnga o'girish yoki dastlabki matnga o'girish protsedurasining tartib raqami;

R – parametr;

$\oplus$ – XOR amalining belgisi (2 modul bo'yicha qo'shish amali);

$\circledR$ – sonlarni p modul bo'yicha R parametr bilan ko'paytirish amalining simvoli.

$\circledR_2$ – p modul bo'yicha diamatritsaviy ko'paytirish amalining simvoli;

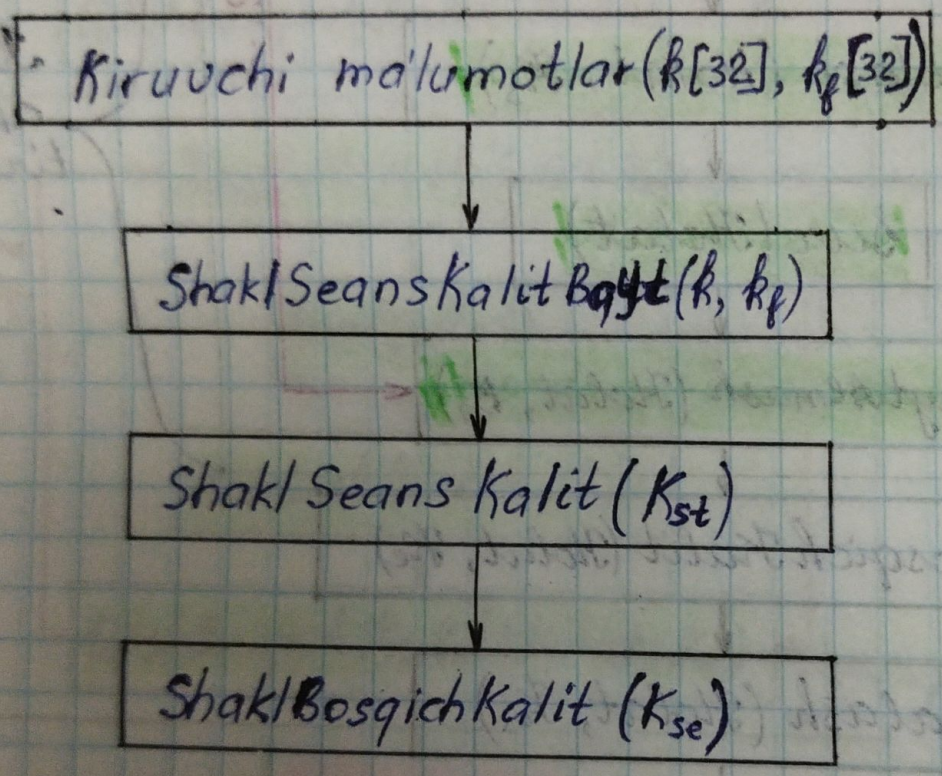
$\backslash -1$ – p modul bo'yicha R parametr bilan teskarilash amalining simvoli;

$\backslash d$ – p modul bo'yicha R parametr bilan d-darajaga oshirish amalining simvoli;

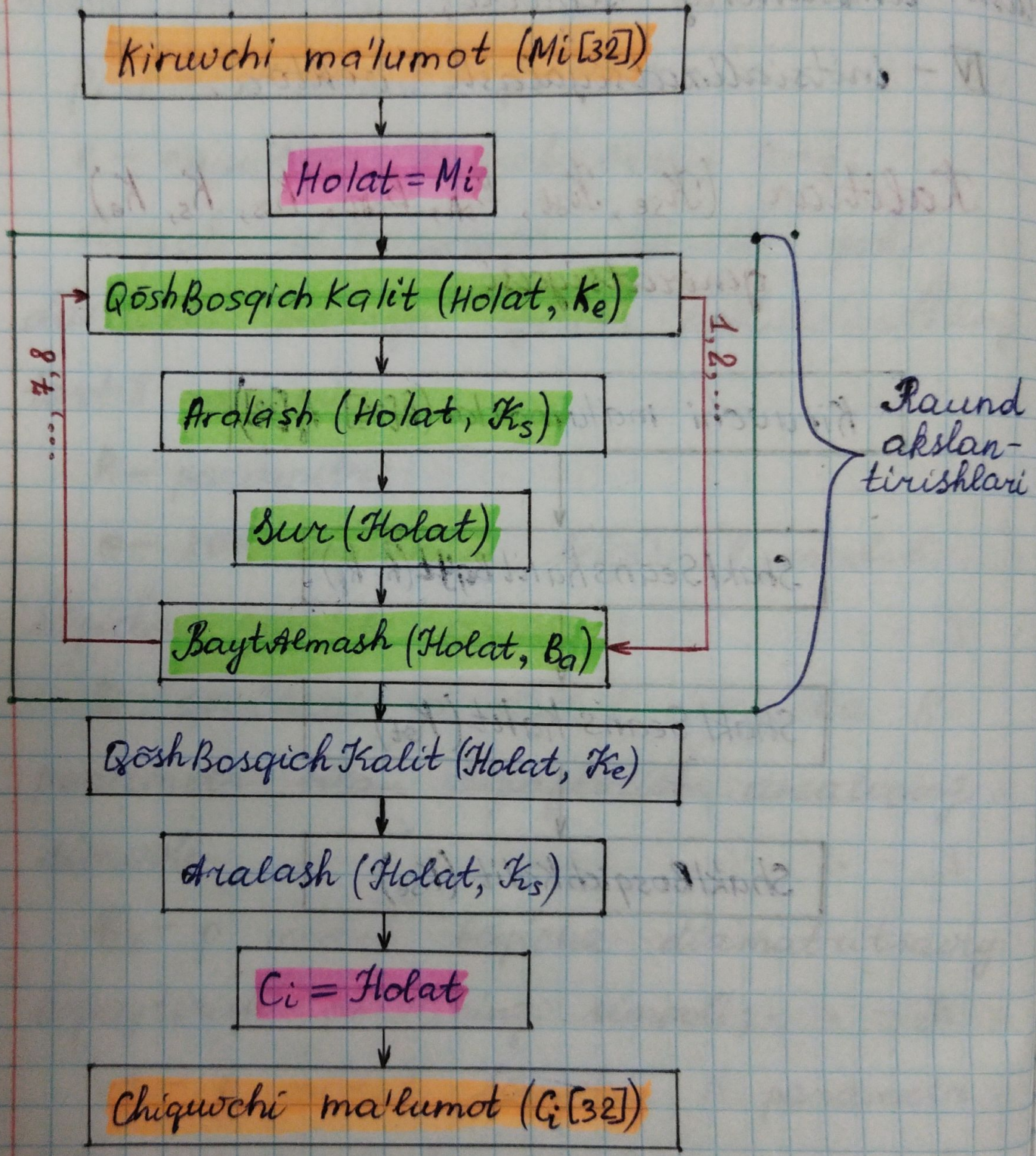
-1 – p yoki p+1 modul bo'yicha teskarolash amalining simooli;

IV – initsializatsiyalash vektori.

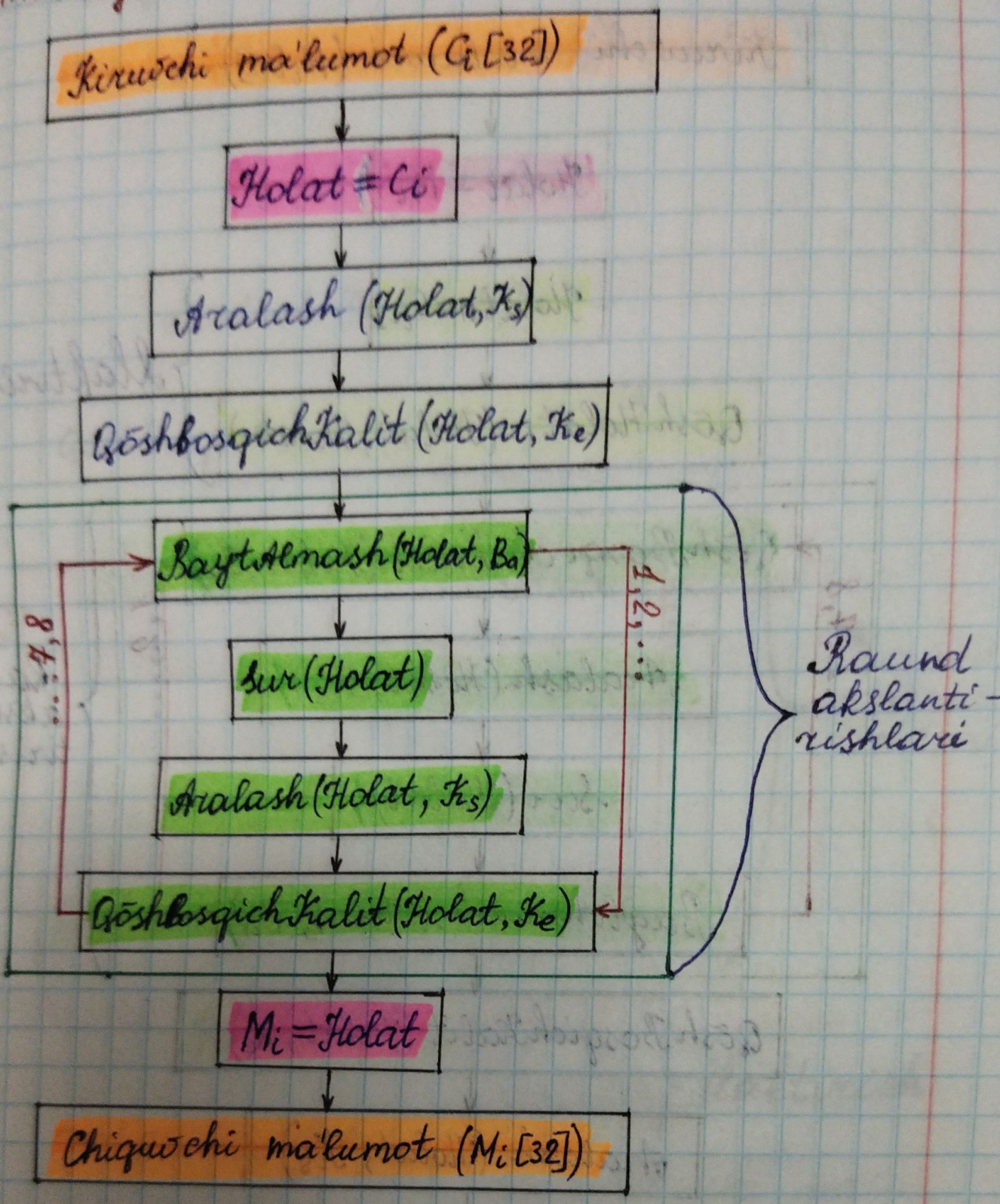
Kalitlar (K_{se}, K_{st}, B_{sA}, B_{sAD}, K_{ss}, K_s, K_e) generatsiyasi.



EKK rejimi (Shifrlash)

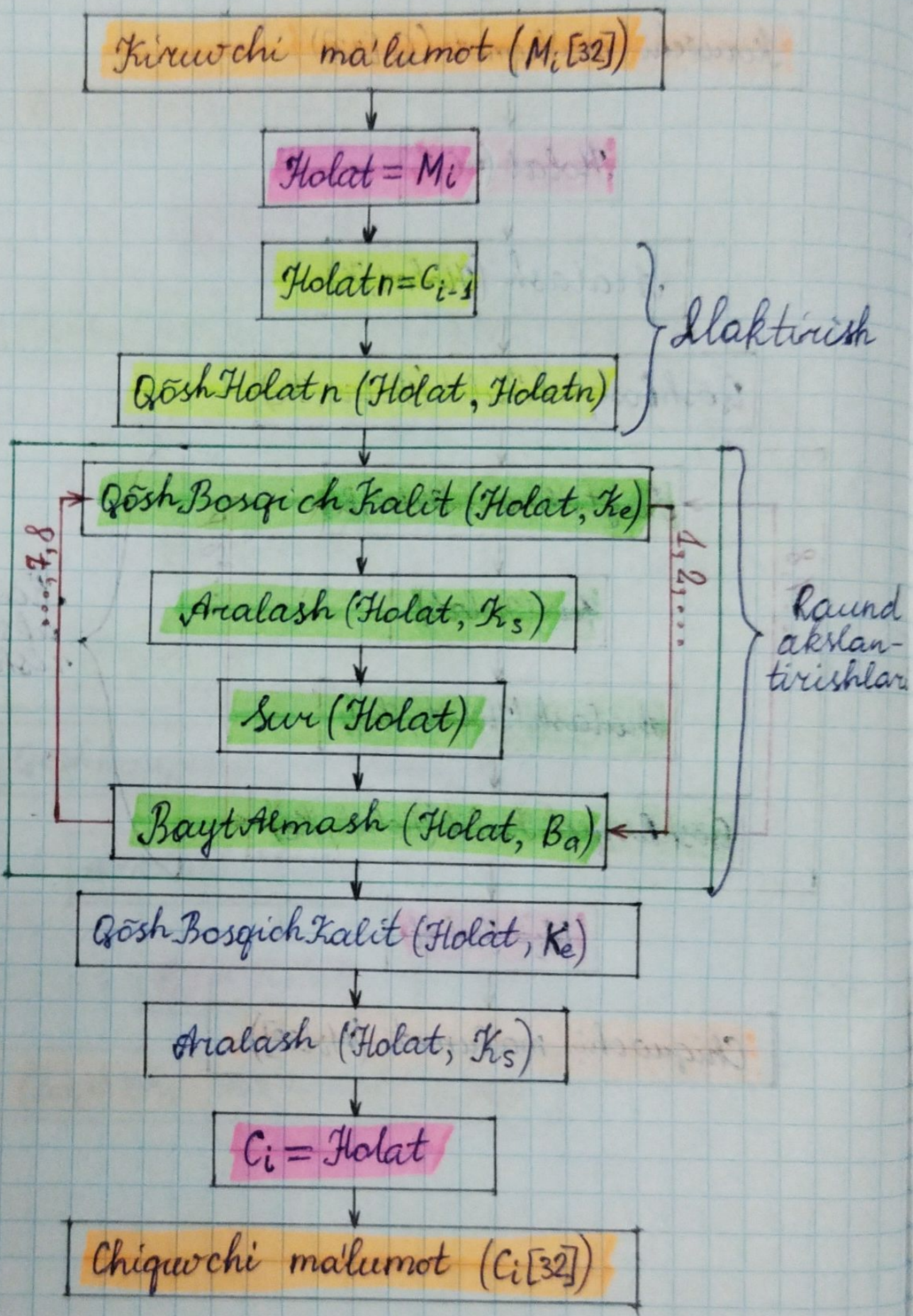


EKK rejimi (deshifrlash)

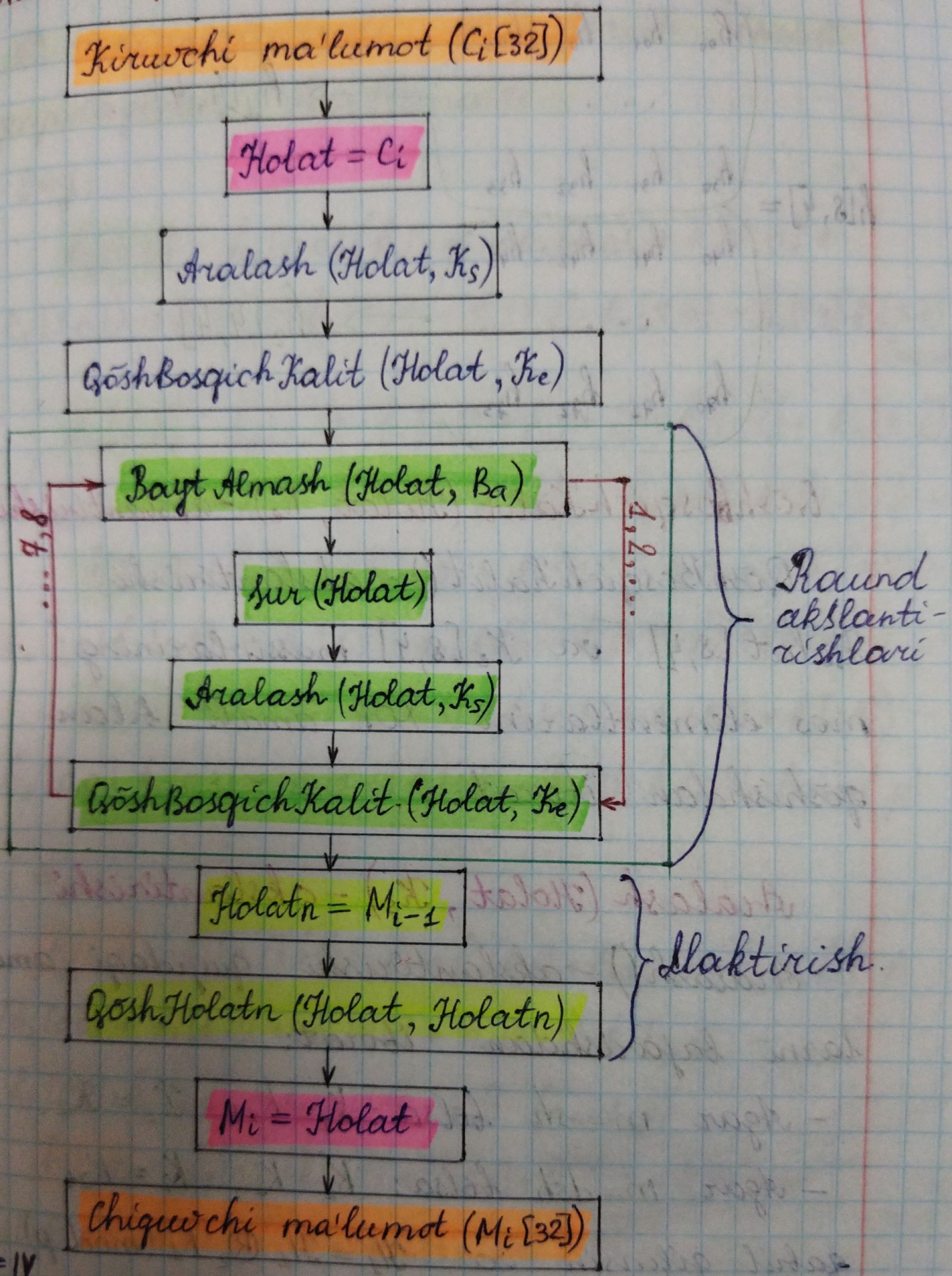


ShBil rejimi (shifrlash)

$C_0 = IV$

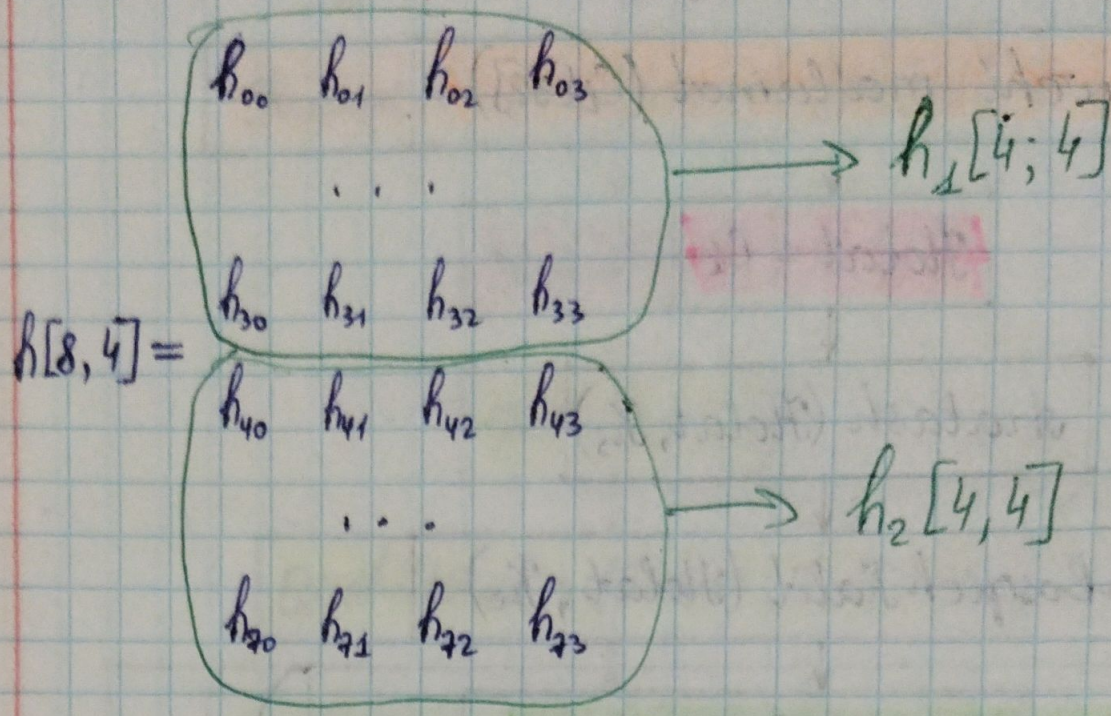


Shifrlash rejimi (Deshifrlash)



$M_0 = IV$

Holat [8,4] – massivi:



QoshBosqichKalit (Holat, Ke) – akslantirishi.

"QoshBosqichKalit() – akslantirishi Holat [8,4] va Ke [8,4] massivlarining mos elementlarini "XOR amali" bilan qoshishdan iborat.

Aralash (Holat, Ks) – akslantirishi

Aralash() – akslantirishi quyidagi amallarni bajarishdan iborat:

– Agar m = sh bolsa: $K_1 = K_{1t}$, $K_2 = K_2$.

– Agar m = dsh bolsa: $K_1 = K_1$, $K_2 = K_{2t}$

qabul qilinsin va $H_1 = H_1 \circledR_2 K_1 \pmod{\rho}$

hamda $H_2 = H_2 \circledR_2 K_2 \pmod p$ hisoblansin.

$\circledR_2$ – diamatritsali kopaytirish amali quyidagicha hisoblanadi:

$i=j$:

$$h_s[0,0] = h_s[0,0]\,(k_s[0,0] + k_s[1,0], + k_s[2,0] + k_s[3,0]) - h_s[1,1]\cdot k_s[1,0] - h_s[2,2]\cdot k_s[2,0] - h_s[3,3]\cdot k_s[3,0] \pmod p$$

$$h_s[1,1] = h_s[1,1]\cdot(k_s[0,1] + k_s[1,1] + k_s[2,1] + k_s[3,1]) - h_s[0,0]\cdot k_s[0,1] - h_s[2,2]\cdot k_s[2,1] - h_s[3,3]\cdot k_s[3,1] \pmod p$$

$$h_s[2,2] = h_s[2,2]\,(k_s[0,2] + k_s[1,2] + k_s[2,2] + k_s[3,2]) - h_s[0,0]\cdot k_s[0,2] - h_s[1,1]\cdot k_s[1,2] - h_s[3,3]\cdot k_s[3,2] \pmod p$$

$$h_s[3,3] = h_s[3,3]\,(k_s[0,3] + k_s[1,3] + k_s[2,3] + k_s[3,3]) - h_s[0,0]\cdot k_s[0,3] - h_s[1,1]\cdot k_s[1,3] - h_s[2,2]\cdot k_s[2,3] \pmod p$$

$i \neq j$:

$$h_s[0,1] = h_s[0,1]\,(k_s[0,1] + k_s[1,1] + k_s[2,1] + k_s[3,1]) + (h_s[0,0] + h_s[1,0] + h_s[2,0] + h_s[3,0])\cdot k_s[0,1] - h_s[0,2]\cdot k_s[2,1] - h_s[0,3]\cdot k_s[3,1] \pmod p$$

$$h_s[0,2] = h_s[0,2]\,(k_s[0,2] + k_s[1,2] + k_s[2,2] + k_s[3,2]) + (h_s[0,0] + h_s[1,0] + h_s[2,0] + h_s[3,0])\cdot k_s[0,2] - h_s[0,1]\cdot k_s[1,2] - h_s[0,3]\cdot k_s[3,2] \pmod p \quad \ldots$$

Sur(Holat) - akslantirishi

- Agar m=sh bolsa, Holat massivining j- ustuni avvalo, pastga (j+1) (mod 8) baytga davriy surilsin, songra hosil bolgan massivning i- satri ongga (i+1) (mod 4) baytga davriy surilsin.

- Agar m=dsh, bolsa, Holat massivining i- satri avvalo, chapga (i+1) (mod 4) baytga davriy surilsin, songra hosil bolgan massivning j- ustuni yuqoriga (j+1) (mod 8) baytga davriy surilsin. (Bu yerda $0 \le i < 8$, $0 \le j < 4$)

BaytAlmash(Holat, Ba) - akslantirishi

- Agar m=sh bolsa, Holat[8,4] massivining har bir bayt elementi $B_{SA}[256]$ massivining adresi boyicha unga mos elementi bilan almashtirilsin.

$$Holat[i,j] = B_{SA}[Holat[i,j]]$$

bu yerda, toq bosqichlarda s=1,

juft bosqichlarda $s=2$, $i=0...8$, $j=0...4$.

- Agar $m=dsh$, bolsa, Holat [8,4] massivining har bir bayt elementi B_{SAD} massivining adresi bo'yicha unga mos elementi bilan almashtirilsin.

$$Holat[i,j] = B_{SAD}[Holat[i,j]]$$

bu yerda, toq bosqichlarda $s=2$, juft bosqichlarda $s=1$, $i=0...8$, $j=0...4$

ShaklSeansKalitBayt (k, k_f) - akslantirishi

- $k_{se} = k + k' \cdot (1 + k_f \cdot k)$ hisoblansin va chapdan 672 bit qoldirilsin, bu yerda k' - k_f ning öngdan 192 bitli qismi.

- k_{se} da öngdan 256+64 bitli qism ajratib olinsin va 256 bitli qismdan baytli chiziqli massiv $K_{st} = [0,1,2,3,...,31]$, qolgan 64 bitli qismdan baytli chiziqli massiv $B = [0,1,2,3,4,5,6,7]$ (hamda $B_1 = [0,1,2,3]$ va $B_2 = [4,5,6,7]$ shakllantirilsin.

– Quyida keltirilgan qoidalar asosida (d_1, R_1, L_1) va (d_2, R_2, L_2) parametrlar uchliklari shakllantirilsin:

1) agar $b[0]<3$ bolsa, $d_1=3$, aks holda $d_1=b[0]$ qabul qilinsin;

agar $b[4]<3$ bolsa, $d_2=3$, aks holda $d_2=b[4]$ qabul qilinsin.

2) agar $b[1]=0$ bolsa, $R_1=1$, aks holda $R_1=b[1]$ qabul qilinsin;

agar $b[5]=0$ bolsa, $R_2=1$, aks holda $R_2=b[5]$ qabul qilinsin.

3) agar $b[2]=0$ bolsa, $L_1=1$, aks holda $L_1=b[2]$ qabul qilinsin;

agar $b[6]=0$ bolsa, $L_2=1$, aks holda $L_2=b[6]$ qabul qilinsin.

4) $d_s \pmod 2 = 0$ uchun agar $d_s \pmod 4 =$ bolsa, $d_s=d_s-1$, aks holda $d_s=d_s+1$ qabul qilinsin, bu yerda $s \in \{1, 2\}$.

5) $d_s \pmod 2 = 1$ uchun agar, $d_s - 1 \pmod 4 = 0$

bolsa, $d_s = d_s - 2$ qabul qilinsin.

6) agar $b[3] = 0$ bolsa, $b[3] = 1$, agar $b[7] = 0$ bolsa, $b[7] = 1$ qabul qilinsin.

- Quyida keltirilgan qoidalar asosida $m = sh$ uchun $B_{1A}[256]$ va $B_{2A}[256]$ chiziqli massivlar juftligi shakllantirilsin:

$i = 0, \ldots, 255$, $s = 1, 2$ uchun:

$- B_{sA}[i] \equiv (((i + L_s) \bmod 256) + 1)^{(ds)} \pmod{257} \pmod{256}$ hisoblansin,

- agar $i - B_{sA}[i] = 0$ va $|B_{sA}[i-1] - B_{sA}[i]| \leq 8$ bolsa, $s = 1$ da $B_{sA}[i]$ ning qiymati $(i - b[3]) \pmod{256}$ yoki $s = 2$ da $B_{sA}[i]$ ning qiymati $(i - b[7]) \pmod{256}$ ga teng manzildagi element bilan joylari almashtirilsin va $s = 1$ uchun $b[3] = b[3] - 5 \pmod{256}$ yoki $s = 2$ uchun $b[7] = b[7] - 5 \pmod{256}$ qabul qilinsin.

Izoh - toq bosqichlarda B_{1A}, juft bosqichlarda B_{2A} massivdan foydalani-

ladi.

- Quyida keltirilgan qoidalar asosida $m = dsh$ uchun $B_{1AD}[256]$ va $B_{2AD}[256]$ chiziqli massivlar juftligi shakllantirilsin:

- chiziqli massiv $B_{sA}[256]$ $(s \in \{1, 2\})$ ning har bir bayt elementi uning manzili (indeksiga) teng qiymatga almashtirish va hosil bolgan massivni manzillar ortib borishi tartibida joylashtirish kifoya.

Dzoh - toq bosqichlarda B_{2AD}, juft bosqichlarda B_{1AD} massivdan foydalaniladi.

Shakllanuvchi kalit (k_{st}) - akslantirishi

- $K_{ss}[0, 1, 2, \ldots, 19] = K_{st}[0, 1, 2, \ldots, 19, 20, 21, \ldots, 31]$ hisoblansin.

• $i = 0, \ldots, 19$ uchun, agar $K_{ss}[i] = 0$ bolsa,

$K_{ss}[i] = K_{ss}[i] - 1 \pmod p$

- agar $K_{ss}[6] \pmod 2 = 0$ bolsa $K_{ss}[6] = K_{ss}[6] - 1 \pmod p$
- agar $K_{ss}[16] \pmod 2 = 0$ bolsa $K_{ss}[16] = K_{ss}[16] - 1 \pmod p$
- agar $K_{ss}[6] + K_{ss}[0] + K_{ss}[8] + K_{ss}[3] + K_{ss}[5] \pmod 2 = 0$

bolsa $K_{ss}[8] = K_{ss}[8] - 1 \pmod p$

- agar $K_{ss}[16] + K_{ss}[10] + K_{ss}[8] + K_{ss}[13] + K_{ss}[15] \pmod 2 = 0$

bolsa $K_{ss}[18] = K_{ss}[18] - 1 \pmod p$

- agar $K_{ss}[6] + K_{ss}[1] + K_{ss}[3] + K_{ss}[9] + K_{ss}[4] \pmod 2 = 0$

bolsa $K_{ss}[9] = K_{ss}[9] - 1 \pmod p$

- agar $K_{ss}[16] + K_{ss}[11] + K_{ss}[13] + K_{ss}[19] + K_{ss}[14] \pmod 2 = 0$

bolsa $K_{ss}[19] = K_{ss}[19] - 1 \pmod p$

- agar $K_{ss}[6] + K_{ss}[2] + K_{ss}[3] + K_{ss}[4] + K_{ss}[7] \pmod 2 = 0$

bolsa $K_{ss}[7] = K_{ss}[7] - 1 \pmod p$

- agar $K_{ss}[16] + K_{ss}[12] + K_{ss}[13] + K_{ss}[14] + K_{ss}[17] \pmod 2 = 0$

bolsa $K_{ss}[17] = K_{ss}[17] - 1 \pmod p$

- $K_{ss1}[0, 1, \dots, 9] = K_{ss}[0, 1, \dots, 9]$ va $K_{ss2}[0, 1, \dots, 9] = K_{ss}[10, 11, 2, 3, \dots, 9]$ hisoblansin.

Shakl Bosqich Kalit (k_{se}) - akslantirishi:

Barcha $e=1, 2, ..., 8$ bosqichlar uchun:

- Agar bosqich = 1 bolib, $m=sh$ bolsa k_{se} (CHIZIQLI SEANS-BOSQICH KALITI) ozgarmay ~~qoladi~~ qolsin, $m=dsh$ bolsa k_{se} massivi davriy ongga $672-(e*83) \bmod 672$ bitga surilsin.

k_{se} ning chap tomonidan 256 bitli qismini ajratib olib, $K_e[8,4]$ massivi shakllantirilsin.

- Agar bosqich > 1 bolib, $m=sh$ bolsa k_{se} davriy ongga 83 bitga surilsin, $m=dsh$ bolsa, unda k_{se} davriy chapga 83 bitga surilsin.

k_{se} ning chap tomonidan 256 bitli qismini ajratib olib, $K_e[8,4]$ massivi shakllantirilsin.

25.03.2024.

O'zDSt 1106:2009. Xeshlash funksiyasi.

Ushbu standart axb. ni qayta ishlash va muhofaza qilishning kriptografik metodlarida, shu jumladan avtomatlashtirilgan tizimlarda axb. uzatish, qayta ishlash va saqlashda raqamli imzo protseduralarini amalga oshirish uchun qo'llaniladigan ikkilik simvollarining istalgan ketma-ketligi uchun xeshlash funksiyasining (XF) algoritmini va hisoblash protsedurasini belgilaydi.

— Mazkur standart algoritmi SP tarmog'iga asoslangan bo'lib, simmetrik blokli shifrlash algoritmlari guruhiga kiradi.

— Xeshlash algoritmida kirish va chiqish ketma-ketligi hamda xeshlash kalitining uzunligi 128 yoki 256 bit.

— Algoritmlar:

1) Parametrli algebraga asoslangan milliy algoritm.

2) ГОСТ Р 34.11-94

— Bosqichlar (raundlar) soni – 10 ta

— Algoritmning ma'lumotlarni xeshlash protsedurasida xeshlash ~~protsedurasi~~ kaliti va xeshlashning oraliq natijasi asosida shakllangan bosqich kalitlaridan foydalaniladi.

— xeshlash funksiyasida modul arifmetikasining bir tomonlama funksiyasi qollaniladi, u bo'yicha hisoblashlar darajaga ko'tarish amallari amalga oshiriladi. Algoritmda ko'paytirish, darajaga ko'tarish va teskarilash kabi asosiy amallar natural sonlar uchligidan iborat bo'lgan (A, B, R) parametr orqali amalga oshiriladi.

Belgilashlar va akslantirishlar

M – dastlabki ma'lumotlar (xabar);

h – xesh-funksiya $(m = h(M))$

k – xeshlash kaliti.

k_e – 4×8 tartibli 2 o'lchamli massiv ko'rinishidagi bosqich kaliti.

holat – 4×8 tartibli 2 o'lchamli massiv ko'rinishidagi xeshlashning oraliq natijasi.

holat n – 4×8 tartibli 2 o'lchamli massiv ko'rinishidagi kirish bloki;

p – modul, bunda $p \in \{16, 256\}$;

b – dastlabki ma'lumotlardagi bloklar soni

$\otimes$ – diamatritsalarni p modul bo'yicha ko'paytirish.

$\circledR$ – parametr bilan p modul bo'yicha ko'paytirish amalining simvoli.

XF – xeshlash funksiyasi.

uzunlik – dastlabki ma'lumotlarning bitlardagi uzunligini o'z ichiga oluvchi, xeshlash funksiyasiga kiruvchi ma'lumotlarning

oxirgidan oldingi bloki.

NY – nazorat summasi bo'lib, onlik sanoq tizimida dastlabki ma'lumotlar qiymatlari summasini o'z ichiga oluvchi; xeshlash funksiyasiga kiruvchi ma'lumotlarning oxirgi bloki;

Algoritmda Qo'sh(holat, holatn), Baytlichlash(holat, holatn), Aralash(holat, ke), SwitHolat(holat), SwiKalit(ke), TuzilmaKalit(ke, k) va boshqa akslantirishlardan foydalanilgan.

Yarim bayt massivi (128 bit)

$$\begin{cases} a_0 = \{kir_0, kir_1, kir_2, kir_3\} \\ a_1 = \{kir_4, kir_5, kir_6, kir_7\} \\ \quad \cdots\cdots\cdots \\ a_{31} = \{kir_{124}, kir_{125}, kir_{126}, kir_{127}\} \end{cases}$$

Bayt massivi (256 bit)

$$\begin{cases} a_0 = \{kir_0, kir_1, kir_2, \dots, kir_7\}; \\ a_1 = \{kir_8, kir_9, kir_{10}, \dots, kir_{15}\}; \\ \dots \\ a_{31} = \{kir_{247}, kir_{248}, \dots, kir_{255}\} \end{cases}$$

Holat massivi

$$\begin{bmatrix} h[0,0], & h[0,1], & \dots, & h[0,7] \\ & \dots & & \\ h[3,0], & h[3,1], & \dots, & h[3,7] \end{bmatrix}$$

Matritsalarni kopaytirish amali bajarilganda holat[4,8] massivini, $H_{ch}[4,4]$ va $H_o[4,4]$ (chap va ong) qismlarga ajratiladi.

O'zgartirishlardan song bu qismlarning nusxalari holat[4,8] massiviga qayta ko'chiriladi.

↑

k_e – Bosqich kaliti

Xeshlash jarayoni.

1-bosqich: Kirish ma'lumotlarini tayyorlash

— Ma'lumot (M) 128 (256) bit uzunlikdagi bloklarga ajratiladi:

$$M = \{M_1, M_2, \dots, M_6\}.$$

— Oxirgi M_6 blokning yetmay qolgan qismi "0" lardan iborat yarim bayt (bayt) ketma-ketligi bilan to'ldiriladi.

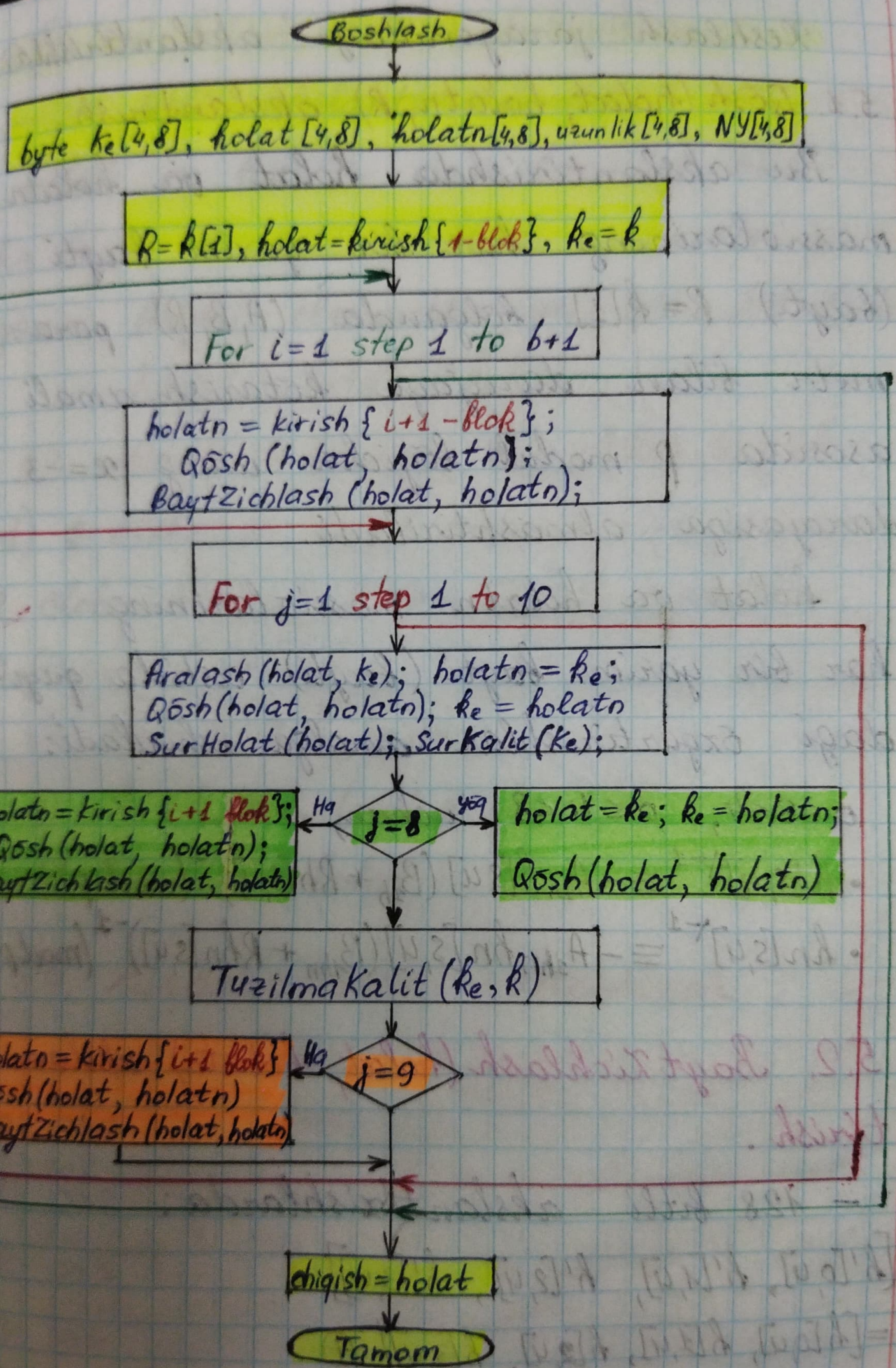
$$M_6 = M_6 + 0$$

— M_{6+1} = uzunlik = len(M)

— $M_{6+2} = NY = (M_1 + M_2 + \dots + M_6)$

— Dastlabki K_e bosqich kalit massivi 128 (256) bit uzunlikdagi k xeshlash kalitidan iborat: $K_e = k$

2-bosqich: Xesh qiymatni shakllantirish ketma-ketligi.


Boshlash
byte Ke[4,8], holat[4,8], holatn[4,8], uzunlik[4,8], NY[4,8]
R=K[1], holat=kirish{1-blok}, Ke=K
For i=1 step 1 to 6+1
holatn = kirish{i+1 -blok};
Qosh(holat, holatn);
BaytZichlash(holat, holatn);
For j=1 step 1 to 10
Aralash(holat, ke); holatn=Ke;
Qosh(holat, holatn); Ke=holatn
SurHolat(holat); SurKalit(Ke);
holatn=kirish{i+1 blok};
Qosh(holat, holatn);
BaytZichlash(holat, holatn)
Ha
j=8
yoq
holat=Ke; Ke=holatn;
Qosh(holat, holatn)
TuzilmaKalit(Ke, K)
holatn=kirish{i+1 blok}
Qosh(holat, holatn)
BaytZichlash(holat, holatn)
Ha
j=9
chiqish=holat
Tamom

Xeshlash jarayonidagi akslantirishlar

5.1. Qosh (holat, holatn, R) akslantirish.

Bu akslantirishda holat va holatn massivlarining har bir yarim bayti (bayti) R=k[1] bolganda (A, B, R) parametr bilan darajaga kotarish amali asosida p modul boyicha uning $x=-3$ darajasiga almashtiriladi.

holat va holatn massivlarining har bir yarim bayti (bayti) ustida quyidagi ozgartirishlar amalga oshiriladi:

$0 \le s < 4$, $0 \le u < 8$ uchun,

- $h[s,4]^{-1} \equiv -A_{1h}\, h[s,4]\,(B_{1h} + Rh[s,4])^{-1} \pmod p$
- $hn[s,4]^{-1} \equiv -A_{1hn}\, hn[s,4]\,(B_{1hn} + Rhn[s,4])^{-1} \pmod p$

5.2. Bayt zichlash (holat, holatn) akslantirish.

! – 128 bitli akslantirishlarda:

$[h'[0,4], h'[1,4], h'[2,4], h'[3,4]] =$

$= [h[0,4], h[1,4], h[2,4], h[3,4]] \oplus$

⊛ [hn[0,4], hn[1,4], hn[2,4], hn[3,4]]

– 256 bitli akslantirishlarda:

holat va holatn massivlarining har bir bayti chiziqli zichlash massivining yarim baytli elementlariga almashtiriladi:

holat = holat$_2$ || holatn$_2$.

5.3. Aralash (holat, k$_c$) akslantirish

1) 2 o'lchamli holat massivining nusxasini 2 o'lchamli $\mathcal{H}_{ch}[4,4]$ va $\mathcal{H}_o[4,4]$ massivlari juftligiga ko'chirish.

2) $\mathcal{H}_{ch} \equiv \mathcal{H}_{ch} \otimes k_{ech} \pmod p$

3) $\mathcal{H}_o \equiv \mathcal{H}_o \otimes k_{eo} \pmod p$

4) $\mathcal{H}_{ch}, \mathcal{H}_o \rightarrow$ holat.

5.4. SurHolat (holat) akslantirish

Satr.

0: >>1

1: >>2

2: >>3

3: >>4

Ustun.

0, 4: $\twoheaddownarrow$ 1

1, 5: $\twoheaddownarrow$ 2

2, 6: $\twoheaddownarrow$ 3.

5.5. Surkalit (k_e) akslantirish.

Satr:

0: $\gg$ 4

1: $\gg$ 3

2: $\gg$ 2

3: $\gg$ 1.

Ustun:

1, 5: $\twoheaduparrow$ 3

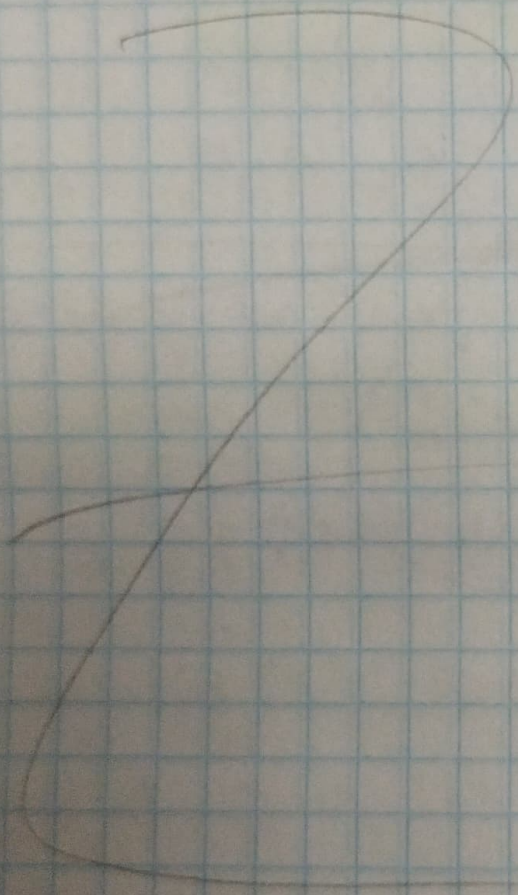
2, 6: $\twoheaduparrow$ 2

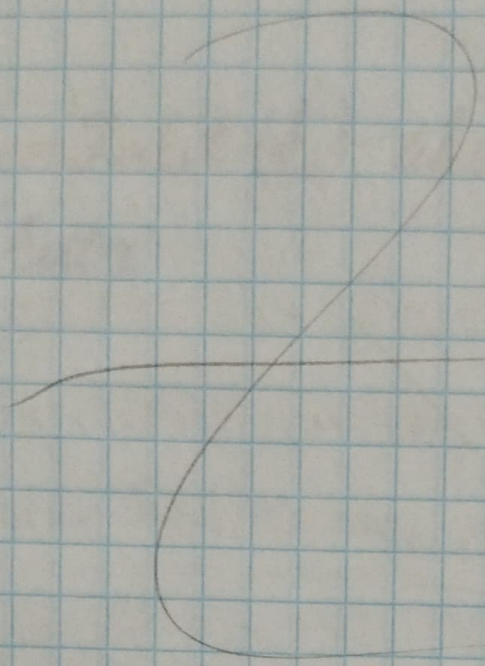
3, 7: $\uparrow$ 1.

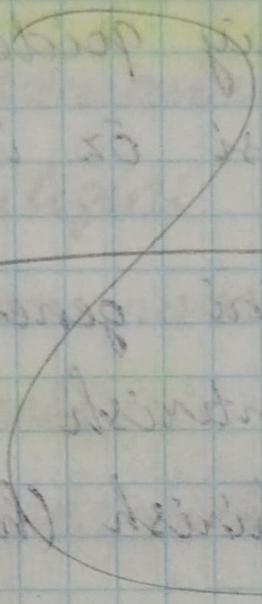
5.6. Tuzilmakalit (k_e, k) akslantirish

- agar $k_e[s,4] \equiv 0 \pmod p$ va $k[s,4] \equiv 0 \pmod 2$ bolsa, u holda $k_e[s,4] = 2$ qabul qilinadi.
- agar $k_e[s,4] \neq 0 \pmod p$ va $k_e[s,4] \pmod 2 \equiv$

$\equiv k[s,4] \pmod 2$ bolsa, u holda $k_e[s,4] = k_e[s,4]$ qabul qilinadi, aks holda $k_e[s,4] \equiv k_e[s,4] - 1 \pmod p$ qabul qilinadi.







29.03.2024

Oz DSt 1092:2009

ERI algoritmi

(28.09.2009 yilda tasdiqlangan va amalga kiritilgan)

1. Umumiy qoidalar

1. ERI sxemasi oz ichiga 3 jarayonni oladi:

- ERI kalitlarini generatsiyalash
- ERI shakllantirish
- ERI ni tekshirish (haqiqiyligini tasdiqlash)

2. Ushbu standart ERI ni shakllantirish hamda ERI ning haqiqiyligini tasdiqlash uchun 2 ta algoritm (1- va 2- algoritm) ni taosiflaydi.

1-algoritm 2 asosiy rejimda qollaniladi:

- seans kalitsiz ($\mu=0$);
- seans kalitli ($\mu=1$).

2-algoritm [ГОСТ Р 34.10-2001] an'anaviy (seans kalitsiz) rejimda qo'llaniladi.

1-algoritmda ERI ni shakllantirish jarayoniga ERI ning haqiqiyligini tasdiqlash jarayonida qo'llaniladigan seans kaliti protsedurasini kiritish bilan ERI qalbakiligini aniqlashning zaxiraviy yo'li ham nazarda tutilgan.

2. Belgilashlar

M – foydalanuvchi xabari;

p – tub son;

q – tub son;

R – natural son – parametr;

(x, u) – butun sonlar juftligi – ERI yopiq kaliti.

(y, z) – butun sonlar juftligi – ERI ochiq kaliti

g – natural son – asos

(R_1, y_1) – butun sonlar juftligi – ERI ni

haqiqiy ekanligini aniqlovchi nazorat va ochiq seans kaliti.

(r, s) – butun sonlar juftligi – M axborotning ERI si;

(R, g, k_h) – vakolatlangan subyektning maxsus shaxsiy kaliti;

k_h – xeshlash kaliti;

Ⓡ – modul bo'yicha sonlarni R parametr (~~bo'yicha~~) bilan ko'paytirish amali;

$\backslash^e$ – modul bo'yicha e darajaga parametr bilan ko'tarish amalining simvoli;

$\backslash^{-1}$ – modul ~~bo'yicha~~ parametr bilan teskarilash amalining simvoli;

$^{-1}$ – modul bo'yicha teskarilash amalining simvoli.

3. Matematik kelishuvlar.

Ta'rif: Modul arifmetikasida parametr $R>1$ bilan darajaga oshirish funksiyasi parametrli funksiya ($F(x)$) deb ataladi.

Teorema: $F(x)$ parametrli funksiya quyidagi ifoda orqali aniqlanadi:

$$[F(x) = 1 + R * x]$$

Misol: $R=2$, $x=21$ da $F(21) = 1 + 2 \cdot 21 = 43$

R koeffitsiyentli p modul bo'yicha ko'paytirish jarayoni quyidagicha aniqlanadi:

$$[X \circledR Y \pmod p \equiv X + F(X) \cdot Y \pmod p \equiv$$
$$\equiv X + (1 + X \cdot R) \cdot Y \pmod p].$$

Misol: $R=5$, $x=3$, $y=4$, $p=7$

$$3 \circledR 4 \bmod 7 \equiv 3 + (1 + 3 \cdot 5) \cdot 4 \pmod 7 \equiv 4$$

R koeffitsiyentli p modul bo'yicha X asosini e darajaga ko'tarish jarayoni quyidagi ko'rinishda ifodalanadi:

$$\left[X^{\circledR e} \pmod p = X \cdot \sum_{i=0}^{i=e-1} (F(X))^i \pmod p = X \cdot \sum_{i=0}^{i=e-1} (1 + R \cdot X)^i \pmod p\right]$$

Misol: $R=7$, $e=4$, $p=3$, $X=2$ da

$$2^{14} \pmod 3 = 2 \cdot \sum_{i=0}^{i=3} (1+7\cdot 2)^i \pmod 3 =$$

$$= 2\cdot(1+15+225+3375) \bmod 3 = 2$$

R koeffitsiyentli p modul bo'yicha X o'zgaruvchiga murojaat (teskarilash) jarayoni

$$\left[X^{|-1} \equiv (p-X)\cdot(F(X))^{-1} \pmod p \equiv (p-X)\cdot(1+X\cdot R)^{-1} \pmod p\right]$$

Misol: $x=3$, $R=2$, $p=11$

$$3^{|-1} \equiv (11-3)\cdot(1+3\cdot 2)^{-1} \pmod{11} \equiv 8\cdot 7^9 \pmod{11} = 9$$

4. ERI algoritmining parametrlari

4.1. ERI algoritmida quyidagi parametrlardan foydalaniladi:

a) p – modul, tub son.

$p > 2^{255}$ – maxsus apparat tipi uchun.

$p > 2^{1023}$ – dasturiy, gibrid va apparat tipidagilar uchun;

b) q – tub son hamda $p-1$ ning bo'luvchisi, bu yerda $2^{254} < q < 2^{256}$

c) R – parametr, $R<q$ shartni qanoatlantiruvchi natural son bo'lib, chegaralangan

guruhdagi foydalanuvchilar uchun ochiq yoki maxfiy kaliti bolishi mumkin;

d) $m = \mathcal{H}(M)$ – M xabarni 256 bit uzunlikdagi qatorda aks ettiruvchi xesh-funksiya;

kriptografik modulning dasturiy, gibrid va apparat tillarida – kalitsiz xesh-funksiya, maxsus apparat tilida esa – kalitli xesh funksiyadan foydalaniladi.

4.2. ERI algoritmida har bir foydalanuvchi ozining shaxsiy kalitiga ega bolishi kerak:

a) (x, u) – yopiq kalit(lar)

$1 < x, u < q$

b) g – yopiq yoki ochiq parametr,

$g \equiv h^{(p-1)/q} \pmod p$ yordamida hisoblanuvchi butun son, bu yerda:

$h < p$ - natural son bolib, $\omega \in [1; q]$ oraligida faqat $\omega = q$ bolgandagina $g^{\omega} \pmod p \equiv 0$ shartni qanoatlantiradi.

c) (y, z) – ochiq kalit (lar).

$$y \equiv g^x \pmod{p}$$

$$z \equiv g^u \pmod{p}$$

Agar ochiq parametr (asos) g dan foydalanilsa, unda $u = 1$, $z = g$.

d) (R_1, y_1) – ERI qabul qilishni aniqlash kaliti (faqat sxema kalitli rejimda);

bu yerda:

R_1 – nazorat kaliti (ochiq yoki yopiq), $1 \div q-1$ diapazondan tanlab olingan.

Agar R_1 yopiq bo'lsa, unda R_1 imzolovchi shaxs va tekshiruvchi tomon uchun birgalikdagi maxfiy kalit bo'lishi kerak;

e) y_1 – sxema (ochiq) kaliti, har bir ERI uchun parametr bilan darajaga ko'tarish natijasi kabi hisoblanadi.

Foydalanuvchilar guruhi uchun p, q tub sonlari ochiq va umumiy bo'lishi mumkin.

5. Asosiy jarayonlar

5.1. ERI ni va seans kalitni shakllantirish.

1) xabarning xesh funksiyasini hisoblang: $m=H(M)$ va $c=x$ ni qabul qiling;

2) $k=H(m+(1+mR)c)$ ni hisoblang.

Agar $k=0$ bolsa, unda $c=c+2$ ni qabul qiling. va 2-qadamga qayting;

3) R parametr bilan $T\equiv g^{1-k} \pmod p$ ni hisoblang.

4) $r\equiv m+(1+mR)T \pmod p$ ni hisoblang;

Agar $r \pmod q = 0$ bolsa, unda $k\equiv k+1 \pmod p$ ni qabul qiling. va 3-qadamga qayting;

5) $s_1\equiv k-rx \pmod q$ ni hisoblang. Agar $s_1=0$ bolsa, unda $k\equiv k+1 \pmod p$ ni qabul qiling va 3-qadamga qayting;

6) $s\equiv s_1u^{-1} \pmod q$ ni hisoblang. Agar $s=0$ bolsa, unda r, s ni chiqishga bering va hisoblashni toxtating;

7) $r_1\equiv R_1+(1+RR_1)r \pmod q$ ni hisoblang.

Agar $r_1=0$ bolsa, unda $k\equiv k+1 \pmod p$ ni

qabul qiling va 3-qadamga qayting;

8) $x_1 \equiv (k - suR_1) r_1^{-1} \pmod q$ ni hisoblang. Agar $x_1 = 0$ bolsa, unda $k \equiv k+1 \pmod p$ ni qabul qiling va 3-qadamga qayting;

9) RR_1 parametr bilan $y_1 \equiv (gR_1^{-1})^{|x_1|} \pmod p$ ni hisoblang va $r, s, y1$ larni chiqishga bering.

5.2. ERI ni haqiqiyligini tasdiqlash.

1) $m = H(M)$ xesh-funksiyani hisoblang;

2) agar $L(s) \le L(q)$ va $L(r) \le L(p)$ bolsa, unda keyingi qadamga oting, aks holda „imzo haqiqiy emas" deb qabul qilinadi.

3) R parametr bilan $z_0 \equiv z^{|s|} \pmod p$ ni hisoblang;

4) $r' = r \pmod q$ ni hisoblang.

5) R parametr bilan $y_2 \equiv y^{|r'|} \pmod p$ ni hisoblang.

6) $z_1 \equiv z_0 + (1 + z_0 R) y_2 \pmod p$ ni hisoblang.

7) $y_3 \equiv z_1 + (1+z_1 R) r \pmod p$ ni hisoblang.

8) agar $\mu = 0$ va $m = y_3$ bolsa, unda chiqishga „imzo haqiqiy" ni bering; agar $\mu = 1$ va $m = y_3$, unda keyingi qadamga oting, agar $m \neq y_3$, unda „imzo haqiqiy emas" deb qabul qilinadi;

9) $g_3 \equiv z_1 R_1^{-1} \pmod p$ ni hisoblang;

10) $s_1 \equiv s R_1 \pmod q$ ni hisoblang;

11) $r_1 \equiv R_1 + r'(1+RR_1) \pmod q$ ni hisoblang;

12) $z_2 \equiv z R_1^{-1} \pmod p$ ni hisoblang.

13) $y_4 \equiv y_1$ ni qabul qiling.

14) RR_1 parametr bilan $z_3 \equiv z_2^{|s_1|} \pmod p$ ni hisoblang;

15) RR_1 parametr bilan $y_5 \equiv y_4^{|r_1|} \pmod p$ ni hisoblang;

16) $g_4 \equiv z_3 + (1+z_3 RR_1) y_5 \pmod p$ ni hisoblang;

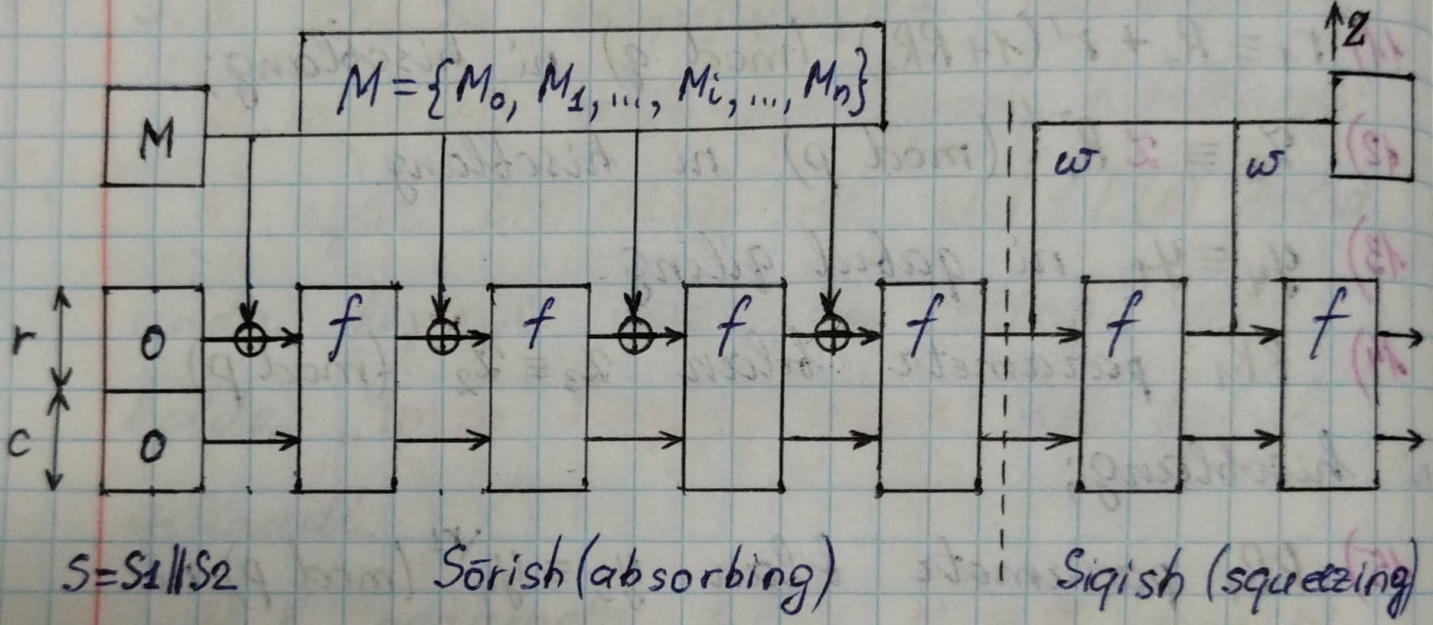
17) agar $g_3 = g_4$, unda "imzo haqiqiy" deb qabul qilinadi, aks holda „imzo haqiqiy emas" deb qabul qilinadi.

01.04.2024

SHA-3 xesh funksiya algoritmi:

SHA-0 — 1993
SHA-1 — 1995
SHA-2 — 2002
SHA-3 — 2012

SHA-3 tanlovining g'olib Keccak algoritmi kriptografik "ГУБКА" ("sponge") prinsipiga asoslangan:



Chiqishda S ning w biti Z chiqishga beriladi va S yana f funksiyadan o'tadi, Z xesh qiymat kerakli uzunlikkacha konkatenatsiya qilib yig'ib boriladi. Keccak algoritmi f funksiyasi uzunligi

$b=\{f\text{-}25,\ f\text{-}50,\ f\text{-}100,\ f\text{-}200,\ f\text{-}400,\ f\text{-}800,\ f\text{-}1600\}$ $(b=r+c)$ bit bolgan ma'lumotlar ustida amallar bajarishi mumkin. w ning qiymati f funksiyaga kiruvchi bit uzunligiga mos ravishda $w=\{1,2,4,8,16,32,64\}$, Masalan f-1600 uchun $w=64$ bit.

Belgilashlar

$A[x,y]$ - olchami 5×5 bolgan holat massivi, $A[x,y]=64$ bit, $x=0...4$, $y=0...4$;

$B[x,y]$ - olchami 5×5 bolgan vaqtinchalik massiv, strukturasi boyicha holat massivining analogi hisoblanadi, $B[x,y]=64$ bit, $x=0...4$, $y=0...4$;

$C[x]$ va $D[x]$ - bir olchovli vaqtinchalik massivlar, $C[x]=64$ bit, $D[x]=64$ bit, $x=0...4$;

$R[x,y]$ - konstanta qilib berilgan ikki olchovli massiv, har bir sozni nechchi bitga sdvig qilish kerakligini bildiradi, $x=[0...4]$, $y=0...4$;

& – bit bo'yicha ko'paytirish (and) amali;

~x – x ga bit bo'yicha inversiya (not) amali.

SHA-3 f funksiyasi

1. Uzunligi 1600 bit bo'lgan S ma'lumot A (5×5) massivga o'zlashtiriladi. Bunda har bir $A(i,j) \in V_{64}$ bit uzunlikka ega.

2. θ-akslantirish: Diffuziyaga yo'naltirilgan chiziqli akslantirish hisoblanadi.

$$C[x] = A[x,0] \oplus A[x,1] \oplus A[x,2] \oplus A[x,3] \oplus A[x,4], \quad x=0,\dots,4$$

$$D[x] = (C[x-1] \oplus (C[x+1] \ggg 1)) \pmod 5, \quad x=0,\dots,4;$$

$$A[x,y] = A[x,y] \oplus D[x], \quad x=0,\dots,4, \quad y=0,\dots,4;$$

3. ρ va π akslantirish. Dispersiyaga yo'naltirilgan chiziqli akslantirish hisoblanadi.

$$B[y, 2x+3y] = A[x,y] \ggg R[x,y], \quad x=0\dots4, \quad y=0\dots4;$$

4. χ-akslantirish. Yagona chiziqsiz akslantirish hisoblanadi.

$A[x,y] = B[x,y] (\sim B[x+1, y] \& B[x+2, y])$, $x = 0..4$, $y = 0..4$;

5. ι akslantirish. Matnga xesh qiymati ustma-ust tushadigan namunaviy matn (прообраз) topish usulidan himoya qilish uchun raund konstantasi qoshiladi.

$A[0,0] = A[0,0] \oplus RC$

bu yerda RC 64 bit uzunlikdagi raund konstantasi.

24 ta raunddan keyin M_i blok uchun hisoblash jarayoni tugaydi va M_{i+1} blokka otiladi.

SHA-3 algoritmi ketma-ketligi

1. Kiruvchi MeV* ma'lumot uzunligi $|M_i| = r$ bit bolgan n ta blokka ajratiladi. $M = \{M_0, M_1, \ldots, M_n\}$; bu yerda r chiquvchi xesh qiymatga mos ravishda SHA-512 uchun $r = 576$, $c = 1024$ (SHA-224 uchun $r = 1156$. va $c = 448$, SHA-256 uchun $r = 1088$ va $c = 512$, SHA-384 uchun

r=832 va c=768) qilib olinadi.

2. $M \in V_{1600}$ kiruvchi ma'lumot r ga karrali qilib quyidagicha to'ldiriladi $M = M \| 0x01 \| 0x00 \| \ldots \| 0x00 \| 0x80$ (agar faqat bitta bayt yetmayotgan bo'lsa 0x81 qo'shiladi).

3. M ma'lumotni uzunligi r bit bo'lgan M_i larga ajratib ular ustida amallar bajariladi.

4. Gubka prinsipiga ko'ra $S (S = S_1 \| S_2)$ ning boshlang'ich holati 0 ga teng, $S_2 \| (S_1 \text{ xor } M_i)$ amali bajarilib 24 raundli f funksiyadan o'tadi va ushbu jarayon M_n blok tugaguncha qadar davom etadi.

5. Chiqishda S ning w biti Z chiqishga beriladi va S yana f funksiyadan o'tadi, Z xesh qiymat kerakli uzunlikkacha konketenatsiya qilib yig'ib

tc86.ru ГОСТ 34.11-2018 12÷ 13÷

boriladi. Keccak algoritmi f funksiyasi uzunligi $b=\{f\text{-}25, f\text{-}50, f\text{-}100, f\text{-}200, f\text{-}400, f\text{-}800, f\text{-}1600\}$ $(b=r+c)$ bit bolgan ma'lumotlar ustida amallar bajarishi mumkin. w ning qiymati f funksiyaga kiruvchi bit uzunligiga mos ravishda $w=\{1, 2, 4, 8, 16, 32, 64\}$, Masalan f-1600 uchun $w=64$ bit.

$$\frac{b}{25}=w \qquad l=\log_2 w \qquad R=12+2\cdot l$$

$R=12+2\cdot 6=24$ (raundlar soni).

08.04.2024

ГОСТ Р 34.11-2012

standart xesh funksiya algoritmlari.

Xesh funksiya 3 bosqichdan iborat:

1-bosqich: O'zgaruvchilarga boshlang'ich qiymatlarni o'zlashtirish:

1.1. $h := \overline{IV}$; $(IV \in V_{512})$

1.2. $N := 0^{512} \in V_{512}$

1.3. $\Sigma := 0^{512} \in V_{512}$

1.4. 2-bosqichga o'tish.

2-bosqich:

2.1. $|M| < 512$ shartni tekshirish, agar shart bajarilsa 3-bosqichga o'tadi, aks holda quyidagi ketma-ketlik bajariladi:

2.2. $m \in V_{512}$, $M : M = M' \| m$

$M' \to M$ ma'lumotning $m \in V_{512}$ qismi olingandan keyin qolgan qismi.

2.3. $h = g_N(h, m)$;

2.4. $N = Vec_{512}(Int_{512}(N) \boxplus 512)$;

2.5. $\Sigma = Vec_{512}(Int_{512}(\Sigma) \boxplus Int_{512}(m))$;

2.6. $M = M'$

2.7. 2.1. qadamga o'tish.

3-bosqich:

3.1. $m = 0^{511-|M|} \| 1 \| M$;

3.2. $h = g_N(h, m)$;

3.3. $N = Vec_{512}(Int_{512}(N) \boxplus |M|)$;

3.4. $\Sigma = Vec_{512}(Int_{512}(\Sigma) \boxplus Int_{512}(m))$;

3.5. $h = g_0(h, N)$;

3.6. $h = \begin{cases} g_0(h, \Sigma) & - 512 \text{ bitli uchun} \\ MSB_{256}(g_0(h, \Sigma)) & - 256 \text{ bitli uchun.} \end{cases}$

3.7. Tamom.

ГОСТ Р 34.11-2012 standart xesh-funksiya algoritmi.

1. Umumiy xesh qiymat quyidagi formula bilan aniqlanadi:

$$g_N(h, m) = \mathcal{E}(LPS(h \oplus N), m) \oplus h \oplus m;$$

2. Har bir M_i blok 13 ta iteratsiyadan iborat siqish funksiyasidan o'tkaziladi:

$$\mathcal{E}(K, m) = X[K_{13}]LPSX[K_{12}] \dots LPSX[K_1](m);$$

bu yerda, X, L, P, S akslantirishlar quyidagicha hisoblanadi:

• X - akslantirish.

$X[k](m): V_{512} \to V_{512}$

$X[k] = K \oplus m, \quad K, m \in V_{512}$

Bu yerda $K = LPS(k \oplus N)$,

$K_i = LPS(K_{i-1} \oplus C_{i-1}), \quad K_1 = K, \quad i = 2, \ldots, 13$

C_i - o'zgarmas; $C_i \in V_{512}$

• Ch S - chiziqsiz akslantirish.

Kiruvchi 512 bit argument 64 bayt massiv ko'rinishida qaraladi.

$S: V_{512} \to V_{512} \quad S(a) = S(a_{63} \| \ldots \| a_0) = \pi(a_{63}) \| \ldots \| \pi(a_0)$;

bu yerda:

$a = a_{63} \| a_{62} \| \ldots \| a_0 \in V_{512}, \; a_i \in V_8, \; i = 0, \ldots, 63.$

$\pi = Vec_8 \pi' Int_8 : V_8 \to V_8,$

$\pi' : Z_{2^n} \to Z_{2^n}.$

• P - baytlarni qayta tartiblash:

Baytlar o'rni belgilangan tartibda almashtiriladi. $P: V_{512} \to V_{512} \quad P(a) = P(a_{63} \| \ldots \| a_0) =$

$= a_{\tau(63)} \| \ldots \| a_{\tau(0)}$; bu yerda:

$a = a_{63} \| a_{62} \| \dots \| a_0 \in V_{512}$, $a_i \in V_8$, $i = 0, \dots, 63$;

. L -chiziqli akslantirish.

Argument 8 ta 64 bitlik vektor sifatida qaraladi. Har bir vektor 64×64 standart matritsaga ko'paytirilib, natija bilan almashtiriladi.

$L: V_{512} \to V_{512}$, $L(a) = L(a_7 \| \dots \| a_0) = l(a_7) \| \dots \| l(a_0)$;

bu yerda, $a = a_{63} \| \dots \| a_0 \in V_{512}$, $a_i \in V_8$, $i = 0, \dots, 63$;
l-chiziqli akslantirish bo'lib, $V_{64} \to V_{64}$ ga A matritsa yordamida akslantiradi.

L akslantirishdan chiqqan 512 bit (V_{512}) ma'lumot keyingi iteratsiya uchun kirish hisoblanadi. 13 ta iteratsiyadan o'tgandan keyin M_i blok uchun xesh qiymat va M_{i+1} blok uchun kiruvchi qiymat hisoblanadi. M_n - blokdan chiqqan xesh qiymat M ma'lumot uchun 512 bit xesh qiymat hisoblanadi.

GOST 34.12-2018 Davlatlararo blokli shifrlash standarti.

GOST 34.12-2018 standarti – RF Davlat xavfsizlik Xizmatining „Maxsus aloqa va axborotlarni himoyalash markazi" va „Kommunikatsiya tizimlar va axborot texnologiyalar" OAJ hamkorligida ishlab chiqilgan hamda „Texnik nazorat va metrologiya federal agentligining 2015-yil 19-iyundagi 216-sonli buyrug'i bilan tasdiqlangan va 2016-yil 1-yanvardan kuchga kirilgan GOST R 34.12-2015 standart shifrlash algoritmiga asoslangan.

GOST R 34.12-2015 standarti simmetrik shifrlash algoritmi bo'lib, blokli shifrlashni amalga oshirish uchun mo'ljallangan quyidagi algoritmlarni o'z ichiga oladi:

1) 64 bit uzunlikda bloklab shifrlovchi „Magma shifri" (GOST 28147-89

tarkibidagi algoritm).

2) 128 bitli "Kuznechik shifri"

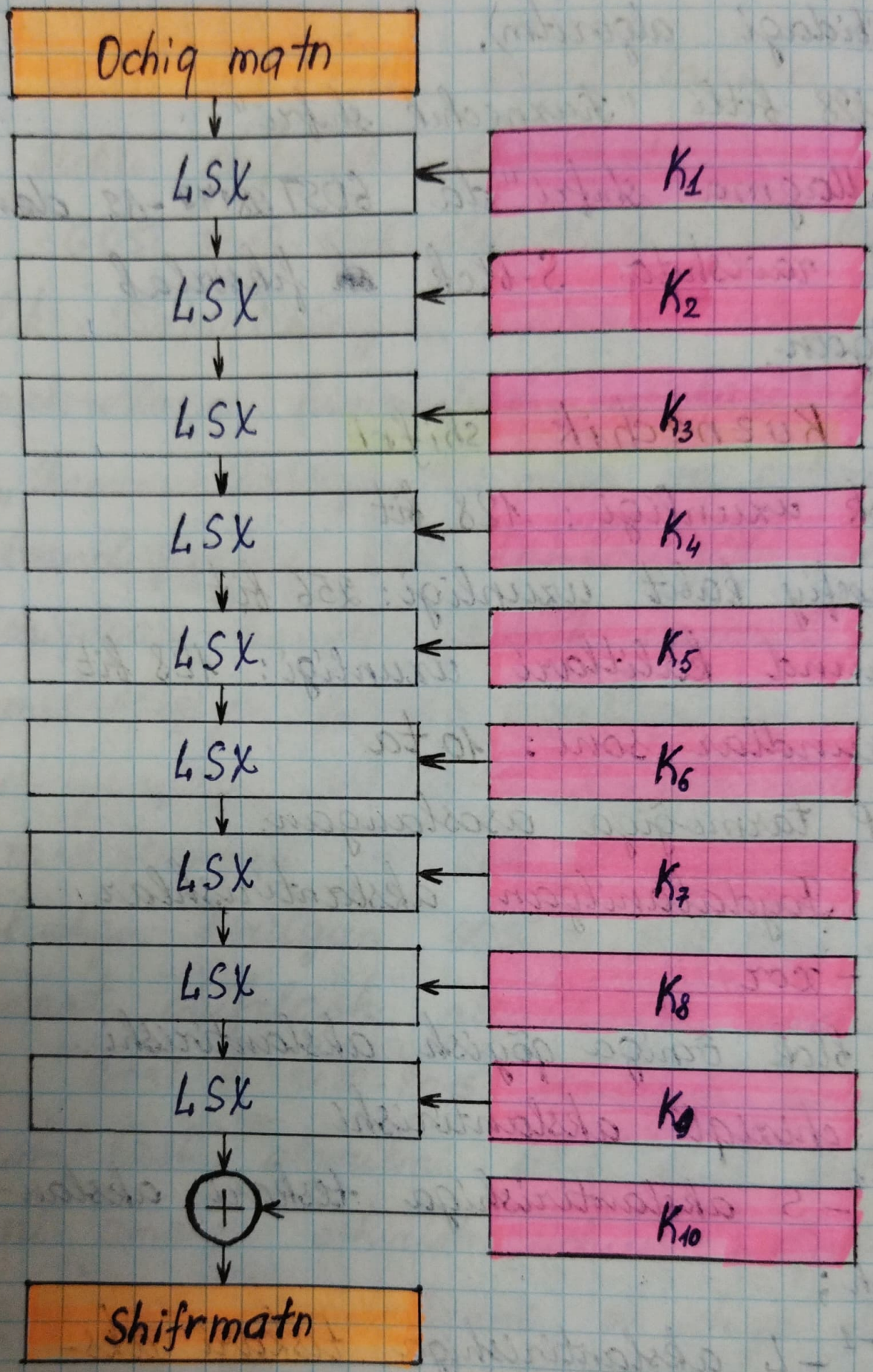
! „Magma shifri" da GOST 28147-89 dan farqli ravishda S-blok fiksirlab qoyilgan.

Kuznechik shifri

- blok uzunligi: 128 bit
- maxfiy kalit uzunligi: 256 bit
- raund kalitlari uzunligi: 128 bit
- raundlar soni: 10 ta
- SP tarmoğiga asoslangan.

Foydalanilgan akslantirishlar.

- X - xor
- S blok ornida qoyish akslantirishi.
- L chiziqli akslantirishi
- S^{-1} - S akslantirishiga teskari akslantirish;
- L^{-1} - L akslantirishiga teskari akslantirish.


Ochiq matn
LSX
LSX
LSX
LSX
LSX
LSX
LSX
LSX
LSX
K1
K2
K3
K4
K5
K6
K7
K8
K9
K10
Shifrmatn

S akslantirishi - chiziqsiz akslantirishi hisoblanib, kiruvchi 128 bitlik qiymatni 128 bitlik qiymatga o'zgartiradi. Buni almashtirish π massividan foydalanib amalga oshiriladi.

L akslantirishi chiziqli akslantirish bo'lib, qisqa masofada maksimal aralashtirish (Maximum Distance Separate) ya'ni MDS prinsipiga asoslanadi. Bu akslantirish 128 bitlik kiruvchi ma'lumot ustida R akslantirishni ketma-ket 16 marta bajarilishini o'z ichiga oladi: $L(a) = R^{16}(a)$.

R akslantirishida ko'phadlar ustida quyidagicha amallar bajariladi:

$$R(a) = R(a_{15} \| \dots \| a_0) = l(a_{15}, \dots, a_0) \| a_{15} \| \dots \| a_1$$

bu yerda $l(a)$ ifoda quyidagicha aniqlanadi:

$$l(a_{15}, \dots, a_0) = (\{148\} \cdot a_5 + \{32\} \cdot a_{14} + \{133\} \cdot a_{13} + \{16\} \cdot a_{12} +$$

$+\{194\}\cdot a_{11} + \{192\}\cdot a_{10} + \{01\}\cdot a_9 + \{251\}\cdot a_8 + \{01\}\cdot a_7 +$
$+\{192\}\cdot a_6 + \{16\}\cdot a_4 + \{133\}\cdot a_3 + \{32\}\cdot a_2 + \{148\}\cdot a_1 +$
$+\{01\}\cdot a_0) \bmod p(x)$

bunda a_i $(i \in 15 \ldots 0)$ 128 bitlik kirish blokining 16 ta 8 bitlik bloklarga bolinishidan hosil bolgan i-blok. Har bir a_i ni sonlarga kopaytirish kophadlar algebrasi boyicha hisoblanadi.

Raund kalitlari generatsiyasi

1. Seans kaliti $K = (k_{255}, \ldots, k_0)$ ikki qismga ajratiladi:

$K_1 = k_{255} \| \ldots \| k_{128}$, $K_2 = k_{127} \| \ldots \| k_0$

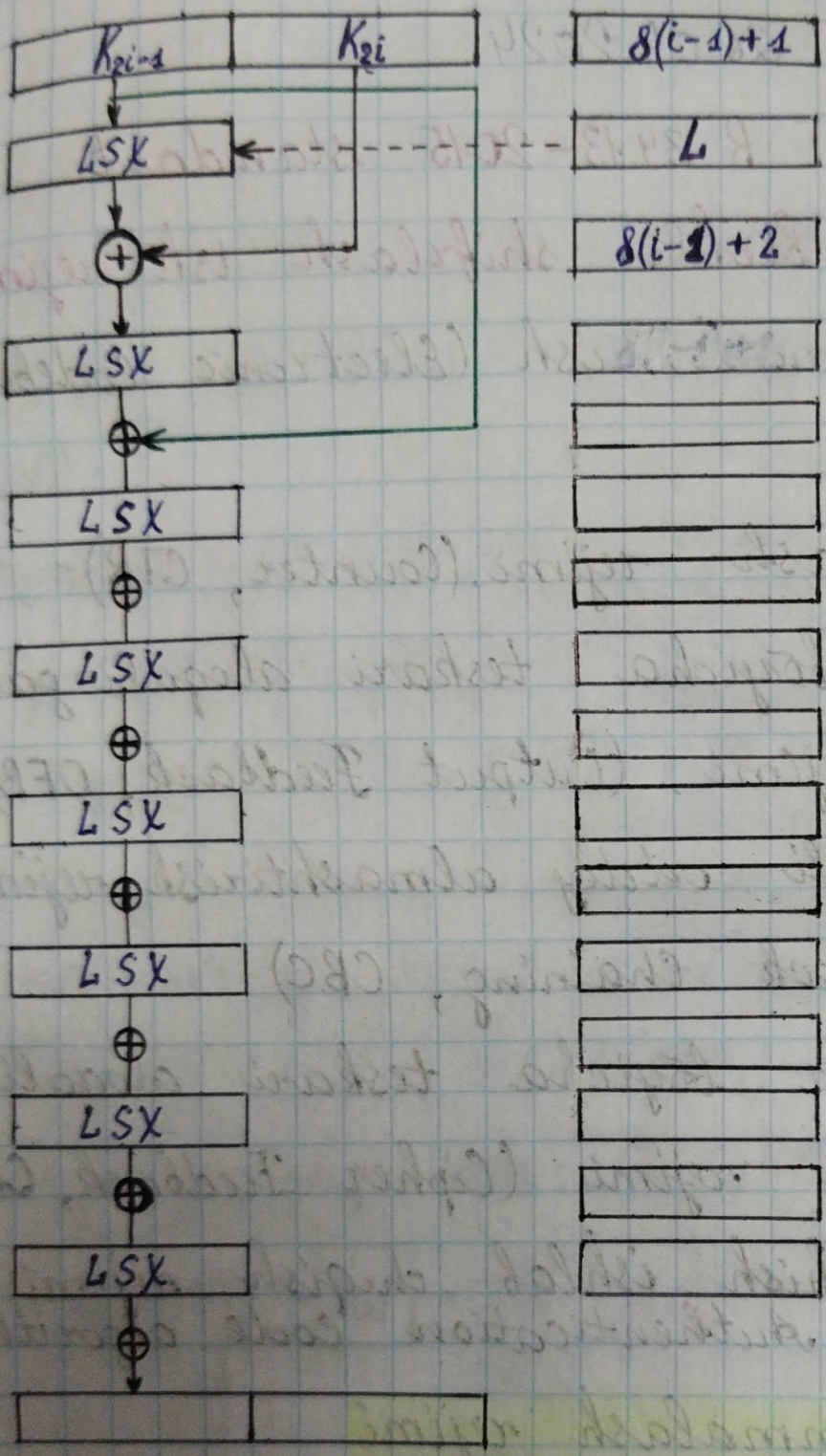
2. Keyingi ikki raund kaliti quyidagicha ishlab chiqiladi:

$(K_{2i+1}, K_{2i+2}) = F[C_{8(i-1)+8}] \ldots F[C_{8(i-1)+1}](K_{2i-1}, K_{2i})$,
$i = 1, 2, 3, 4$.

bu yerda

$C_i = L(i)$, $i = 1, 2, \ldots, 32$

$F[k](a_1, a_0) = (LSX[k](a_1) \oplus a_0, a_1)$


K_{2i-1}
K_{2i}
$8(i-1)+1$
LSX
L
$8(i-1)+2$
LSX
LSX
LSX
LSX
LSX
LSX
LSX

26.04.2024

GOST R 34.13-2015 standart bo'yicha blokli shifrlash ish rejimlari

1. Oddiy almashtirish (Electronic Codebook, ECB) rejimi.
2. Gammalash rejimi. (Counter, CTR)
3. Chiqish bo'yicha teskari aloqali gammalash rejimi. (Output Feedback, OFB)
4. Uzatishli oddiy almashtirish rejimi (Cipher Block Chaining, CBC)
5. Shifrmatn bo'yicha teskari aloqali gammalash rejimi (Cipher Feedback, CFB)
6. Imitoqo'shish ishlab chiqish rejimi (Message Authentication Code algorithm).

2. Gammalash rejimi (Counter, CTR)

Shifrlash jarayoni:

P - ochiq matn quyidagi ko'rinishda bloklarga ajratiladi:

$$P = P_1 \| P_2 \| \dots \| P_q,$$

P_i – s bit, $i = 1, 2, \dots, q-1$,

P_q – r bit, $r \le s$.

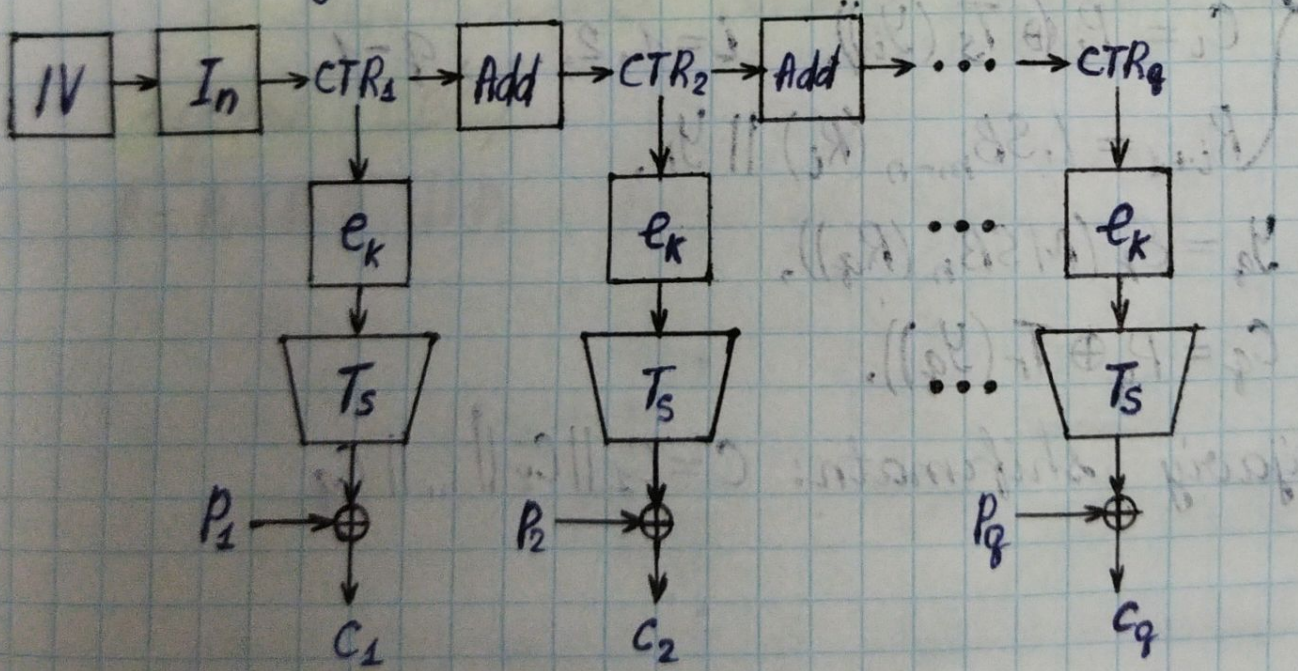
r-ochiq matnning oxirgi bloki uzunligi. Shifrmatn bloklari quyidagicha hisoblanadi:

$$\begin{cases} C_i = P_i \oplus T_s(E_k(CTR_i)), & i = 1, 2, \dots, q-1, \\ C_q = P_q \oplus T_r(E_k(CTR_q)). \end{cases}$$

Bu yerda $CTR_1 = I_n(IV) = \underline{IV} \| 0^{n/2}$,

IV – sinxrojõnatma, T_s, T_r – mos holda kiruvchi qiymatning kichik s va r bitini olish funksiyasi, $CTR_{i+1} = Add(CTR_i) = CTR_i \boxplus_n 1$

Natijaviy shifrmatn: $C = C_1 \| C_2 \| \dots \| C_q$



Deshifrlash jarayoni:

C va P lar orni almashadi (sxemada ham).

3. Chiqish bo'yicha teskari aloqali gammalash rejimi.

Shifrlash jarayoni:

$P = P_1 \| P_2 \| \dots \| P_q$, P_i - s bit, $i = 1, 2, \dots, q-1$.

P_q - r bit, $r \le s$.

Shifrmatn ~~(bo'yicha)~~ bloklari quyidagi qoida bo'yicha hisoblanadi:

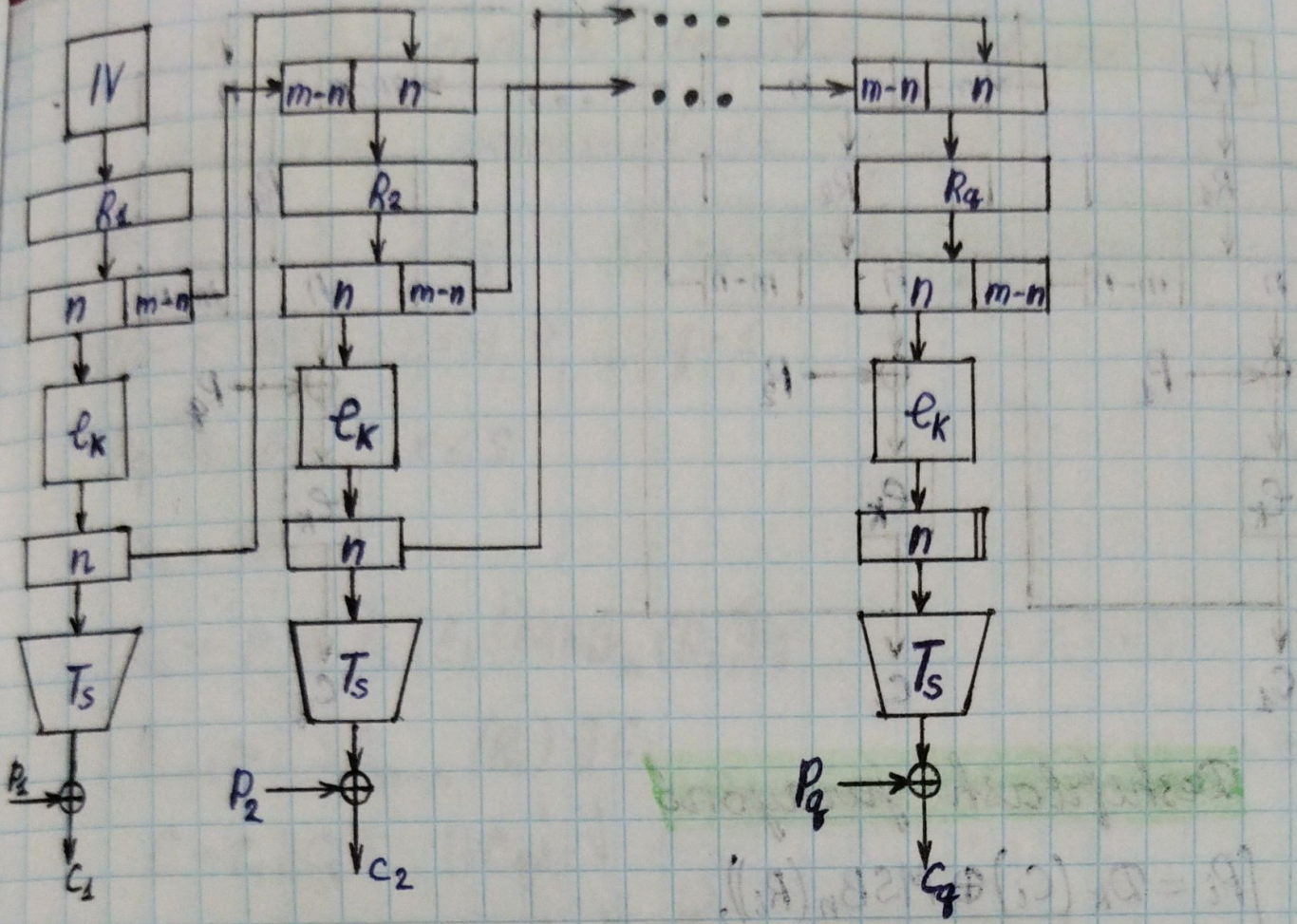
$R_1 = IV$.

$$\begin{cases} Y_i = \mathcal{E}_k(MSB_n(R_i)) \\ C_i = P_i \oplus T_s(Y_i)), \quad i = 1, 2, \dots, q-1. \\ R_{i+1} = LSB_{m-n}(R_i) \| Y_i. \end{cases}$$

$Y_q = \mathcal{E}_k(MSB_n(R_q))$.

$C_q = P_q \oplus T_r(Y_q))$.

Natijaviy shifrmatn: $C = C_1 \| C_2 \| \dots \| C_q$



Deshifrlash jarayoni

C va P lar o'rni almashadi.

4. Uzaktirishli oddiy almashtirish rejimi (CBC)

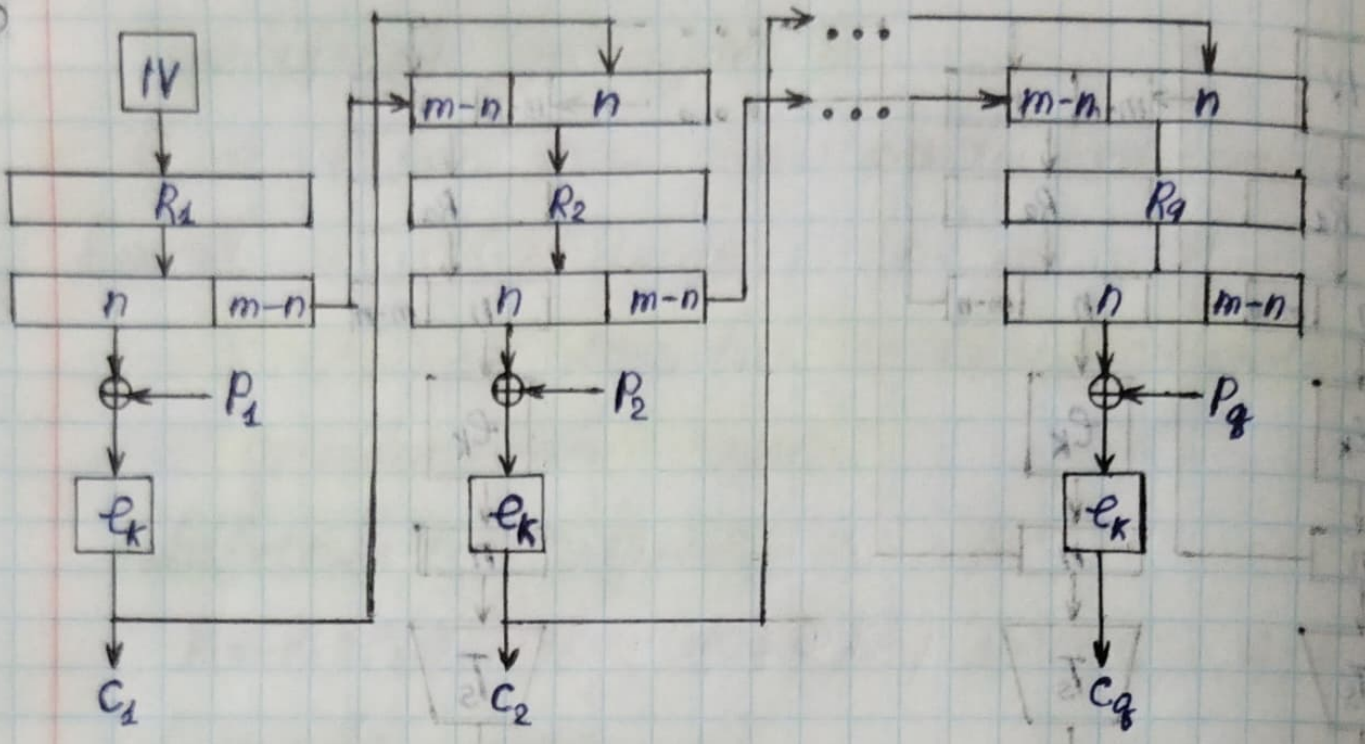
$P = P_1 \| P_2 \| \dots \| P_n$, $P_i = n$ bit, $i = 1, 2, \dots, q$.

$R_1 = IV$

$$\begin{cases} C_i = \mathcal{E}_k(P_i \oplus MSB_n(R_i)), \\ R_{i+1} = LSB_{m-n}(R_i) \| C_i \end{cases}$$

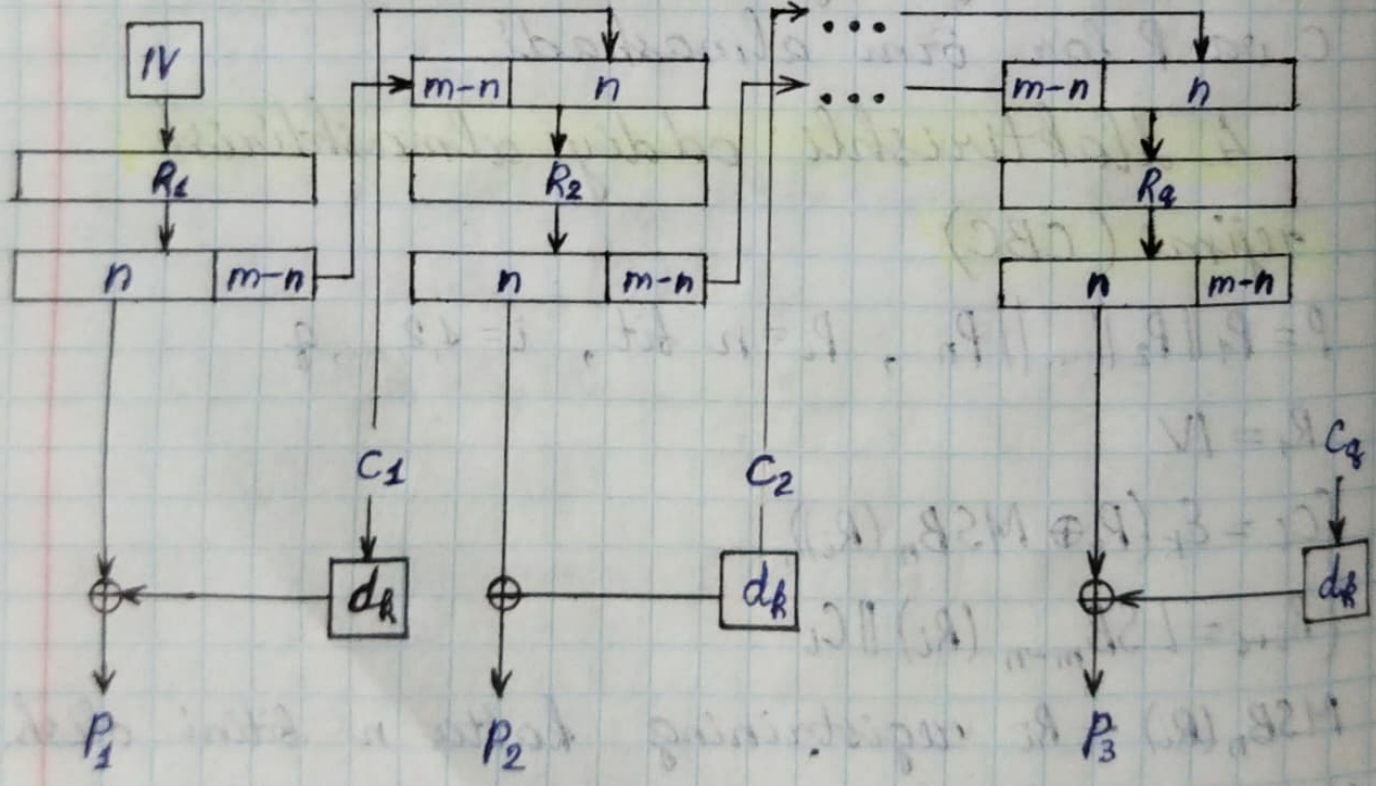
$MSB_n(R_i)$ – R_i registrining katta n bitini olish.

Natijaviy shifrmatn: $C = C_1 \| C_2 \| \dots \| C_q$



Deshifrlash jarayoni

$$\begin{cases} P_i = D_K(C_i) \oplus MSB_n(R_i), \\ R_{i+1} = LSB_{m-n}(R_i) \,||\, C_i \end{cases}$$



$$P = P_1 \,||\, P_2 \,||\, \ldots \,||\, P_q$$

5. Shifrmatn boyicha teskari aloqali gammalash rejimi.

$P = P_1 \| P_2 \| \dots \| P_q$ Shifrlash jarayoni

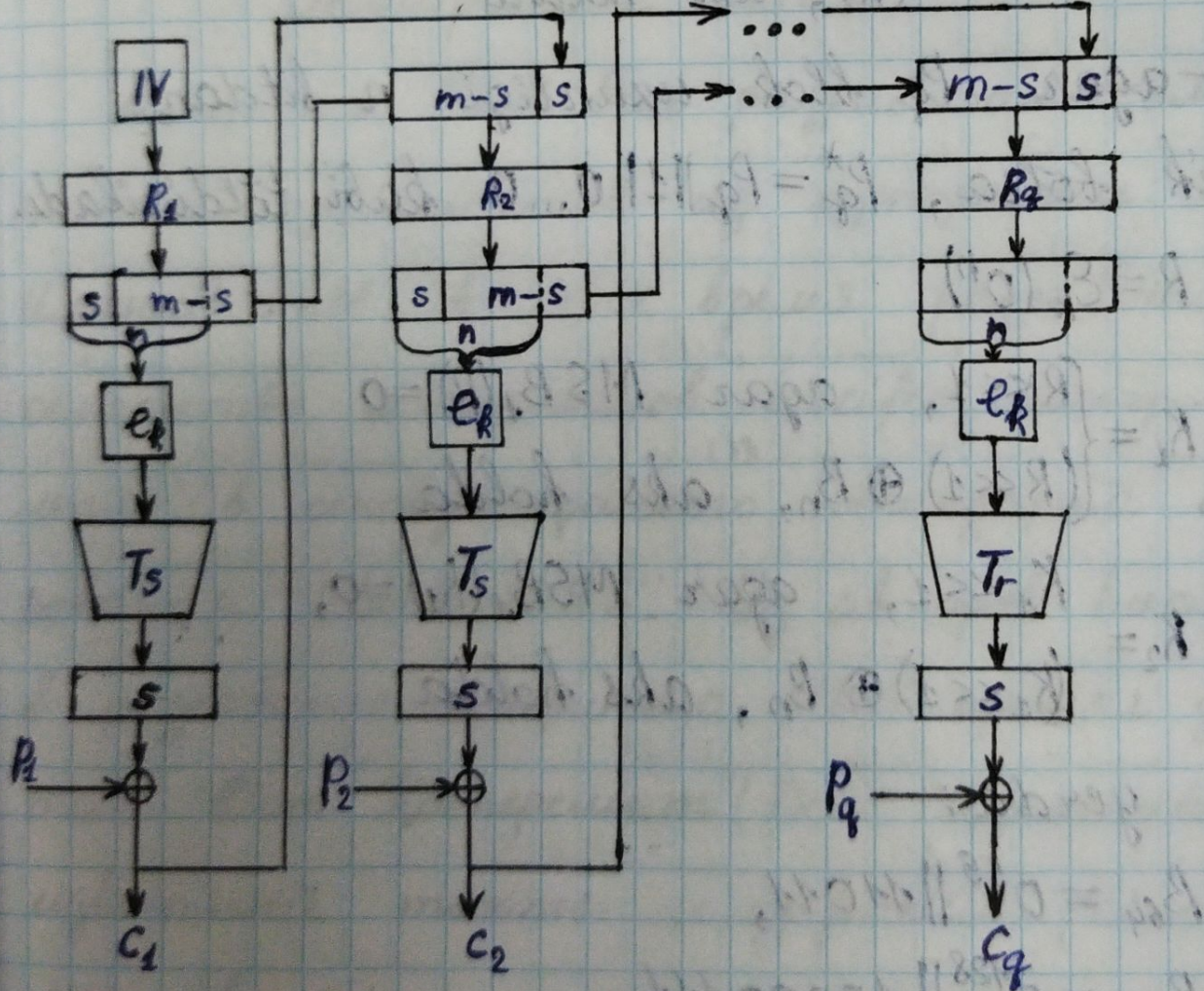
P_i - s bit, $i = 1, 2, \dots, q-1$

P_q - r bit. $r \le s$

$R_1 = IV$

$$\begin{cases} C_i = P_i \oplus T_s(\mathcal{E}_k(MSB_n(R_i))); \\ R_{i+1} = LSB_{m-n}(R_i) \| C_i \end{cases}$$

$C = C_1 \| C_2 \| \dots \| C_q$.



Deshifrlash jarayoni

Pora C lar o'rni almashadi.

6. Imitoqo'shish ishlab chiqish rejimi

$$\begin{cases} C_0 = 0^n \\ C_i = \mathcal{E}_k(P_i \oplus C_{i-1}), \ i = 1, 2, \dots, q-1 \\ MAC = T_s(\mathcal{E}_k(P_q^* \oplus C_{q-1} \oplus K^*)) \end{cases}$$

bu yerda,

$$K^* = \begin{cases} K_1, \text{ agar } |P_q| = n, \\ K_2, \text{ aks holda} \end{cases}$$

P_q^* - agar P_q blok - uzunligi n bitdan kichik bo'lsa, $P_q^* = P_q \| 1 \| 0 \dots 0$ kabi to'ldiriladi.

$$R = \mathcal{E}_k(0^n)$$

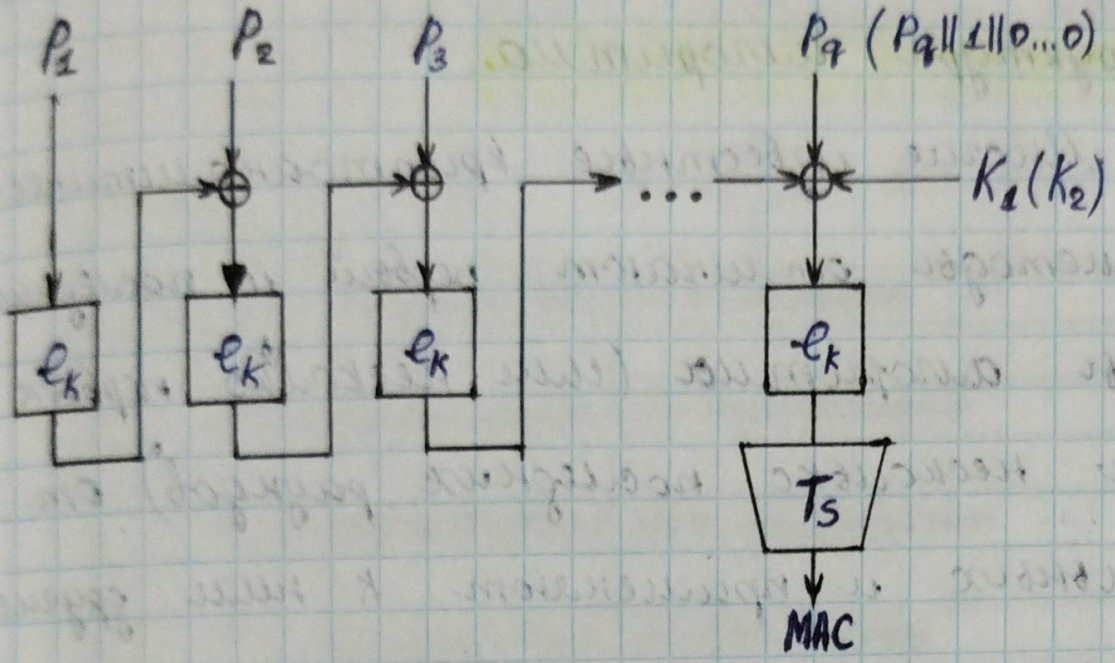
$$K_1 = \begin{cases} R \ll 1, & \text{agar } MSB_1(R) = 0 \\ (R \ll 1) \oplus B_n, & \text{aks holda} \end{cases}$$

$$K_2 = \begin{cases} K_1 \ll 1, & \text{agar } MSB_1(K_1) = 0, \\ (K_1 \ll 1) \oplus B_n, & \text{aks holda.} \end{cases}$$

bu yerda:

$B_{64} = 0^{59} \| 11011,$

$B_{128} = 0^{128} \| 10000111$



Алгоритм MARS

Алгоритм MARS был разработан коллективом криптологов из корпорации IBM. Именно IBM в своё время разработала семейство алгоритмов Lucifer, которое легло в основу прошлого стандарта шифрования США-DES. Из авторов Lucifer в разработке алгоритма MARS принял участие Дон Копперсмит (Don Coppersmith), известный также и другими работами в области криптологии.

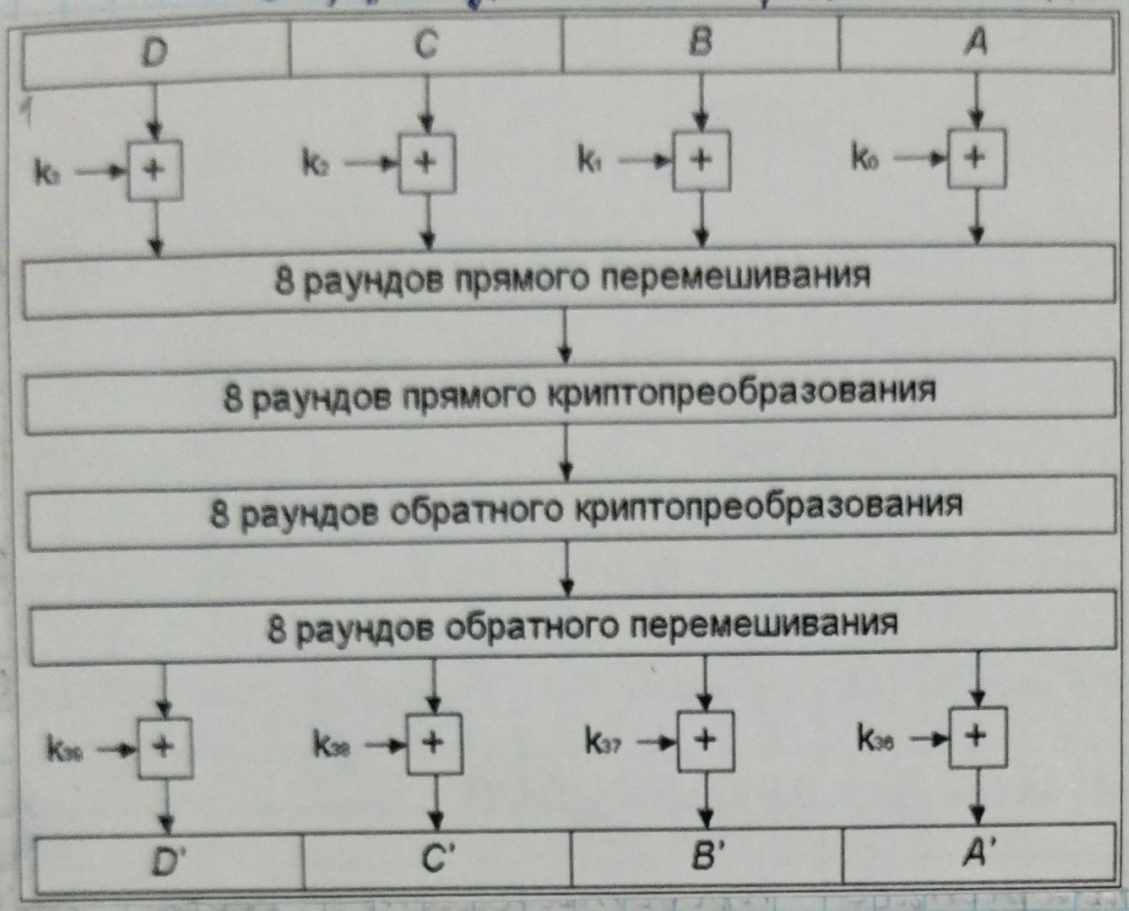
Структура алгоритма.

1. Многие известные криптоаналитические методы отличают первый и последний раунды алгоритма (или несколько первых и/или несколько последних раундов) от остальных и применяют к ним другие приемы, нежели к "центральным" раундам алгоритма. Таким образом, различные раунды алгоритма шифрования играют различное значение в обеспечиваемой алгоритмом криптостойкости.

2. Скорее всего, алгоритм с гетерогенной структурой будет лучше противостоять криптоаналитическим методам будущего, чем алгоритм, все раунды которого идентичны.

Создатель – Керолин Барвик,
Фон Копперсмит.
создан и опубликован – 1998 г
Размер ключа – 128–448 бит (32·X)
Размер блока – 128 бит.
Число раундов – 32
Тип – сеть Фейстеля.

Структура алгоритма MARS



1. Предварительное наложение ключа: на 32-битные субблоки A, B, C, D накладываются 4 фрагмента расширенного ключа $k_0 \dots k_3$ операцией сложения по модулю 2^{32}.

2. Выполняются 8 раундов прямого перемешивания (без участия ключа шифрования).

3. Выполняются 8 раундов прямого криптопреобразования.

4. Выполняются 8 раундов обратного криптопреобразования. Этапы 3 и 4 на-

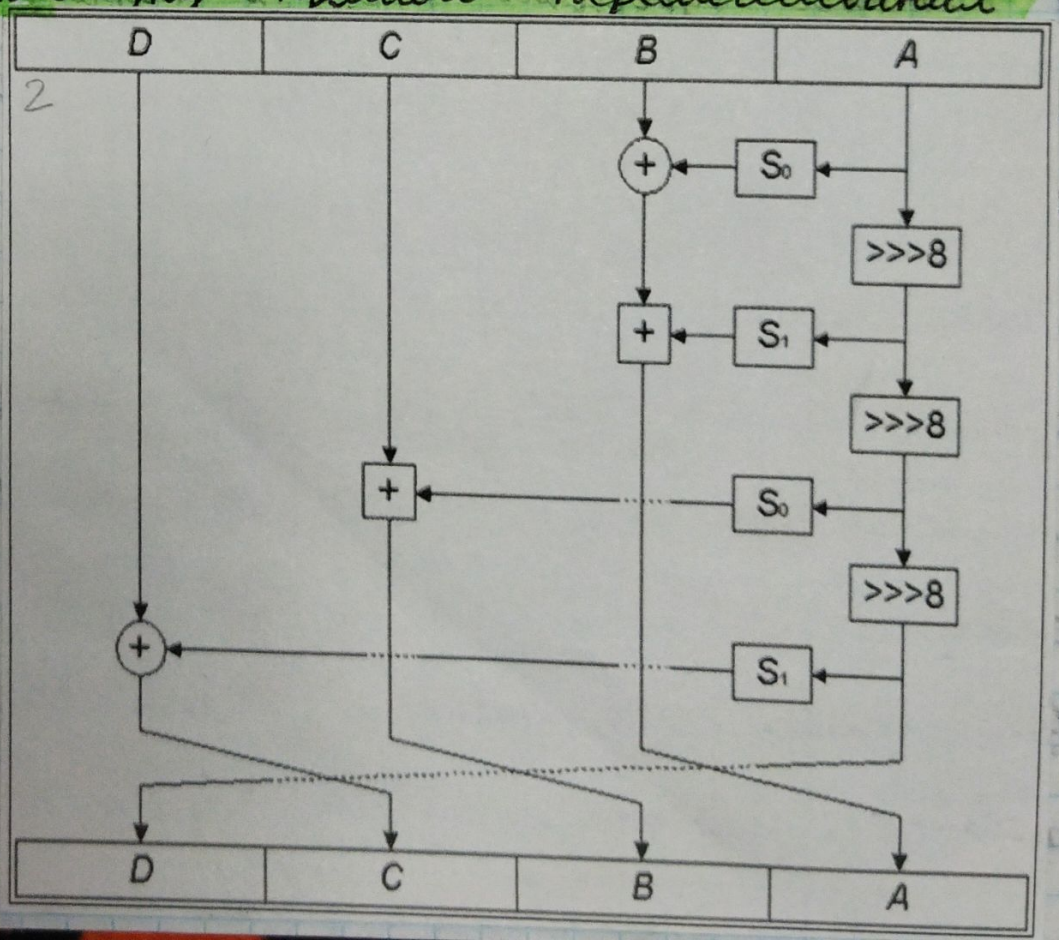
зываются "криптографическим ядром" алгоритма MARS.

5. Выполняются 8 раундов обратного перемешивания, также без участия ключа шифрования.

6. Финальное наложение фрагментов расширенного ключа $k_{36} \dots k_{39}$ операцией вычитания по модулю 2^{32}.

Алгоритм представляет собой расширенную сеть Фейстеля.

1. Раунд прямого перемешивания



1. Значение субблока A прогоняется через таблицу замен S_0 и накладывается на субблок B операцией XOR.

2. Исходное значение субблока A вращается на 8 бит вправо.

3. Результат предыдущего шага обрабатывается таблицей замен S_1 и снова накладывается на субблок B операцией сложения по модулю 2^{32}.

4. Результат шага 2 вращается на 8 бит вправо.

5. Результат предыдущего шага обрабатывается таблицей замен S_0 и накладывается на субблок C операцией сложения по модулю 2^{32}.

6. Результат шага 4 вращается на 8 бит вправо.

7. Результат предыдущего шага обрабатывается таблицей замен S_1 и накладывается на субблок D операцией XOR.

8. Субблоки меняются местами.

Кроме того, в некоторых раундах прямого перемешивания выполняются следующие дополнительные операции (не приведены на рис.):

1. В раундах 0 и 4 после шага 7 выполняется наложение значения субблока D на субблок A операцией сложения по модулю 2^{32}.

2. В раундах 1 и 5 субблок B аналогичным образом накладывается на субблок A.

По словам авторов алгоритма, эти операции существенно усиливают алгоритм MARS против дифференциального криптоанализа.

Таблицы замен S_0 и S_1

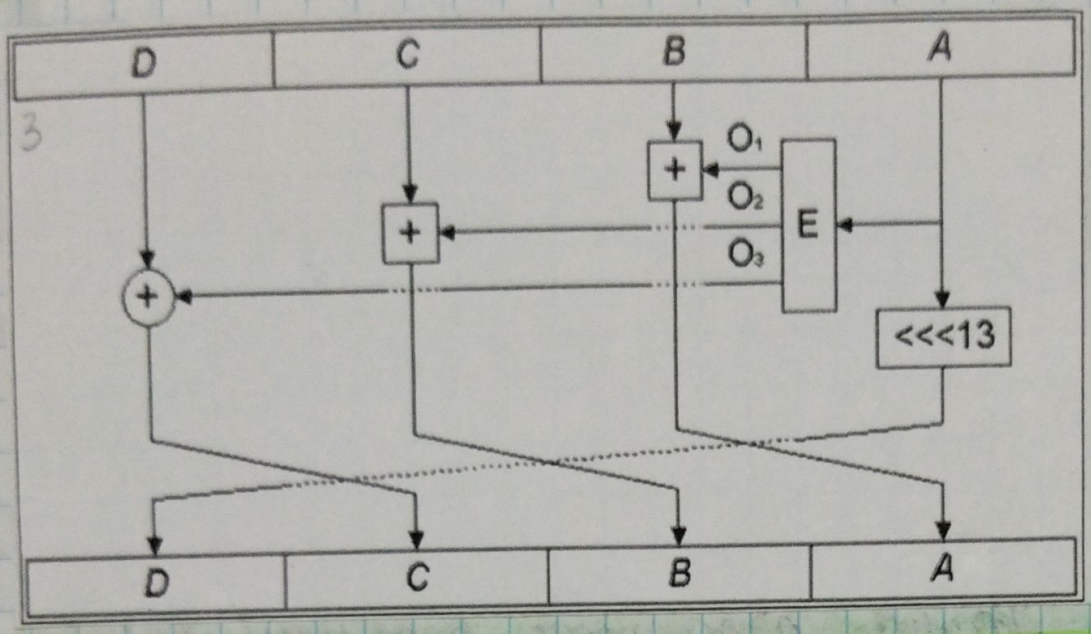
$S_0[0] = 09D0C479$ $\quad$ $S_1[0] = 6B57E354$

$S_0[1] = 28C8FFE0$ $\quad$ $S_1[1] = AD913CF7$

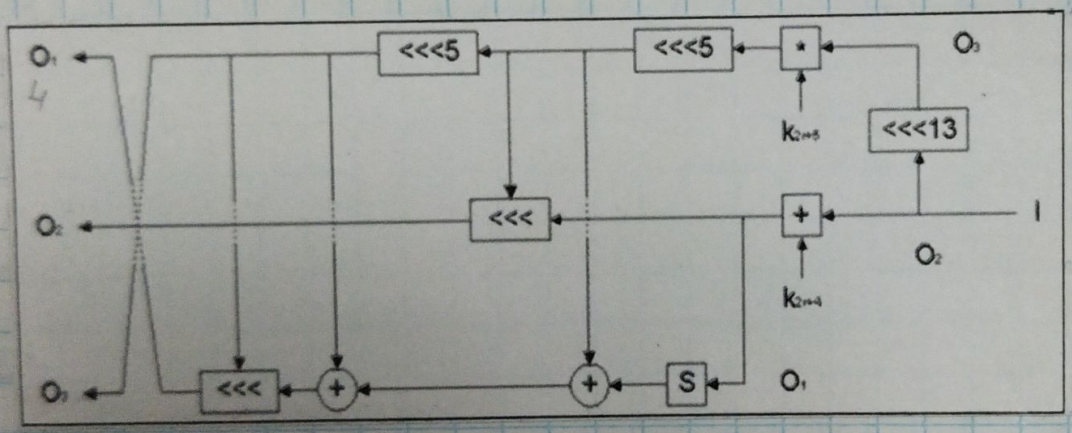
... $\quad$ ...

$S_0[255] = 9E5FD030$ $\quad$ $S_1[255] = 13AF70EE$

2. Структура раунда прямого криптопреобразования



Расширяющее криптопреобразование E (E – Expansion)

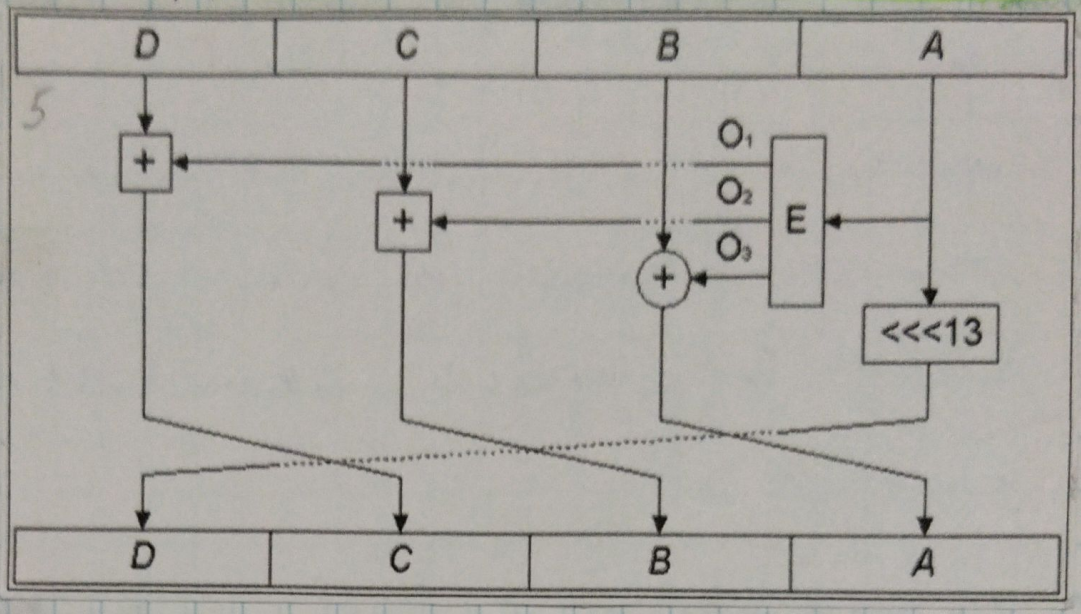


Из входного значения формируется три потока $O_1 \dots O_3$, над которыми производятся следующие действия:

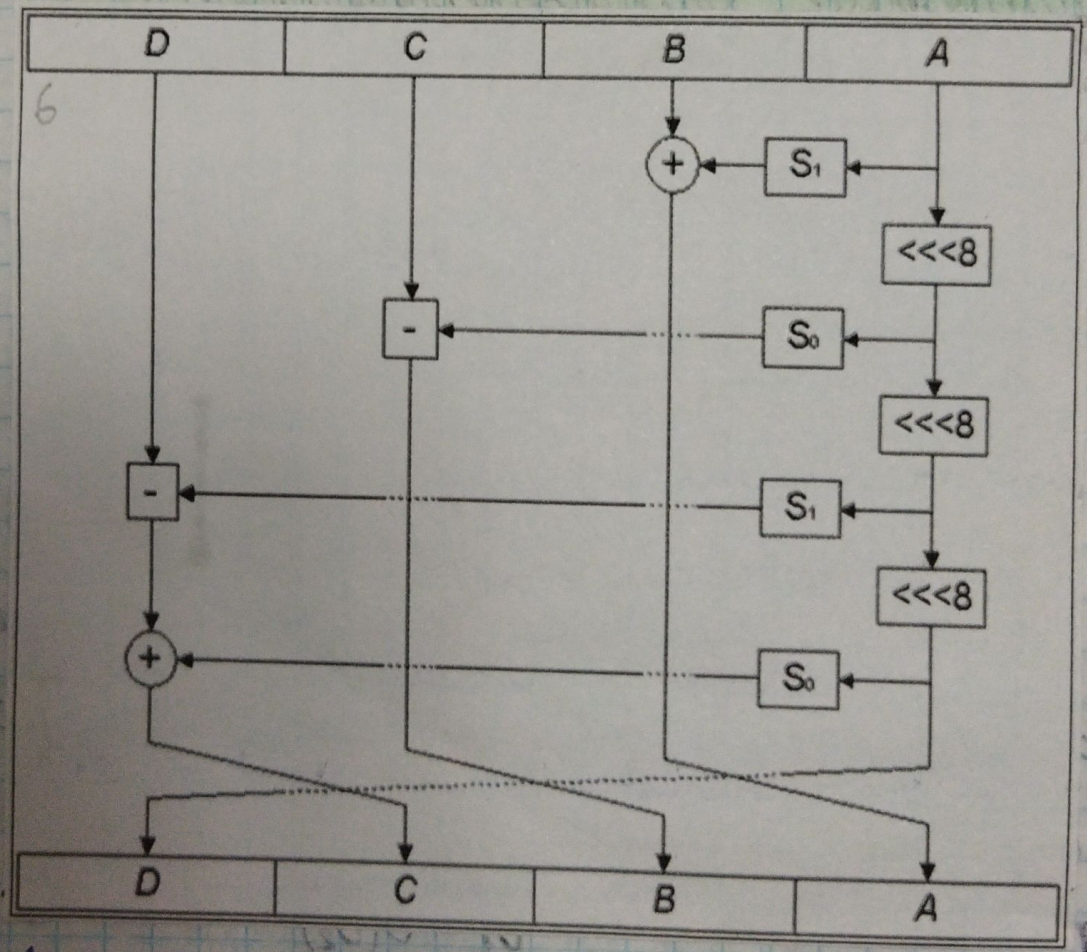
$O_2 = I$
$O_3 = O_2 <<< 13$
$O_2 = O_2 + k_{2r+4} \bmod 2^{32}$
$O_3 = O_3 * k_{2r+5} \bmod 2^{32}$
$O_3 = O_3 <<< 5$

$O_1 = S(O_2)$
$O_1 = O_1 \oplus O_3$
$O_2 = O_2 <<< O_3$
$O_3 = O_3 <<< 5$
$O_1 = O_1 \oplus O_3$
$O_1 = O_1 <<< O_3$

3. Структура обратного криптораунда



4. Раунд обратного перемешивания



Аналогично прямому перемешиванию, в некоторых раундах выполняются следу

ющие дополнительные операции не показанные на рис.

1. В раундах 1 и 5 после шага 7 выполняется наложение значения субблока A на субблок B операцией вычитания по модулю 2^{32}

2. В раундах 3 и 6 субблок C аналогичным образом накладывается на субблок B.

Расшифрование

Расшифрование выполняется применением обратных операций в обратной последовательности.

Процедура расширения ключа

Ключ шифрования алгоритма MARS может иметь любой размер в диапазоне от 128 до 448 бит включительно, кратный 32 битам. Расширение ключа представляет собой формирование на основе ключа шифрования 40 подключей по

32 бита каждый (причем, подключи должны обладать) определёнными характеристиками, о которых будет сказано ниже, для чего выполняются следующие операции:

1. Формируется временный массив $T_0 \ldots T_{14}$

$$T_0 = KI_0$$
$$T_1 = KI_1$$
$$\ldots$$
$$T_{n-1} = KI_{n-1}$$
$$T_n = n$$
$$T_{n+1} = T_{n+2} = \ldots = T_{14} = 0$$

где $KI_0 \ldots KI_{n-1}$ – исходный ключ шифрования, n – его размер в 32-битных словах – от 4 до 14 включительно.

2. Данный шаг повторяется 4 раза (каждая итерация дает 10 вычисленных фрагментов расширенного ключа) и содержит следующие операции:

1) линейное преобразование:

$$T_i = T_i \oplus ((T_{i-7 \bmod 15} \oplus T_{i-2 \bmod 15}) \lll 3) \oplus (4i + j)$$

где j – номер итерации, начиная с 0, $i = 0 \ldots 14$

2) перемешивание массива T:

$$T_i = (T_i \oplus S(T_{i-1 \bmod 15})) \lll 9$$

где S – табличная замена, выполняемая по тем же правилам, что и в операции E;

3) заключительная перестановка:

$$k_{10j+n} = T_{4n \bmod 15}$$

где $n = 0 \ldots 9$

3. Наложение на вычисленные подключи дополнительных требований, состоящих в следующем:

1) Каждый подключ, используемый для умножения в операции E (то есть подключи с нечётными индексами от k_5 до k_{35} включительно), должен иметь нечетное значение. Мало того, единичными должны быть два младших бита подключа, а не один.

2) Вычисляется 32-битная маска M, которая будет использована для модификации подключа – для обеспечения отсутствия в нем 10 подряд нулевых или единичных бит. Маска вычисляется следующим образом:

A) Устанавливаются в 1 биты M, соответствующие тем битам обрабатываемого подключа, которые входят в последовательности из 10 нулевых или 10 единичных бит. Остальные биты устанавливаются в 0.

B) Обнуляются те единичные биты маски M, которые соответствуют любому из условий:

$i < 2$

$i = 31$

$w_i != w_{i-1}$

$w_i != w_{i+1}$

где i – номер бита, начиная с 0, а w_i – значение i-го бита.

3) Используется таблица корректирующих значений B, определённая следующим образом:

$$B_0 = A4A8D57B,$$
$$B_1 = 5B5D193B,$$
$$B_2 = C8A8309B,$$
$$B_3 = 73F9A978$$

Элементы таблицы B эквивалентны элементам с 265-го по 268-й включительно объединенной таблицы S; именно эти элементы используются для коррекции подключей благодаря их специфическим свойствам.

С помощью данной таблицы следующим образом вычисляется финальное значение подключа K_i:

$$K_i = w \oplus ((B_j <<< K_{i-1}) \,\&\, M)$$

где & – логическая побитовая операция "и", а вращение B_j определяется пятью младшими битами предыдущего подключа K_i

Описанная процедура гарантирует требуемые свойства у подключей.

Алгоритм RC6

Алгоритм RC6 был разработан в 1998 г. рядом специалистов научного подразделения известнейшей фирмы RSA Data Security – RSA Laboratories: Ronald Rivest (основатель RSA Data Security), Matt Robshaw, Ray Sidney, Yiqun Lisa Yin специально для участия в конкурсе AES. Алгоритм во многом унаследовал черты предыдущего алгоритма Рональда Ривеста – 64-битного блочного шифра RC5, разработанного в 1997 г. Фактически, алгоритм претерпел два принципиальных изменения:

- в отличие от RC5 в алгоритме используется умножение по модулю 2^{32}.
- для сохранения 32-битных вычислений вместо разбиения шифруемого блока данных (128 бит согласно принципиальному требованию конкурса AES) на два 64-битных субблока выполняется его разбиение на 4 32-битных субблока и их обработка по несколько изменённой схеме.

Размер ключа: 128, 192 или 256 бит (от 0 до 2040 бит).

Размер блока: 128 бит (для 32-разрядных платформ).

Число раундов: 20 (по умолчанию)

Тип: Сеть Фейстеля.

Структура алгоритма

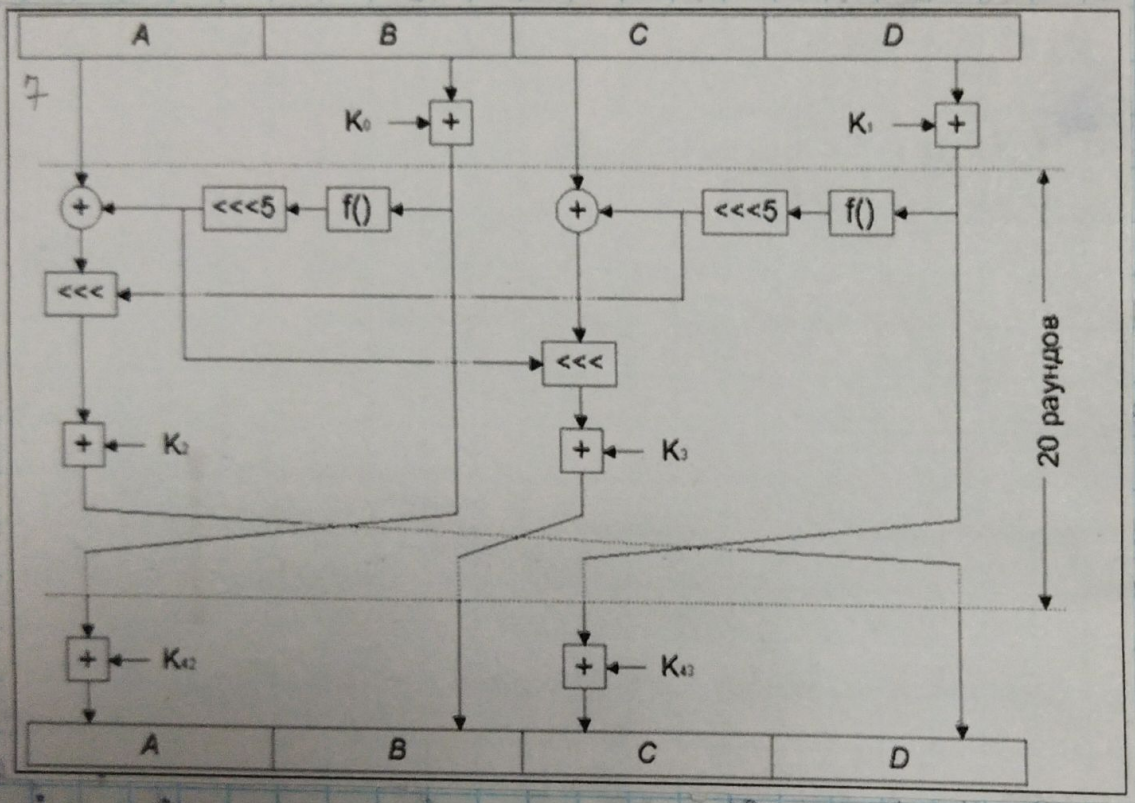
Как и алгоритм RC5, RC6 имеет гибкую структуру: помимо секретного ключа, параметрами алгоритма являются следующие:

- размер слова w; RC6 шифрует блоками по 4 слова.
- количество раундов алгоритма R;
- размер секретного ключа в байтах b;

Аналогично RC5, для уточнения параметров алгоритма, используемых в его конкретной реализации, применяется обозначение RC6-w/R/b. Поскольку в конкурсе AES 128-битный блок является обязательным, значение w фиксировано и равно 32. В спецификации алгоритма фиксируется также количество раундов: $R=20$. Поскольку конкурс AES предусматривает использование трех фиксированных размеров ключей, рассмотрим три следующих варианта алгоритма: RC6-32/20/16, RC6-32/20/24,

и RC6-32/20/32, совокупность которых и будет подразумеваться далее под названием "RC6".

Структура алгоритма RC6.



Шифрование и расшифрование

Процедура шифрования:

Вход:

- r - количество раундов (r = 20)
- w - разрядные ключи для каждого K[0, ..., 2r+3] (w = 32)

Выход:

- шифрованный текст сохраняется в A, B, C и D.

```
B = B + K[0]
D = D + K[1]
for i=1 to r do
{
    t = (B(2B+1)) <<< lgw
    u = (D(2D+1)) <<< lgw
    A = ((A ⊕ t) <<< u) + K[2i]
    C = ((C ⊕ u) <<< t) + K[2i+1]
    (A, B, C, D) = (B, C, D, A)
}
A = A + K[2r+2]
C = C + K[2r+3]
```

Процедура расшифровки:

Вход:

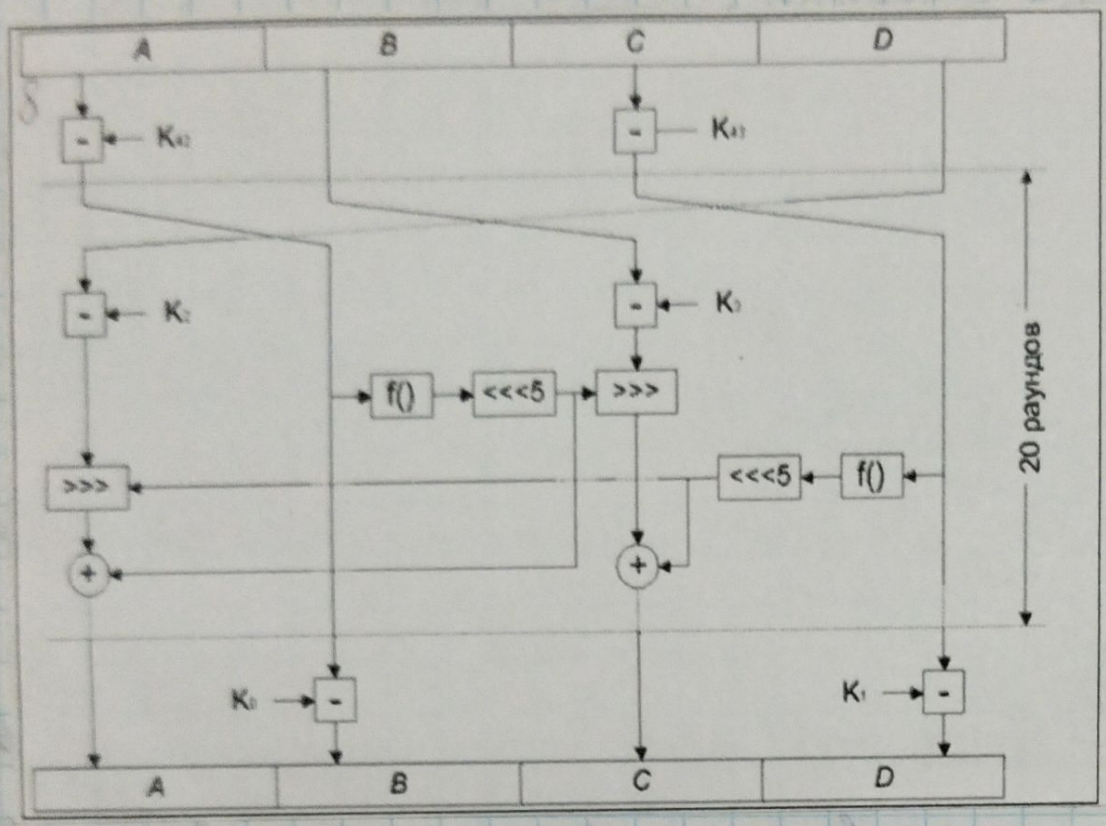
- шифрованный текст, сохранённый в A, B, C и D.
- r - количество раундов (r=20)
- w - разрядные ключи для каждого раунда K[0, ..., 2r+3]

Выход:

- исходный текст сохраняется в A, B, C и D.

```
C = C - K[2r+3]
A = A - K[2r+2]
for i=r downto 1 do
{
    (A, B, C, D) = (D, A, B, C)
    u = (D(2D+1)) <<< lgw
    t = (B(2B+1)) <<< lgw
    C = ((C - K[2i+1]) >>> t) ⊕ u
    A = ((A - K[2i]) >>> u) ⊕ t
}
```

$D = D - K[1]$
$B = B - K[0]$



Расширение ключа

Генерация констант:

Так же, как и в RC5, в RC6 генерируются две псевдослучайные величины, используя две математические константы: экспонента (e) и золотое сечение (f)

$$O_w \leftarrow Odd((f-1)\cdot 2^w)$$
$$P_w \leftarrow Odd((e-2)\cdot 2^w)$$

где Odd() - это округление до ближайшего нечетного целого. При $w=32$ бита

в шестнадцатеричном виде:

$Q_{32} = 9E3779B9$

$P_{32} = B7E15163$

в десятичном виде:

$P_{32} = 3084996963$

$Q_{32} = 2654435769$

Процедура расширения ключа для RC6-w/r/b

Вход:

• b – байтный ключ, заданный пользователем, предварительно преобразованный в массив из c слов $L[0, \dots, c-1]$. $(c=4)$

• r – количество раундов $(r=20)$

Выход:

• w – битная таблица ключей $K[0, \dots, 2r+3]$

$K[0] = P_w$
for $i=1$ to $2r+3$ do
    $K[i] = K[i-1] + Q_w$

$A = B = i = j = 0$

$v = 3 * \max\{c, 2r+4\}$

```
for s=1 to v do
{
    A = K[i] = (K[i]+A+B) <<< 3
    B = L[j] = (L[j]+A+B) <<< (A+B)
    i = (i+1) mod (2r+4)
    j = (j+1) mod c
}
```

Достоинства и недостатки алгоритмов-финалистов.

№	Категория	Serpent	Twofish	MARS	RC6	Rijndael
1	Криптостойкость	+	+	+	+	+
2	Запас криптостойкости	++	++	++	+	+
3	Скорость шифрования при программной реализации	-	±	±	+	+
4	Скорость расширения ключа при программной реализации	±	-	±	±	+
5	Смарт-карты с большим объемом ресурсов	+	+	-	±	++
6	Смарт-карты с ограниченным объемом ресурсов	±	+	-	±	++
7	Аппаратная реализация (ПЛИС)	+	+	-	±	+
8	Аппаратная реализация (специализированная микросхема)	+	±	-	-	+
9	Защита от атак по времени выполнения и потребляемой мощности	+	±	-	-	+
10	Защита от атак по потребляемой мощности на процедуру расширения ключа	±	±	±	±	-
11	Защита от атак по потребляемой мощности на реализации в смарт-картах	±	+	-	±	+
12	Возможность расширения ключа «на лету»	+	+	±	±	±
13	Наличие вариантов реализации (без потерь в совместимости)	+	+	±	±	+
14	Возможность параллельных вычислений	±	±	±	±	+

Криптостойкость всех алгоритмов-финалистов оказалось достаточной - в процессе исследований не было обнаружено каких-либо реально реализуемых атак на полноценные и полнораундовые версии алгоритмов. В данном случае криптоаналитики обычно исследуют варианты алгоритмов с усеченным числом раундов, либо с некоторыми внесенными изменениями, незначительными, но ослабляющими алгоритм.

10

Алгоритм	Достоинства	Недостатки
Serpent	1. Простая структура алгоритма облегчает его анализ с целью нахождения возможных уязвимостей. 2. Serpent эффективно реализуем аппаратно и в условиях ограниченных ресурсов. 3. Serpent легко модифицируется с целью защиты от атак по времени выполнения и потребляемой мощности (однако, за счет снижения скорости).	1. Является самым медленным из алгоритмов-финалистов в программных реализациях. 2. Процедуры зашифрования и расшифрования абсолютно различны, т.е. требуют раздельной реализации. 3. Распараллеливание вычислений при шифровании алгоритмом Serpent реализуемо с ограничениями.
Twofish	1. Twofish эффективно реализуем аппаратно и в условиях ограниченных ресурсов. 2. Зашифрование и расшифрование в алгоритме Twofish практически идентичны. 3. Является лучшим из алгоритмов-финалистов с точки зрения поддержки расширения ключа «на лету». 4. Несколько вариантов реализации позволяют оптимизировать алгоритм для конкретных применений.	1. Сложность структуры алгоритма затрудняет его анализ. 2. Сложная и медленная процедура расширения ключа. 3. Относительно сложно защищается от атак по времени выполнения и потребляемой мощности. 4. Распараллеливание вычислений при шифровании алгоритмом Twofish реализуемо с ограничениями.

11

MARS	Зашифрование и расшифрование в алгоритме MARS практически идентичны.	1. Исключительно сложная структура алгоритма с раундами различных типов затрудняет как анализ алгоритма, так и его реализацию. 2. Возникают проблемы при программной реализации на тех платформах, которые не поддерживают 32-битное умножение и вращение на переменное число бит. 3. Алгоритм MARS не может быть эффективно реализован аппаратно и в условиях ограниченных ресурсов. 4. Сложно защищается от атак по времени выполнения и потребляемой мощности. 5. MARS хуже других алгоритмов-финалистов поддерживает расширение ключей «на лету». 6. Распараллеливание вычислений при шифровании алгоритмом MARS реализуемо с ограничениями.

RC6	1. Простая структура алгоритма облегчает его анализ. Кроме того, алгоритм унаследовал часть преобразований от своего предшественника – алгоритма RC5, тщательно проанализированного до конкурса AES. 2. Самый быстрый из алгоритмов-финалистов на 32-битных платформах. 3. Зашифрование и расшифрование в алгоритме RC6 практически идентичны.	1. Скорость шифрования при программной реализации сильно зависит от того, поддерживает ли платформа 32-битное умножение и вращение на переменное число бит. 2. RC6 сложно реализуем аппаратно и в условиях ограниченных ресурсов. 3. Достаточно сложно защищается от атак по времени выполнения и потребляемой мощности. 4. Недостаточно полно поддерживает расширение ключей «на лету». 5. Распараллеливание вычислений при шифровании алгоритмом RC6 реализуемо с ограничениями.

Serpent

Создатель: Росс Андерсон, Эли Бихам, Ларс Кнудсен.

Создан: 1998

Опубликован: 1998

Размер ключа: 128/192/256 бит.

Размер блока: 128 бит

Число раундов: 32

Тип: SP-сеть.

При разработке Serpent (с латыни – „змея") использовался более консервативный подход к безопасности, нежели у других

финалистов AES, проектировщики шифра считали, что 16 раундов достаточно, чтобы противостоять известным видам криптоанализа, но увеличили число раундов до 32, чтобы алгоритм мог лучше противостоять ещё неизвестным методам криптоанализа.

Став финалистом конкурса AES, алгоритм Serpent в результате голосования занял 2 место.

Шифр Serpent не запатентован и является общественным достоянием.

Основные принципы.

Основным требованием к участникам конкурса AES было то, что алгоритм-претендент должен быть быстрее, чем 3DES, и предоставлять как минимум такой же уровень безопасности: он должен

иметь блок данных длиной 128 бит и ключ длиной 256-бит. 16-раундовый Serpent был бы таким же надежным, как 3DES, но в два раза быстрее. Однако, авторы посчитали, что для большей надежности стоит увеличить количество раундов до 32. Это сделало шифр таким же быстрым, как, и намного более надежным, чем 3DES.

Структура алгоритма

Serpent шифрует открытый текст P длиной 128 бит в шифротекст C длиной так же 128 бит за 32 раунда с помощью 33 подключей $K_0, \ldots, K_{32}$ длиной 128 бит. Для используемого ключа может принимать различные значения, но для конкретики зафиксируем их длину в 128, 192 или 256 бит. Короткие ключи длиной менее 256 бит дополняются до полной длины в 256 бит.

Шифрование состоит из следующих основных шагов:

- начальная перестановка
- 32 раунда, каждый из которых состоит из операций смешивания с 128-битным ключом (побитовое логическое исключающее "или"), табличная замена (S-box) и линейное преобразование. В последнем раунде линейное преобразование заменяется дополнительным наложением ключа;
- конечная перестановка.

Уравнения структуры алгоритма шифрования:

$$\begin{cases} \hat{B}_0 = IP(P) \\ \hat{B}_{i+1} = Re_i(\hat{B}_i) \\ C = FP(\hat{B}_{32}) \end{cases}$$

где

$$Re_i(X) = L(\hat{S}_i(X \oplus \hat{K}_i)), \quad i = 0, \ldots, 30$$

$$Re_i(X) = \hat{S}_i(X \oplus \hat{K}_i) \oplus \hat{K}_{32}, \quad i = 31$$

Уравнение структуры алгоритма расшифрования:

$$\begin{cases} \hat{B}_0 = IP(P) \\ \hat{B}_{i+1} = Rd_{31-i}(\hat{B}_i) \\ C = FP(\hat{B}_{32}) \end{cases}$$

где

$$Rd_i(X) = \hat{S}_i(X \oplus \hat{K}_{32}) \oplus \hat{K}_i, \quad i = 31$$

$$Rd_i(X) = \hat{S}_i(L(X)) \oplus \hat{K}_i, \quad i = 30, \ldots, 0$$

где Re, Rd – раундовые функции шифрования и расшифрования соответственно, $\hat{S}_i$ – это применение таблицы подстановки $\hat{S}_{i \bmod 8}$ 32 раз параллельно и L – линейное преобразование.

Расширение ключа

Как и другие алгоритмы, участвовавшие в конкурсе AES, Serpent имеет возможные длины ключа 128, 192 или 256 бит. „Неполный" ключ длиной меньше 256 бит дополняется по следующему правилу: прибавляется единичный бит справа, за ним следует столько нулевых битов, чтобы длина ключа стала равна 256 битам.

Алгоритм выбора подключей из ключа. Сначала при необходимости ключ допол-

няется до 256 бит и преобразуется в 33 подключа $K_0, \ldots, K_{32}$ длиной 128 бит следующими способами:

Исходный ключ представляется в виде 8 32-битных слов $w_{-8}, \ldots, w_{-1}$ для вычисления промежуточного ключа по правилу:

$$w_i = (w_{i-8} \oplus w_{i-5} \oplus w_{i-3} \oplus w_{i-1} \oplus \phi \oplus 1) \lll 11,$$

где ϕ – это дробная часть золотого сечения $\frac{\sqrt{5}+1}{2}$, или 0x9e3779b9 в шестнадцатеричной системе счисления, а $\lll$ – это циклический битовый сдвиг.

Раундовые ключи вычисляются из промежуточных ключей использованием таблиц подстановок следующими образом:

$$\{k_0, k_1, k_2, k_3\} = S_3(w_0, w_1, w_2, w_3)$$
$$\{k_4, k_5, k_6, k_7\} = S_2(w_4, w_5, w_6, w_7)$$
$$\{k_8, k_9, k_{10}, k_{11}\} = S_1(w_8, w_9, w_{10}, w_{11})$$
$$\{k_{12}, k_{13}, k_{14}, k_{15}\} = S_0(w_{12}, w_{13}, w_{14}, w_{15})$$
$$\{k_{16}, k_{17}, k_{18}, k_{19}\} = S_7(w_{16}, w_{17}, w_{18}, w_{19})$$
$$\ldots\ldots\ldots\ldots\ldots\ldots$$
$$\{k_{124}, k_{125}, k_{126}, k_{127}\} = S_4(w_{124}, w_{125}, w_{126}, w_{127})$$
$$\{k_{128}, k_{129}, k_{130}, k_{131}\} = S_3(w_{128}, w_{129}, w_{130}, w_{131})$$

Промежуточные 32-битные величины K_j перенумеровываются и получаются 128-битные подключи:

$$K_i = \{K_{4i}, K_{4i+1}, K_{4i+2}, K_{4i+3}\}.$$

Key Schedule in C++[edit]

```
#define FRAC 0x9e3779b9      //  fractional part of the golden ratio
#define ROTL(A, n) ((A) << n | (A) >> 32-n)

uint32_t key[8];          // key provided by user
uint32_t subkey[33][4]; // roundkeys
const uint8_t S[8][16] = {}; // S-boxes

/* key schedule: get prekeys */
void get_pre(uint32_t w[4*33], const uint32_t k[8]) {
    uint32_t x[4*33+8];
    for (int i = 0; i < 8; i++)
        x[i] = k[i];
    for (int i = 8; i < 140; i++) {
        x[i] = ROTL(x[i-8] ^ x[i-5] ^ x[i-3] ^ x[i-1] ^ FRAC ^ (i-8), 11);
        w[i-8] = x[i];
    }
}

/* key schedule: get subkeys */
void get_sk(const uint32_t w[4*33], uint32_t (*sk)[4]) {

        uint8_t i, p, j, s, k;

        for (i = 0; i < 33; i++) {
                p = 32 + 3 - i;
        for (j = 0; j < 4; j++)
            sk[i][j] = 0;
                for (k = 0; k < 32; k++) {
                        s = S[p % 8][((w[4 * i + 0] >> k) & 0x1) << 0 |
                                                ((w[4 * i + 1] >> k) & 0x1) << 1
|
                                                ((w[4 * i + 2] >> k) & 0x1) << 2
|
                                                ((w[4 * i + 3] >> k) & 0x1) << 3
];
                        for (j = 0; j < 4; j++) {
                                sk[i][j] |= ((s >> j) & 0x1) << k;
                        }
                }
        }
}

void key_schedule() {
    uint32_t w[4*33];
    get_prs(w, key);
    get_sk(w, subkey);
}
```

При стандартном описании алго-

ритма мы должны применить начальную перестановку IP к раундовому ключу, чтобы расположить биты ключа в надлежащем порядке, то есть

$$\hat{K}_i = IP(K_i)$$

Начальная перестановка IP

Initial permutation (IP)[edit]

The initial permutation works on 128 bits at a time moving bits around.

```
for i in 0 .. 127
    swap( bit(i), bit((32 * i) % 127) )
```

Начальная перестановка IP

0	32	64	96	1	33	65	97	2	34	66	98	3	35	67	99
4	36	68	100	5	37	69	101	6	38	70	102	7	39	71	103
8	40	72	104	9	41	73	105	10	42	74	106	11	43	75	107
12	44	76	108	13	45	77	109	14	46	78	110	15	47	79	111
16	48	80	112	17	49	81	113	18	50	82	114	19	51	83	115
20	52	84	116	21	53	85	117	22	54	86	118	23	55	87	119
24	56	88	120	25	57	89	121	26	58	90	122	27	59	91	123
28	60	92	124	29	61	93	125	30	62	94	126	31	63	95	127

S-боксы (таблицы замен)

В алгоритме Serpent таблицы замен являются 4-битовыми перестановками со свойствами стойкости к дифференциальному криптоанализу и тем свойствам,

что порядок выходных бит, как функции входных должен быть максимален, то есть быть равным 3.

Таблица подстановок S_i

S0:	3	8	15	1	10	6	5	11	14	13	4	2	7	0	9	12
S1:	15	12	2	7	9	0	5	10	1	11	14	8	6	13	3	4
S2:	8	6	7	9	3	12	10	15	13	1	14	4	0	11	5	2
S3:	0	15	11	8	12	9	6	3	13	1	2	4	10	7	5	14
S4:	1	15	8	3	12	0	11	6	2	5	4	10	9	14	7	13
S5:	15	5	2	11	4	10	9	12	0	3	14	8	13	6	7	1
S6:	7	2	12	5	8	4	6	11	14	9	1	15	13	3	10	0
S7:	1	13	15	0	14	8	2	11	7	4	12	10	9	3	5	6

Таблица инверсных подстановок S_i^{-1}

InvS0:	13	3	11	0	10	6	5	12	1	14	4	7	15	9	8	2
InvS1:	5	8	2	14	15	6	12	3	11	4	7	9	1	13	10	0
InvS2:	12	9	15	4	11	14	1	2	0	3	6	13	5	8	10	7
InvS3:	0	9	10	7	11	14	6	13	3	5	12	2	4	8	15	1
InvS4:	5	0	8	3	10	9	7	14	2	12	11	6	4	15	13	1
InvS5:	8	15	2	9	4	1	13	14	11	6	5	3	7	12	10	0
InvS6:	15	10	1	13	5	3	6	0	4	9	14	7	2	12	8	11
InvS7:	3	0	6	13	9	14	15	8	5	12	11	7	10	1	4	2

Линейное преобразование LT.

Таблица LT

{16 52 56 70 83 94 105} {72 114 125} {2 9 15 30 76 84 126} {36 90 103}

{12 48 52 66 79 90 101} ... {68 110 114} {5 11 26 80 122 126} {32 86 99}

} 32 qator (jami 128 ta element)

Таблица обратного L.T

{53 55 72} {1 5 20 90} {15 102} {3 31 90}

. . . .

{49 51 68} {1 16 86 97 125} {11 98} {4 27 86 97 113 115 127}

Конечная перестановка FP

$$FP = IP^{-1}$$

Final permutation (FP)[edit]

The final permutation works on 128 bits at a time moving bits around.

```
for i in 0 .. 127
    swap( bit(i), bit((4 * i) % 127) )
```

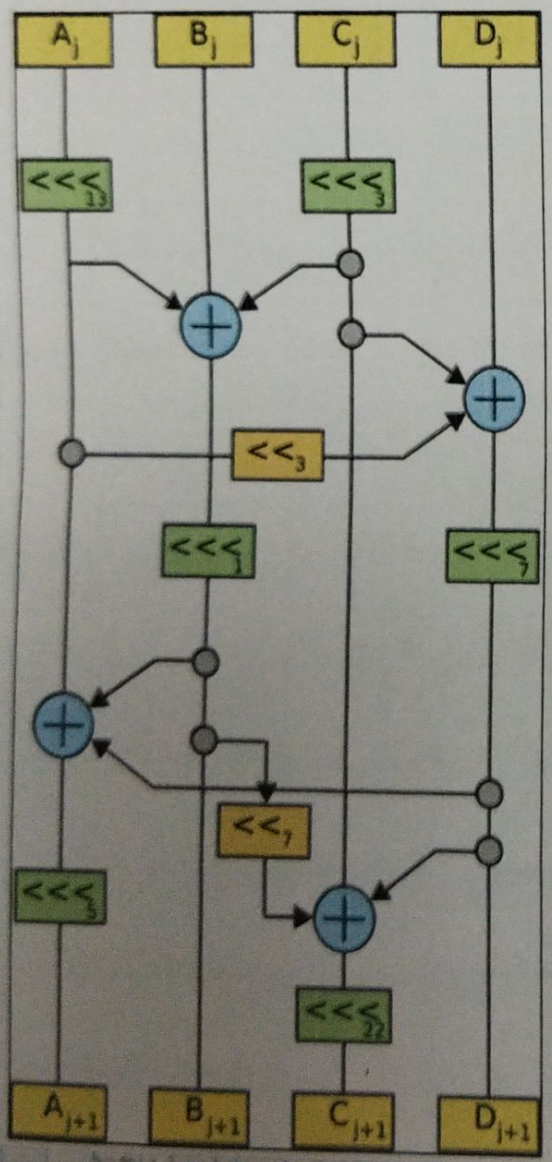
Конечная перестановка FP

0	4	8	12	16	20	24	28	32	36	40	44	48	52	56	60
64	68	72	76	80	84	88	92	96	100	104	108	112	116	120	124
1	5	9	13	17	21	25	29	33	37	41	45	49	53	57	61
65	69	73	77	81	85	89	93	97	101	105	109	113	117	121	125
2	6	10	14	18	22	26	30	34	38	42	46	50	54	58	62
66	70	74	78	82	86	90	94	98	102	106	110	114	118	122	126
3	7	11	15	19	23	27	31	35	39	43	47	51	55	59	63
67	71	75	79	83	87	91	95	99	103	107	111	115	119	123	127

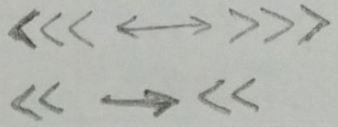
Linear transformation (LT)[edit]

Consists of XOR, S-Box, bit shift left and bit rotate left operations. These operations are performed on 4 32-bit words.

```
for (short i = 0; i < 4; i++) {
    X[i] = S[i][B[i] ^ K[i]];
}
X[0] = ROTL(X[0], 13);
X[2] = ROTL(X[2], 3 );
X[1] = X[1] ^ X[0] ^ X[2];
X[3] = X[3] ^ X[2] ^ (X[0] << 3);
X[1] = ROTL(X[1], 1 );
X[3] = ROTL(X[3], 7 );
X[0] = X[0] ^ X[1] ^ X[3];
X[2] = X[2] ^ X[3] ^ (X[1] << 7);
X[0] = ROTL(X[0], 5 );
X[2] = ROTL(X[2], 22);
for (short i = 0; i < 4; i++) {
    B[i + 1] = X[i];
}
```



Расшифрование



<<< ⟷ >>>

<< → <<

<<< – циклический битовый сдвиг

<< – битовый сдвиг.

Twofish

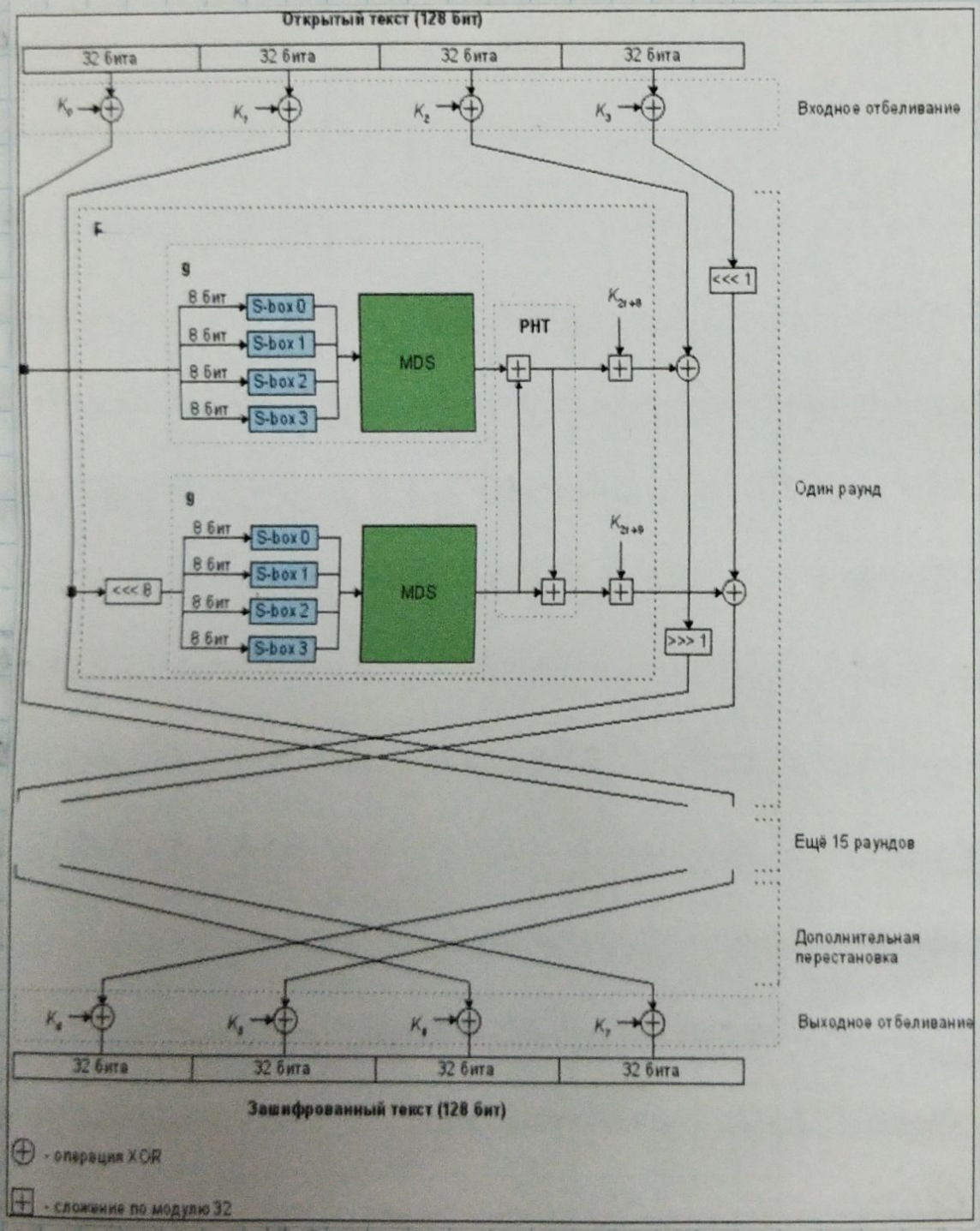
Создатель	группа специалистов во главе с Брюсом Шнайером.
Создан	1998
Размер ключа	128/192/256 бит
Размер блока	128 бит
Число раундов	16
Тип	Сеть Фейстеля.

Алгоритм разработан на основе алгоритмов Blowfish, SAFER, и SQUARE.

Отличительными особенностями алгоритма являются использование предварительно вычисляемых и зависящих от ключа узлов замены и сложная схема развёртки подключей шифрования. Половина n-битного ключа шифрования используется как собственно ключ шифрования, другая - для модификации

(от неё зависят узлы замены).

Алгоритм Twofish возник в результате попытки модифицировать алгоритм Blowfish для 128-битового входного блока.



В функции F при вычислении PHT (pseudo-Hadamar transform) и сложении с

частью ключа K намеренно используется слагаемое вместо традиционного XOR. Это дает возможность использовать команду LEA семейства процессоров Pentium, которая за один такт позволяет вычислить преобразование вида $(T_0 + 2T_1 + K_{2r+9}) \bmod 2^{32}$ (правда, в таком случае код приходится компилировать под конкретное значение ключа).

Алгоритм Twofish не запатентован и может быть использован как угодно без какой-либо платы или отчислений. Он используется во многих программах шифрования, хотя и получил меньшее распространение, чем Blowfish.

Описание алгоритма

1) Отбеливание (whitening).

Отбеливание – это процедура XOR'а данных с подключами перед первым

раундам и после последнего раунда. Впервые эта техника была использована в шифре Khufu/Khafre и независимо, Рональдом Ривестом (Ron Rivest) в алгоритме шифрования DESX. Joe Kilian (NEC) и Phillip Rodaway (Калифорнийский университет) показали, что отбеливание действительно усложняет задачу поиска ключа полным перебором (exhaustive key-search) в DESX. Разработчики Twofish утверждают, что в Twofish отбеливание также существенно усложняет задачу подбора ключа, поскольку криптоаналитик не может узнать, какие данные попадают на вход функции F первого раунда.

Тем не менее, обнаружились и негативные стороны. Интересное исследование провели специалисты, исследовательского центра компании IBM. Они выполнили

реализацию алгоритма Twofish для типичной смарт-карты с CMOS-архитектурой и проанализировали возможность атаки с помощью дифференциального анализа потребляемой мощности (DPA - Differential Power Analysis). Атаке подвергалось именно процедура входного отбеливания, поскольку она напрямую использует хэх подключей с входными данными. В результате исследователи показали, что можно полностью вычислить 128-битный ключ, проанализировав всего 100 операций шифрования произвольных блоков.

Функция g

Функция g - основа алгоритма Twofish. На вход функции подается 32-битное число X, которое затем разбивается на 4 байта x0, x1, x2, x3. Каждый

из получившихся байтов пропускается через свой S-box. (следует отметить, что S-box'ы в алгоритме не фиксированы, а зависят от ключа). Получившиеся 4 байта на выходах S-box'ов интерпретируются как вектор с четырьмя компонента. Этот вектор умножается на фиксированную матрицу MDS (maximum distance separable) размером 4x4, причём вычисления проводятся в поле Галуа $GF(2^8)$ по модулю неприводимого многочлена $x^8 + x^6 + x^5 + x^3 + 1$.

Свойство максимальной разнесённости (maximum distance separable)

$$b = (MDS)a$$

Любое изменение только одного байта в a приводит к изменению всех четырёх байтов в b.

$$MDS = \begin{pmatrix} 01 & EF & 5B & 5B \\ 5B & EF & EF & 01 \\ EF & 5B & 01 & EF \\ EF & 01 & EF & 5B \end{pmatrix}$$

Криптопреобразование Адамара (Pseudo-Hadamar Transform, PHT)

Криптопреобразование Адамара – обратимое преобразование битовой строки длиной $2n$. Строка разбивается на две части a и b одинаковой длины в n бит. Преобразование вычисляется следующим образом:

$$a' = a + b \pmod{2^n}$$
$$b' = a + 2b \pmod{2^n}$$

Это операция часто используется для „рассеивания" кода (например в шифре SAFER).

В Twofish это преобразование используется при смешивании результатов двух g-функций ($n = 32$)

Циклический сдвиг на 1 бит

В каждом раунде два правых 32-битных блока, которые хоч-ятся с

результатами функции F, дополнительно циклически сдвигаются на один бит. Третий блок сдвигается до операции хог, четвёртый блок – после. Эти сдвиги специально добавлены, чтобы нарушить выравнивание по байтам, которое свойственно S-box'ам и операции умножения на MDS-матрицу. Однако шифр перестаёт быть полностью симметричным, так как при шифрации и расшифровке сдвиги следует осуществлять в противоположные стороны.

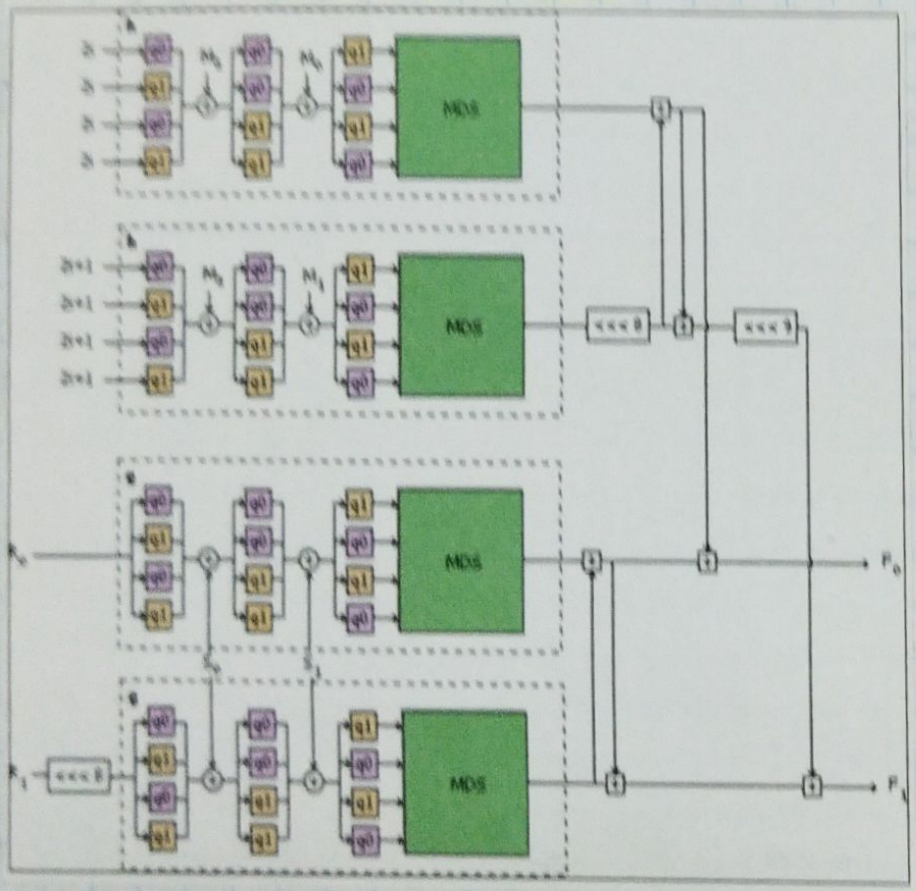
Генерация ключей.

Twofish рассчитан на работу с ключами длиной 128, 192, и 256 бит. Из исходного ключа генерируется 40 32-битных подключей, первые восемь из которых используются только в операциях входного и выходного отбеливания, а остальные

32 - в раундах шифрования, по два подключа на раунд. Особенностью Twofish является то, что исходный ключ используется также и для изменения самого алгоритма шифрования, так как используемые в функции g S-боксы не фиксированы, а зависят от ключа.

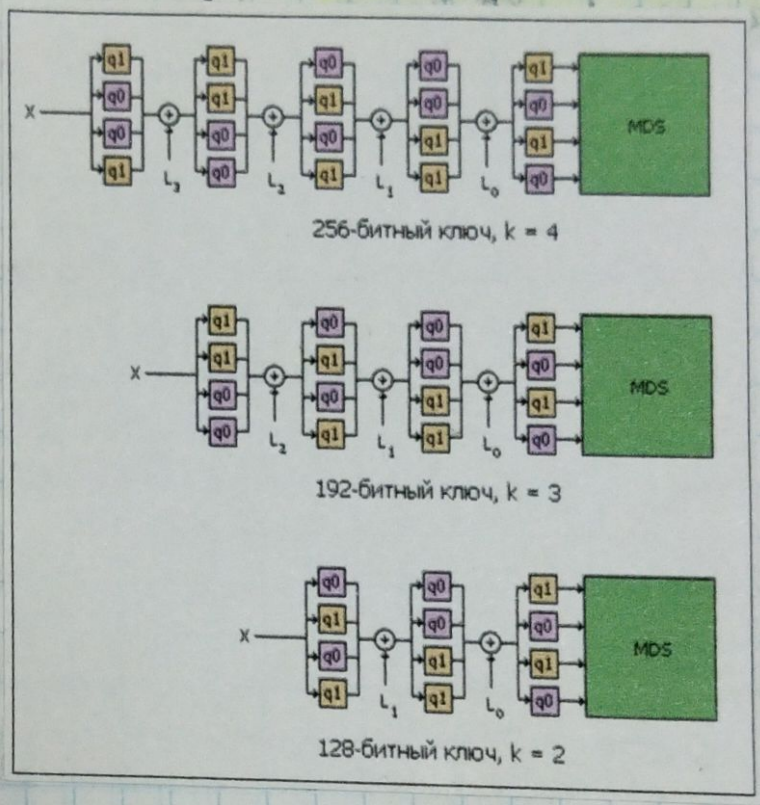
Для формирования раундовых подключей исходный ключ M разбивается с перестановкой байт на два одинаковых блока M_o и M_e. Затем с помощью блока M_o и функции h шифруется значение 2*i, а с помощью блока M_e шифруется значение 2*i + 1, где i - номер текущего раунда (0 - 15). Полученные зашифрованные блоки смешиваются криптопреобразованием Адамара, и затем используются в качестве раундовых подключей.

Схема одного раунда шифрования для 128-битного ключа.



Как для формирования подключей, так и для формирования функции g в Twofish используется одна основная функция: $h(X, L_0, L_1, \ldots, L_k)$. Здесь $X, L_0, L_1, \ldots, L_k$ – 32 битные слова, а $k = N/64$, где N – длина исходного ключа в битах. Результатом функции является одно 32-битное слово.

Функция h



На каждом этапе входное 32-битное слово разбивается на 4 байта, и каждый байт подвергается фиксированной перестановке битов q_0 и q_1.

Результат представляется в виде вектора из 4 байтов и умножается на MDS матрицу. Вычисления производятся в поле Галуа $GF(2^8)$ по модулю неприводимого многочлена $x^8+x^6+x^5+x^3+1$

Перестановки q_0 и q_1

q_0 и q_1 – фиксированные перестановки 8 битов входного байта x.

Байт x разбивается на две 4-битные половинки a_0 и b_0, над которыми проводятся следующие преобразование:

$a_0 = x/16$ $\quad b_0 = x \bmod 16$

$a_1 = a_0 \oplus b_0$ $\quad b_1 = a_0 \oplus ROR_4(b_0, 1) \oplus 8a_0 \bmod 16$

$a_2 = t_0[a_1]$ $\quad b_2 = t_1[b_1]$

$a_3 = a_2 \oplus b_2$ $\quad b_3 = a_2 \oplus ROR_4(b_2, 1) \oplus 8a_2 \bmod 16$

$a_4 = t_2[a_3]$ $\quad b_4 = t_3[b_3]$

$y = 16b_4 + a_4$

Здесь ROR_4 – 4 битный циклический сдвиг вправо, а t_0, t_1, t_2, t_3 – табличные замены 4 битных чисел.

Генерация ключей.

Пусть M – исходный ключ, N – его длина в битах. Генерация подключей происходит следующим образом:

– Исходный ключ разбивается на $8*k$ байтов $m_0, \ldots, m_{8k-1}$, где $k = N/64$.

– Эти $8*k$ байтов разбиваются на слова по четыре байта, и в каждом слове байты переставляются в обратном порядке. В итоге получается $2*k$ 32-битных слова M_i

$$M_i = \sum_{j=0}^{3} m_{(4i+j)} \cdot 2^{8j} \quad i = 0, \ldots, 2k-1$$

– Полученные $2*k$ 32-битных слов разбиваются на два вектора M_e и M_o размером в k 32-битных. слов.

$$M_e = (M_0, M_2, \ldots, M_{2k-2})$$
$$M_o = (M_1, M_1, \ldots, M_{2k-1})$$

Подключи для i-го раунда вычисляются по формулам:

$$P = 2^{24} + 2^{16} + 2^8 + 2^0$$
$$A_i = h(2iP, M_e)$$
$$B_i = ROL(h((2i+1)P, M_o), 8)$$
$$K_{2i} = (A_i + B_i) \bmod 2^{32}$$
$$K_{2i+1} = ROL((A_i + 2B_i) \bmod 2^{32}, 9)$$

Uzluksiz shifrlar. Psevdotasodifiy va tasodifiy sonlar generatorlari.

Bugungi kunda kriptografiyada tasodifiy sonlar quyidagi sohalarda keng qo'llaniladi:

1. SSHT kalitlarini generatsiya qilishda
2. SHA tegishli ish rejimlarida (IV).
3. ASHA da (shifrlash va ERI).
4. Turli protokollarda

Amalda psevdotasodifiy ketma-ketlik-lar generatorlari quyidagi yondashuvlar asosida quriladi:

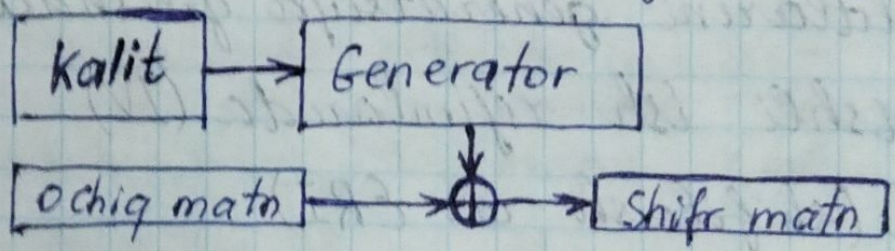
1. Fizikli - nazariy yondashuv
2. Murakkablikka asoslangan nazariy yondashuv asosida
3. Generatorlarni kombinatsiyalash.

Bir tomonlama funksiyalarga ANSI X9.17, FIPS-186, Yarrow-160 genera-torlari kiradi.

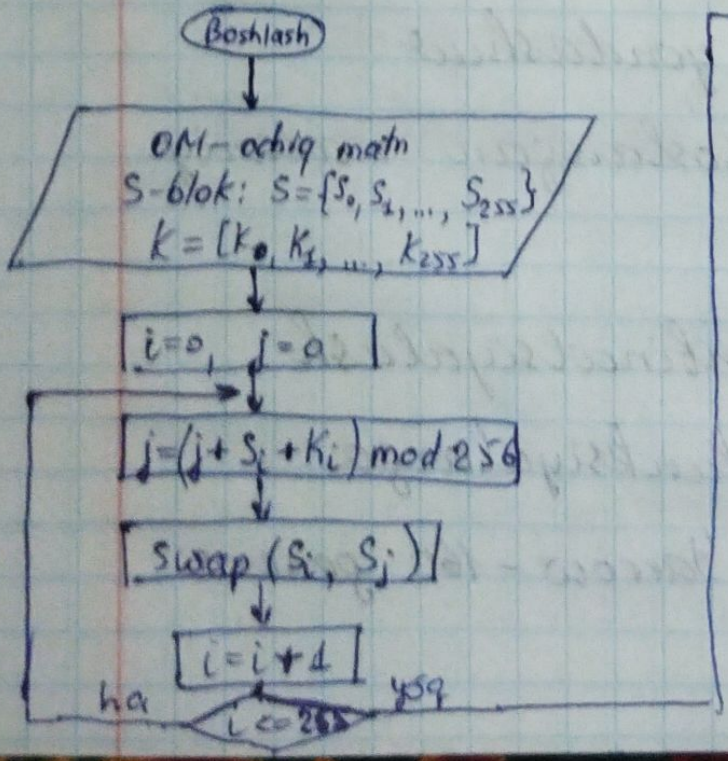
Uzluksiz shifrlar 2 xil:

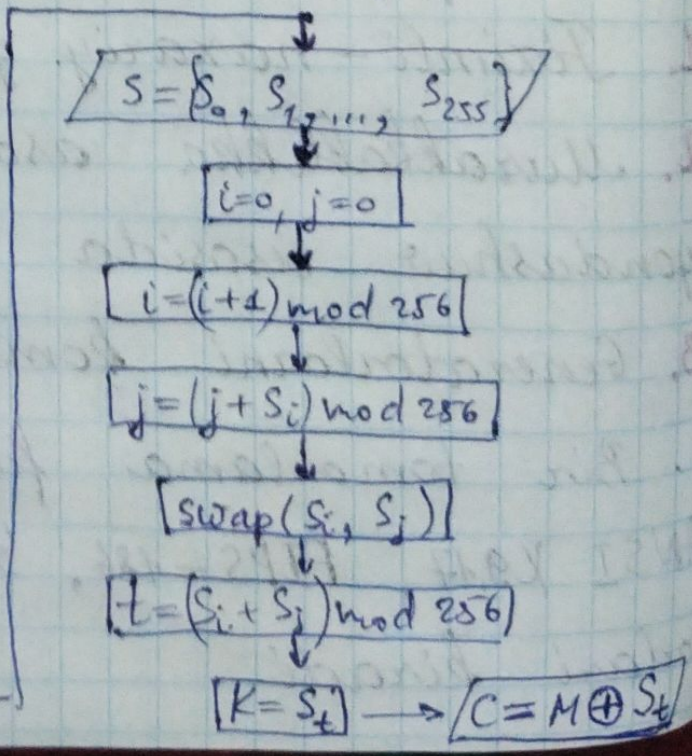
1. Sinxron
2. Asinxron

Sinxron uzluksiz shifrlarda generatsiya qilingan gamma ketma-ketlik shifrlanuvchi ochiq matnga bog'liq bo'lmaydi



Oz-o'zidan sinxronlanuvchi (asinxron) uzluksiz shifrlash tizimlarida qo'llanilayotgan generatorning ichki holati qabul qilinayotgan shifrmatnga bog'liq.





Uzluksiz shifrlash algoritmlarining yaratilish asoslari.

Uzluksiz shifrlash algoritmlari gammalashga asoslangan shifrlar bolib, ochiq matnning ketma-ket keluvchi har bir (p_i) 1 bit (1 bayt)ini generatordan chiqqan mos 1 bit (1 bayt) gamma kalit (k_i)ga XOR amali bilan qoshish orqali shifrmatn (c_i)ga aylantiradi.

$$c_i = p_i \oplus k_i$$

Deshifrlashda shifrmatnni xuddi shu generator yordamida faqat axborot almashinuvchi abonentlargagina ma'lum bolgan maxfiy simmetrik kalitdan foydalanib hosil qilingan mos 1 bit (1 bayt) gammaga qaytadan XOR amali bilan qoshish natijasida ochiq matn qayta tiklanadi.

$$c_i \oplus k_i = p \oplus k_i \oplus k_i = p_i$$

Psevdotasodifiy ketma-ketliklar generatorlari

Tizimli-nazariy yondashuv asosida

1. Elementar rekurrent generatorlar.
2. Siljitish registrlariga asoslangan generatorlar.
3. Bir tomonlama funksiyalarga asoslangan generatorlar.
4. Baytlar va bitlar bloklarining o'rnini bog'liqsiz almashtirishga asoslangan.

1. Elementar rekurrent generatorlar.

1.1. Multiplikativ

1.2. Chiziqli

1.3. Chiziqsiz

Misol: GOAL generatori.

2. Siljitish registrlariga asoslangan generatorlar:

2.1. Chiziqli teskari bog'lanishli

2.2. Chiziqsiz teskari bog'lanishli.

Misol: - Giffold generatori
- Fibonachi generatori
- Galua generatori.

3. Bir tomonlama funksiyalarga asoslangan generatorlar.

Misol: - ANSI X9.17.
- FIPS 186
- Yarrow - 160
- WAKE

4. Baytlar va bitlar bloklarining o'rnini bog'liqsiz almashtirishga asoslangan.

Misol: - RC4
- SEAL
- ISAAC

Murakkablikka asoslangan nazariy yondashuv.

1. Katta sonlarni tub ko'paytuvchilarga ajratish qiyinchiligiga asoslangan.

Misol: - RSA
- BBS
- Kvadrat chegirma generatori.

2. Diskret logarifmlash qiyinchiligiga asoslangan

Misol: - Blyum-Mikali algoritmi

Generatorlarni kombinatsiyalash asosida

1. Maklaren – Mersali usuli

Misol: – Chameleon algoritmi

2. Tasodifiy parametrli generatorlar

3. Polinomikal kombinatsiyalashgan generatorlar.

Misol:
- A5
- Yigiluvchi generator
- Gollmann kaskadi
- Chegaraviy generator
- "Start-stop" generatori
- Geffe generatori
- Additiv generator
- PIKE generatori
- ORYX generatori.

Sonlar nazariyasi muammolariga asoslangan samarali generatorlar.

- RSA algoritmi asosidagi generator.
- Mikali – Shnorr RSA algoritmi asosidagi generator
- BBS (Blum – blum – Shub) – algoritmi asosidagi generator.

Tarmoqning transport bosqichida xavfsiz ulanishni ta'minlash xalqaro protokollari tadqiqi.

TLS.

TLS (transport layer security) – Протокол защиты транспортного уровня, как и его предшественник SSL (secure sockets layer - слой защищённых сокетов), – криптографические протоколы, обеспечивающие защищённую передачу данных между узлами в сети Интернет. TLS и SSL используют асимметричное шифрование для аутентификации, симметричное шифрование для конфиденциальности и коды аутентичности сообщений для сохранения целостности сообщений.

Данный протокол широко используется в приложениях, работающих с сетью Интернет, таких как веб-браузеры, работа с электронной почтой, обмен мгновенными

сообщениями и IP-телефония (VoIP)

TLS – протокол основан на спецификации протокола SSL версии 3.0, разработанной компанией Netscape Communications. Сейчас развитием стандарта TLS занимается IETF. ~~Последнее обновление протокола было в RFC 5246~~

Dolzarbligi

Bugungi kunda xalqaro standart kriptografik algoritmlarni ozida mujassam etgan TLS (Transport Layer Security) protokollari internet brauzerlar, IP-telefoniya, messenjerlar hamda mobil ilovalar faoliyatida xavfsiz aloqa kanalini shakllantirish uchun xizmat qiladi.

Ushbu protokollar aktiv hujum imkoniyati mavjud bolgan aloqa kanali orqali uzatiluvchi ma'lumotlarning

maxfiyligi va yaxlitligini hamda tomonlarning autentifikatsiyasini ta'minlash uchun xalqaro kriptografik standartlar bilan belgilangan algoritmlardan foydalangan holda mijoz - server ilovalari o'rtasida xavfsiz tarmoq ulanishini o'rnatish uchun mo'ljallangan va davlat sirini tashkil etmaydigan axborotlarni himoya qilishda qo'llash uchun tavsiya qilingan.

TLS/SSL versiyalari

- SSL v1.0 (1994)
  - Netscape tomonidan 1994-yilda ishlab chiqilgan;
  - Hech qachon ommaga oshkor qilinmagan;
  - Ma'lumotlar yaxlitligini himoya qilmadi;
  - Xesh funksiyasi o'rniga oddiy siklik ortiqcha tekshiruvdan foydalandi;
  - Ochiq matnli xabarlarga o'zgartirishlar kiritish imkonini berdi.

- SSL v2.0 (1995)
  - Netscape tomonidan 1995-yilda ishlab chiqarilgan;
  - MD5 xesh funksiyasidan maxfiy prefiks bilan foydalangan, bu esa uni uzunlikni kengaytirish hujumiga bardoshsiz qilgan;
  - Xabarni autentifikatsiya qilish va shifrlash uchun bir xil kalit ishlatadi;

– MAC yaratish uchun faqat MD5 xesh funksiyasidan foydalanadi.
– 2011-yilda eskirgan.

SSL v3.0 (1996, RFC 6101)

– Netscape tomonidan 1996-yil noyabr oyida Internet loyihasi sifatida nashr etilgan;
– Biz bugun foydalanayotgan TLS versiyalarining asosi;
– Qoshimcha siqish qobiliyati qoshildi.
– Qoshimcha kalit almashinuvi uchun ixtiyoriy yordam qoshildi.
– SSL 1.0 va SSL 2.0 faqat RSA kalit almashinuvini qollab-quvvatlagan.
– SSL ning 2014-yilgacha bolgan asosiy versiyasi.
– SSL 3.0 da MD5 SHA-1 bilan toldirilади va MAC konstruksiyasi yanada murakkab;
– IETF tomonidan RFC 6101 da tarixiy hujjat sifatida nashr etilgan.
– 2014-yilda barcha blok shifrlariga ta'sir qiluvchi POODLE hujumiga qarshi zaif ekanligi aniqlandi.
– 2015-yil iyun oyida RFC 7568 tomonidan eskirgan.

TLS v1.0 – (RFC 2246, 1999)

– IETF tomonidan 1999-yil yanvar oyida RFC 2246 versiyasi bilan SSL 3.0 ning yangilanishi sifatida yaratildi.

– SSL ochiq standartga aylanganda, TLS nomiga ozgartirildi va protokol egaligi IETF ga otkazildi.

- SSL 3.0 bilan deyarli bir xil.
- HMAC-ni qo'llab-quvvatlash qo'shilgan
- Qo'shimcha kalit almashinuvi uchun zarur yordam;
- Asosiy zaiflik CBC shifrlariga hujum hisoblangan va ko'pgina zamonaviy brauzerlarda yumshatilgan.
- 2020-yilda eskirgan.

	Major version	Minor version
SSL v1.0	1	0
SSL v2.0	2	0
SSL v3.0	3	0
TLS v1.0	3	1
TLS v1.1	3	2
TLS v1.2	3	3
TLS v1.3	3	4

TLS v1.1. (2006, RFC 4346)

- IETF tomonidan 2006-yil aprel oyida RFC 4346 versiyasi bilan yaratilgan.
- Asosan TLS 1.0 ga o'xshaydi.
- 2020-yil yozida bekor qilinishi rejalashtirilgan lekin, covid-19 tufayli qoldirildi. va TLS 1.0 va TLS 1.1 2021-yil mart oyida bekor qilingan (RFC 8996) hozirda foydalanish tavsiya etilmaydi.
- TLS 1.0 va 1.1 versiyalari pasaytirish hujumlariga nisbatan zaifdir, chunki ular almashtirilgan xabarlarning yaxlitligi uchun SHA-1 xeshga tayanadi. Xatto qol sig'ishlaaning autentifikatsiyasi ham SHA-1 asosida amalga oshiriladi. bu esa tajovuzkorga MITM hujumlari uchun server nomini o'zgartirishni osonlashtiradi. TLS 1.1 yoki undan pastroq versiyalar

yangi protokollar bajaradigan mustahkamroq xeshlash algoritmlarini tanlash imkoniyatini bermaydi.

TLS. V1.2 – (RFC 5246, 2008).

IETF tomonidan 2008-yil avgust oyida RFC 5246 versiyasi bilan yaratildi. AEAD (Authenticated encryption with associated data) shifrlarini qollab quvvatlash joriy etildi.

TLS 1.1 ga asoslangan. Asosiy farqlarga quyidagilar kiradi.

– Psevdotasodifiy funksiyadagi (PRF) MD5 va SHA-1 kombinatsiyasi SHA-256 bilan almashtirildi, shifrlar toplamini korsatilgan PRF lardan foydalanish imkoniyati bilan;

– Yakunlangan xabar xeshidagi MD5 va SHA-1 kombinatsiyasi shifrlar toplamining maxsus xesh algoritmlaridan foydalanish imkoniyati bilan SHA-256 bilan almashtirildi. Biroq tugallangan xabardagi xesh hajmi hali ham kamida 96 bit bolishi kerak;

– Raqamli imzolangan elementdagi MD5 va SHA-1 kombinatsiyasi qol siqish paytida kelishilgan yagona xesh bilan almashtirildi, bu standard SHA-1 hisoblanadi;

– Asosan Galois/Counter Mode (GCM) va Advanced Encryption Standard (AES) shifrlashning CCM rejimi uchun ishlati

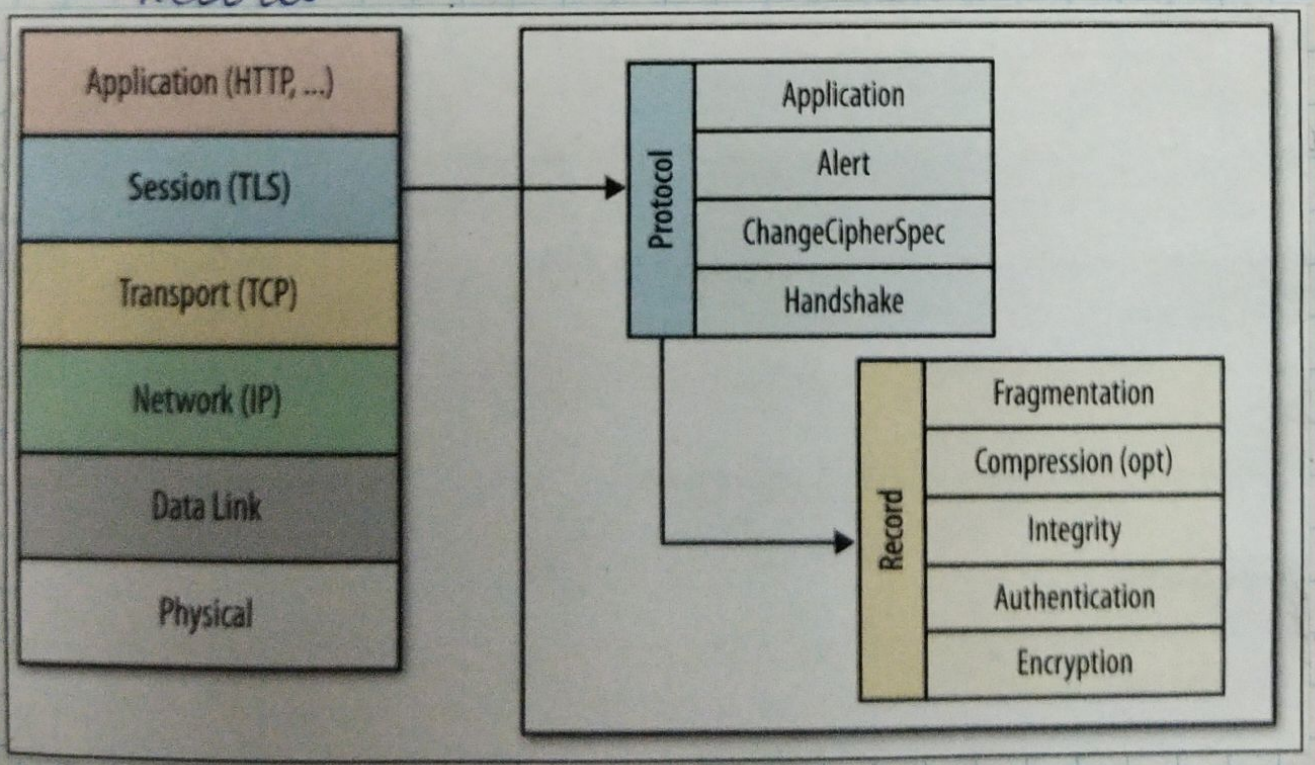
ladigan autentifikatsiya qilingan shifrlash shifrlarini qollab-quvatlashni kengaytirish;

- TLS kengaytmalari ta'rifi va AES shifr to'plami qoshildi.

TLS 1.2 protokoli

Protokol 2 ta yuqori va quyi qatlamlarni oz ichiga oladi.

- Yuqori qatlam:
  - Handshake
  - Alert
  - ChangeCipherSpec
  - Application Data
- Quyi qatlam:
  - Record

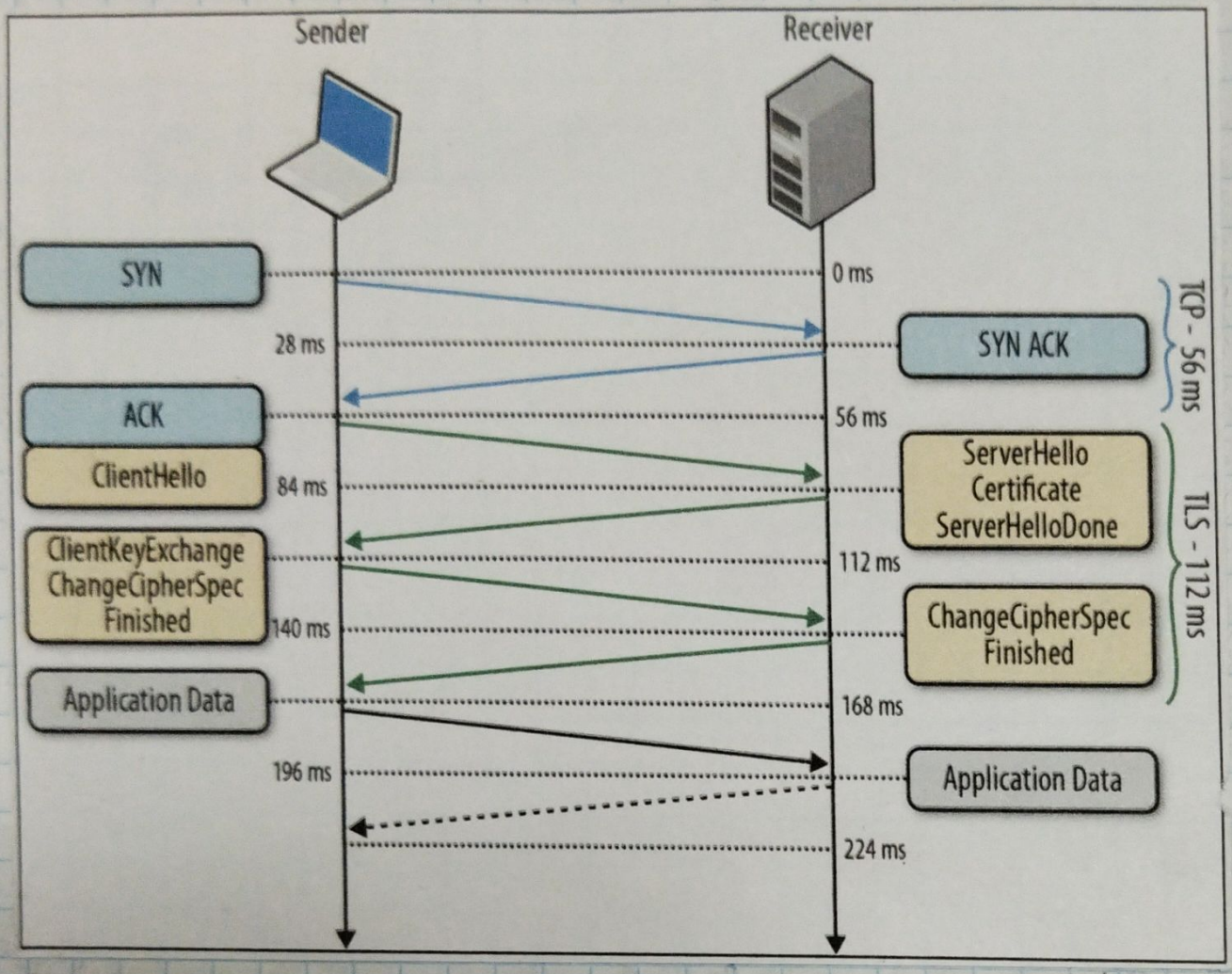


SSL and TLS protocols

Protocol	Published	Status
SSL 1.0	Unpublished	Unpublished
SSL 2.0	1995	Deprecated in 2011 (RFC 6176)
SSL 3.0	1996	Deprecated in 2015 (RFC 7568)
TLS 1.0	1999	Deprecated in 2021 (RFC 8996)[20][21][22]
TLS 1.1	2006	Deprecated in 2021 (RFC 8996)[20][21][22]
TLS 1.2	2008	In use since 2008[23][24]
TLS 1.3	2018	In use since 2018[24][25]

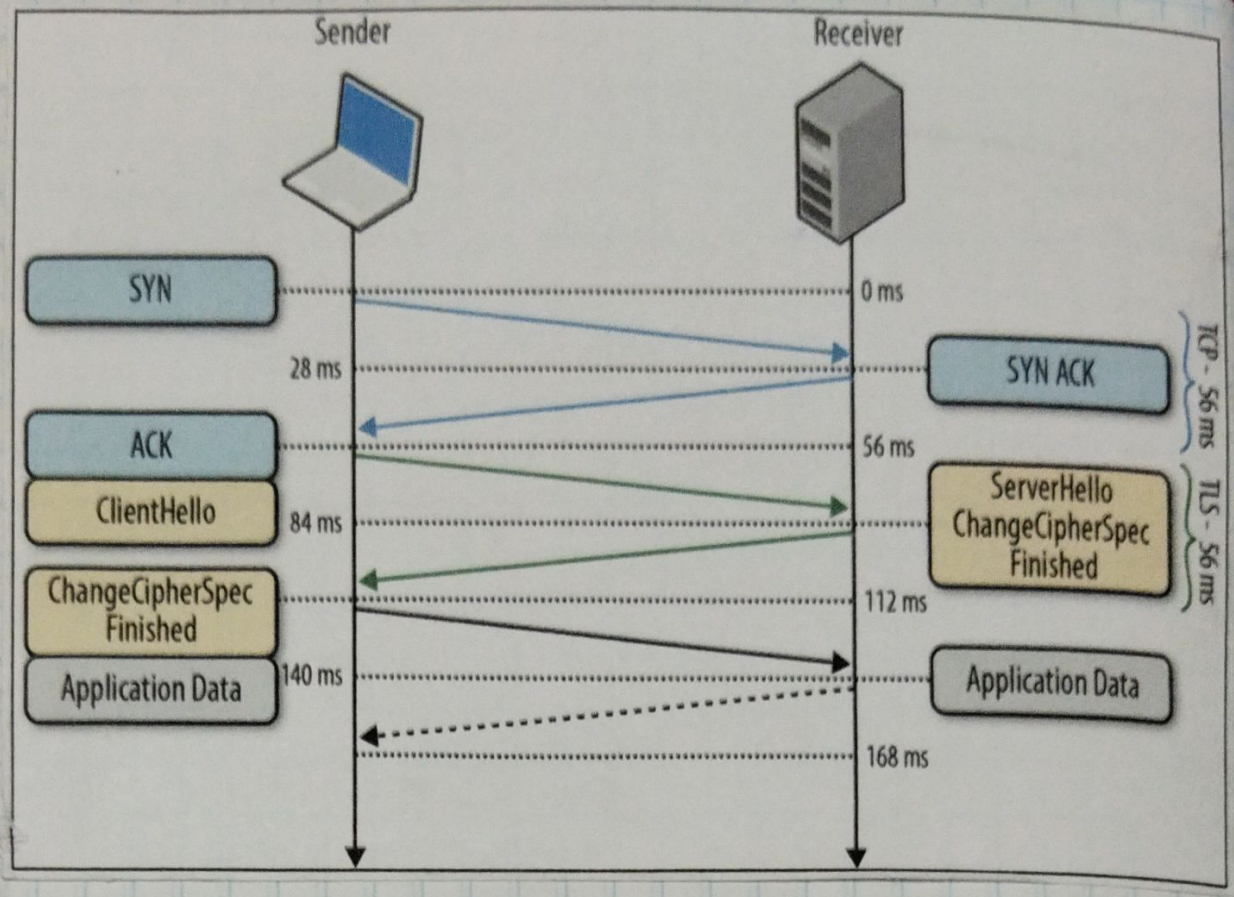
TLS Handshake

Прежде чем, как начать обмен данными через TLS, клиент и сервер должны согласовать параметры соединения а именно: версия используемого протокола способ шифрования данных, а также проверить сертификаты, если это необходимо. Схема начала соединения называется TLS Handshake и показано на рисунке:



Возобновление сессии TLS

Как уже отмечалось ранее, полная процедура TLS Handshake является довольно длительной и дорогой с точки зрения вычислительных затрат. Поэтому была разработана процедура, которая позволяет возобновить ранее прерванное соединение на основе уже сконфигурированных данных:

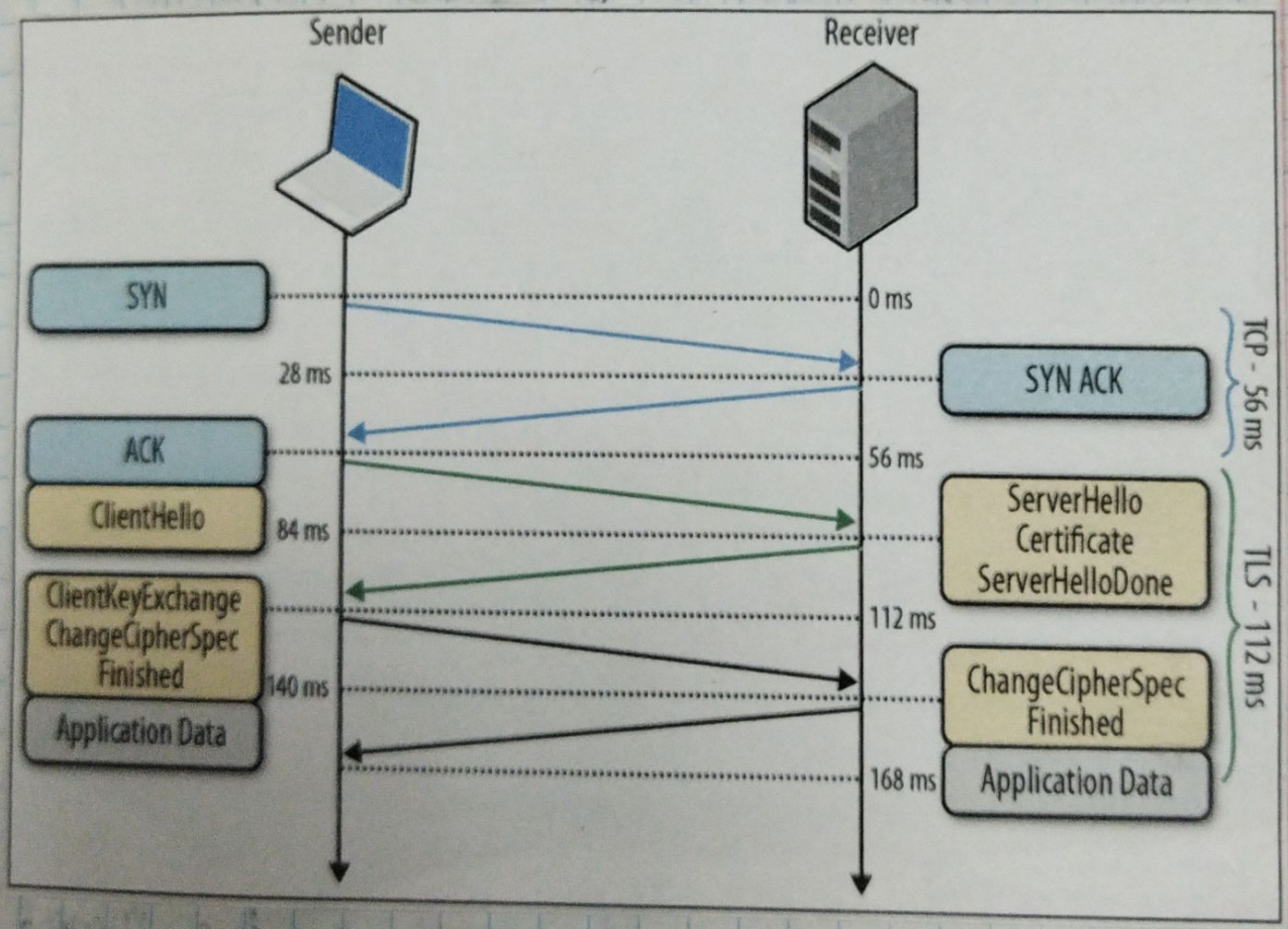


TLS False Start

Технология возобновления сессии бесспорно повышает производительность протокола и снижает вычислительные затраты, однако она не применима в первоначальном соединении с сервером, или в случае, когда предыдущая сессия уже истекла.

Для получения ещё большего быстродействия была разработана технология TLS False Start, являющаяся опциональ-

ным расширением протокола и позволяющая отправлять данные, когда TLS Handshake завершён лишь частично. Подробная схема TLS False Start представлена на рисунке:



A5 (алгоритм шифрования).

A5 – это поточный алгоритм шифрования, используемый для обеспечения конфиденциальности передаваемых данных между телефонов и базовой станцией в европейской системе мобильной цифровой связи GSM (Groupe Special Mobile).

Шифр основан на побитовом сложении по модулю два (булева операция „исключающее или") генерируемой псевдослучайной последовательности и шифруемой информации.

В A5 псевдослучайная последовательность реализуется на основе трёх линейных регистров сдвига с обратной связью. Регистры имеют длины 19, 22 и 23 бита соответственно.

Сдвигами управляет специальная схема, организующая на каждом шаге смещение как минимум двух регистров, что приводит к их неравномерному движению. Последовательность формируется путём операции „исключающее или" над выходными битами регистров.

Алгоритм A5 в настоящее время – это целое семейство шифров. A5 переименовали в **A5/1** и стали распространять как в Европе, так и в США. Для остальных стран (в том

числе и России) алгоритм модифицировали, значительно понизив криптостойкость шифра. A5/2 был специально разработан как экспортный вариант для стран, не входивших в Евросоюз. Криптостойкость A5/2 была понижена добавлением ещё одного регистра (17 бит), управляющего сдвигами остальных. В A5/0 шифрование отсутствует совсем. В настоящее время разработан также алгоритм A5/3, основанный на алгоритме Касуми и утверждённый для использования в сетях 3G.

Потоковое шифрование

В этом алгоритме каждому символу открытого текста соответствует символ шифротекста.

Таким образом, безопасность системы полностью зависит от свойств последовательности. В идеальном случае каждый бит гаммы – это независимая случайная величина, и сама последовательность является случайной. Такая схема была изобретена Вернамом в 1917 году и названа в его честь. Как доказал Клод Шеннон в 1949 году, это обеспечивает абсолютную криптостойкость. Но использование случайной последовательности означает передачу по защищённому каналу сообщения, равного по объёму открытому тексту, что значительно усложняет задачу и практически нигде не используется.

В реальных системах создаётся

ключ заданного размера, который без труда передаётся по закрытому каналу. Последовательност генерируется на его основе и является псевдослучайной. Большой класс поточных шифров (в том числе A5) составляют шифры, генератор псевдослучайной последовательности, которы основан на регистрах сдвига с линейной обратной связью.

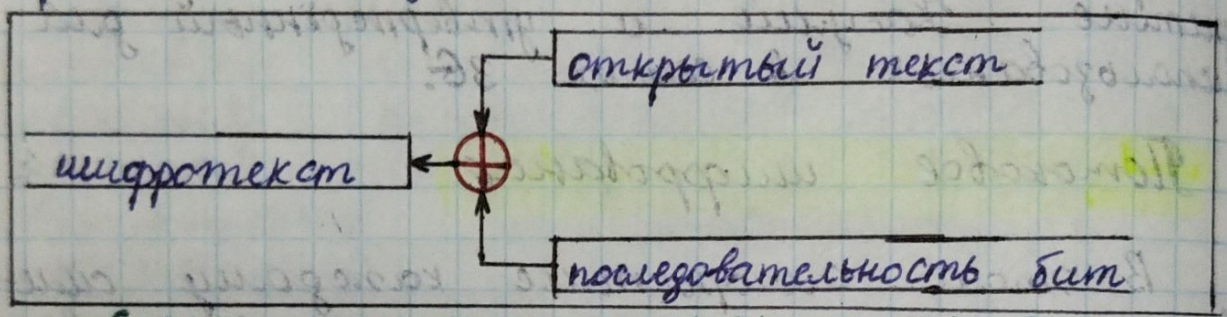
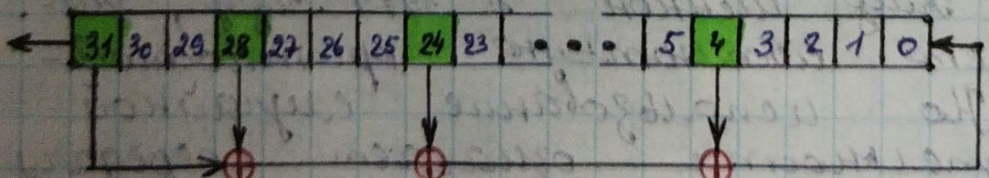


Схема поточного шифра: сложение открытого текста и последовательности бит даёт шифротекст.

РСЛОС

Регистр сдвига с линейной обратной связью состоит из собственно регистра (последовательности бит заданной длины) и обратной связи.



Регистр сдвига с линейной обратной связью, многочлен обратной связи $x^{32}+x^{29}+x^{25}+x^5+1$

На каждом такте происходит следующие действия:

1) крайний левый бит (старший бит)

извлекается, последовательность сдвигается

2) последовательность сдвигается влево и

3) в опустевшую правую ячейку (младший бит) записывается значение функции обратной связи.

Эта функция является суммированием по модулю два определённых битов регистра и записывается в виде многочлена, где степень указывает номер бита. Извлечённые биты формируют выходную последовательность.

Для РСЛОС основным показателем является период псевдослучайной последовательности. Он будет максимален (и равен $2^n - 1$), если многочлен функции обратной связи примитивен по модулю 2. Выходная последовательность в таком случае называется М-последовательностью.

Система РСЛОС в А5

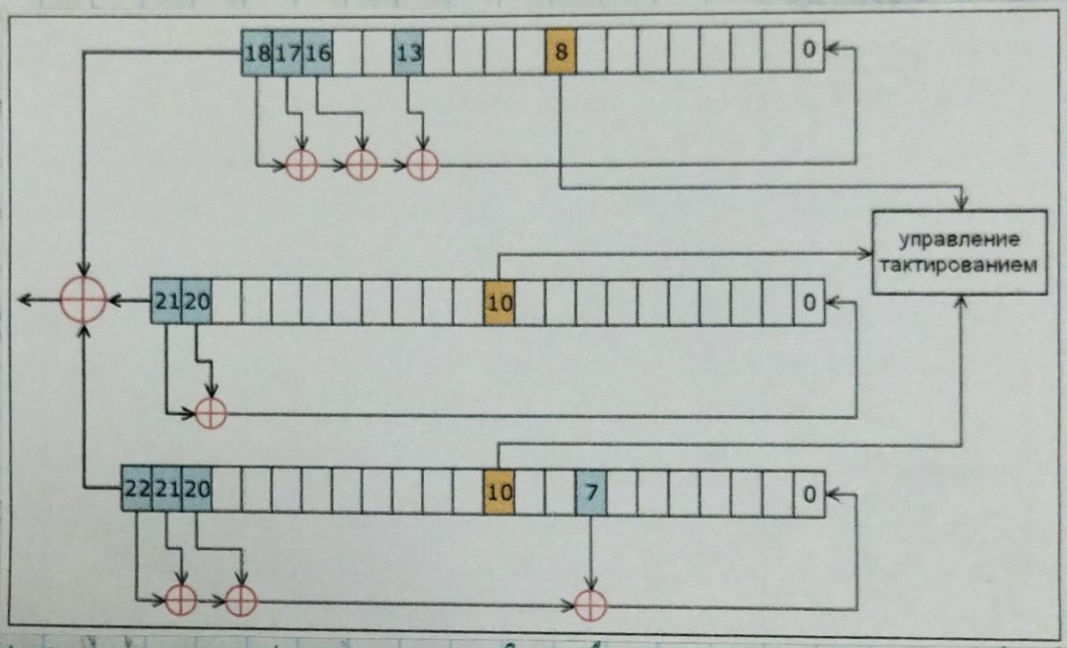
Сам по себе РСЛОС легко поддаётся криптоанализу и не является достаточно надёжным для использования в шифровании. Практическое применение имеют системы регистров переменного тактирования с различными длинами и функциями обратной связи.

Структура алгоритма А5 выглядит следующими образом:

- Три регистра (R1, R2, R3) имеют длины 19, 22 и 23 бита.

– Многочлены обратных связей:

- $x^{19} + x^{18} + x^{17} + x^{14} + 1$ для R1
- $x^{22} + x^{21} + 1$ для R2
- $x^{23} + x^{22} + x^{21} + x^8 + 1$ для R3.



Система регистров в алгоритме A5/1.

– Управление тактированием осуществляется специальными механизмами:

- в каждом регистре есть биты синхронизации:

  8(R1), 10(R2), 10(R3);

- вычисляется функция

  $F = x\&y \mid x\&z \mid y\&z,$

где & – булево AND, | – булево OR, а x, y, z – биты синхронизации R1, R2 и R3 соответственно.

- сдвигаются только те регистры, ~~синхробит~~ которые принадлежат

у которых бит синхронизации равен F.

• фактически, сдвигаются регистры, синхробит которых принадлежит большинству.

– Выходной бит системы – результат операции XOR над выходными битами регистров.

Функционирование алгоритма A5

Рассмотрим особенности функционирования алгоритма на основе известной схемы. Передача данных осуществляется в структурированном виде – с разбивкой на кадры (114 бит). Перед инициализацией регистры обнуляются, на вход алгоритма поступают сеансовый ключ (K – 64 бита), сформированный A8, и номер кадра (Fn – 22 бита). Далее последовательно выполняются следующие действия:

– Инициализация:

• 64 такта, при которых очередной бит ключа XOR-ится с младшими битам каждого регистра, регистры при этом сдвигаются на каждом такте,

• аналогичные 22 такта, только операция XOR производится с номерам кадра,

• 100 тактов с управлением сдвигами регистров, но без генерации последовательности

7 228 (114+114) тактов рабочие, происходит шифрование передаваемого кадра (первые 114 бит) и дешифрование (последние 114 бит) принимаемого

– далее инициализация производится заново, используется новый номер кадра.